



Preliminary Data Book

VISA 3000

High Performance Video Processor

Ver 1.1
Aug 9, 2002

Advanced Digital Chips Inc.

VISA3000
High Performance Video Processor
Data Book

© 2002 Advanced Digital Chips Inc.

All right reserved. No part of this document may be reproduced in any form without written permission from Advanced Digital Chips Inc.

Advanced Digital Chips Inc. reserves the right to change in its products or product specification to improve function or design at any time, without notice.

Office

4th Floor, Sam Kwang Bldg. 21-4, Samsung-Dong,
Kangnam-Ku, Seoul, 135-090 Korea

Tel : +82-2-545-4898
Fax : +82-2-545-4897
URL : <http://www.adc.co.kr>



Advanced Digital Chips Inc.

CONTENTS

1	DESCRIPTIONS AND FEATURES.....	8
1.1	GENERAL DESCRIPTION	8
1.2	FEATURES	8
2	BLOCK DIAGRAM & PIN DESCRIPTION	9
2.1	BLOCK DIAGRAM	9
2.2	PACKAGE DIAGRAM	10
2.3	PACKAGE DIMENSION	11
2.4	PIN INFORMATION	12
2.5	PIN DESCRIPTION	17
3	REGISTER DESCRIPTIONS.....	23
3.1	SYSTEM / GENERAL REGISTER	25
3.1.1	System ID Register (SYSID)	25
3.1.2	Configuration Register (CFGR)	25
3.1.3	System Control Register (SYSCON)	26
3.1.4	Watch Dog Timer Control Register (WDCON)	26
3.1.5	Watch Dog Timer Count Register (WDCNT)	26
3.1.6	Pin Mux Control Register 1 (PINMUX1)	27
3.1.7	Pin Mux Control Register 2 (PINMUX2)	28
3.2	LOCAL MEMORY CONTROL REGISTERS	29
3.2.1	Local ROM Control Register (LROMC)	29
3.2.2	Local DRAM Control Register (LDCON)	30
3.3	DMA	31
3.3.1	DMA0 Control Register (DMAC0)	31
3.3.2	DMA0 Source Address Register (DMAS0)	31
3.3.3	DMA0 Destination Address Register (DMADA0)	31
3.3.4	DMA0 Transfer Count Register (DMATC0)	31
3.3.5	DMA1 Control Register (DMAC1)	32
3.3.6	DMA1 Source Address Register (DMAS1)	32
3.3.7	DMA1 Destination Address Register (DMADA1)	32
3.3.8	DMA1 Transfer Count Register (DMATC1)	32
3.4	INTERRUPT CONTROLLER	33
3.4.1	Interrupt Mode Register (INTMOD)	33
3.4.2	Interrupt Vector Register (INTVEC)	33
3.4.3	Interrupt Enable Register (INTEN)	34
3.5	UART	35
3.5.1	UART Channel 0/1 Control Register (UCON0 / UCON1)	35
3.5.2	UART Channel 0/1 Status Register (USTAT0 / USTAT1)	36
3.5.3	UART Channel 0/1 Transmit Buffer Register (UTXB0 / UTXB1)	37
3.5.4	UART Channel 0/1 Receive Buffer Register (URXB0 / URXB1)	37
3.5.5	UART Channel 0/1 Baud Rate Divisor Register (UBDR0 / UBDR1)	37
3.6	TIMER	38
3.6.1	Timer Control Register	38
3.6.2	Timer Count Register	38
3.7	PULSE WIDTH MODULATION	39
3.7.1	PWM Control Register (PMCON)	39
3.7.2	PWM Duty Register (PMDTY)	39
3.7.3	PWM Period Register (PMPRD)	39
3.7.4	Pulse Count Register (PULCNT)	39
3.8	PULSE PERIOD MEASUREMENT	40
3.8.1	PPM 0 Control Register (PDCON0)	40
3.8.2	PPM 0 Pulse Width Register (PDPW0)	40

3.8.3	PPM 1 Control Register (PDCON1).....	41
3.8.4	PPM 1 Pulse Width Register (PDPW1).....	41
3.9	PIO.....	42
3.9.1	PIO Mode Register (PIOMOD).....	42
3.9.2	PIO Latched Output Data Register (PIOLDAT).....	43
3.9.3	PIO External Data Register (PIOEDAT).....	43
3.9.4	PIO Mode Register (PIOMOD2).....	43
3.9.5	PIO Latched Output Data Register2 (PIOLDAT2).....	44
3.9.6	PIO External Data Register2 (PIOEDAT2).....	44
3.10	PERIPHERAL DEVICE CHIP SELECT.....	45
3.10.1	PCS Control Register.....	45
3.11	VIDEO FRAME MEMORY CONTROL.....	46
3.11.1	Video Memory Configuration Register (VMCFG).....	46
3.11.2	Video Memory Control Register (VMCON).....	47
3.12	CRT CONTROLLER.....	48
3.12.1	CRTC Status / Mode Register (CRTMOD).....	48
3.12.2	CRTC Timing Control Register (CRTTIM).....	49
3.12.3	Horizontal Sync Width / Back Porch Register (HSWBP).....	49
3.12.4	Horizontal Display Total Register (HDISP).....	49
3.12.5	Horizontal Sync Front Porch Register (HSFP).....	50
3.12.6	Field Window Bound Register (FWINB).....	50
3.12.7	Vertical Sync Back Porch Register (VSBP).....	50
3.12.8	Vertical Display Total Register (VDISP).....	50
3.12.9	Horizontal Total Register (HTOT).....	50
3.12.10	Vertical Total Register (VTOT).....	50
3.13	GRAPHIC CONTROLLER.....	51
3.13.1	Video Page Base Address Register (VBASE).....	51
3.13.2	Horizontal Resolution Register (HRES).....	51
3.13.3	Raster Operation Value (ROPV).....	52
3.13.4	Pattern Data Register (PATD).....	52
3.13.5	Source Top-Left Coordinate Register (SRCXY).....	53
3.13.6	Destination Top-Left Coordinate Register (DSTXY).....	53
3.13.7	Window Size Register (WINSZ).....	53
3.13.8	Block Transfer Control Register (BITBLT).....	53
3.13.9	Image Sprite Control Register (SPRITE).....	54
3.13.10	Reserved for Test Mode.....	54
3.14	VIDEO CONTROLLER.....	56
3.14.1	Video Page Control Register.....	56
3.14.2	Video Page LUT Base Address Register.....	56
3.15	RAMDAC AND FREQUENCY SYNTHESIZER.....	57
3.15.1	RAMDAC & PLL Control Register (DPCON).....	57
3.15.2	PLL Program Register (PLPGM).....	57
3.15.3	Pixel Data Register (PIXEL).....	59
3.16	PALETTE MEMORY.....	59
3.16.1	Palette Memory Bank 0 (PAL0).....	59
3.16.2	Palette Memory Bank 1 (PAL1).....	59
3.17	SE3208 SYSTEM CO-PROCESSOR.....	60
3.17.1	General Descriptions.....	60
3.17.2	System Co-processor Register Descriptions.....	61
4	FUNCTION DESCRIPTION.....	63
4.1	DMA.....	63
4.1.1	Timing Diagram.....	63
4.1.2	DMA Mode.....	64
4.2	UART.....	65
4.2.1	Function Block Diagram.....	65
4.2.2	Timing Diagram.....	66
4.2.3	Baud generation.....	67
4.3	VIDEO SYNC와 CLOCK.....	68
4.3.1	Video Sync.....	68

4.3.2	Video Clock.....	72
4.3.3	Csync Generation.....	75
4.3.4	Horizontal Sync.....	78
4.3.5	Vertical Sync.....	80
4.3.6	CBF(Color Burst Frame) Generation.....	81
4.3.7	Csync로부터 Vsync/Hsync의 분리.....	82
4.3.8	External Sync Detector.....	83
4.4	GRAPHIC CONTROLLER.....	84
4.4.1	Block Overlapping.....	84
4.4.2	Sprite.....	86
4.4.3	Frame Memory Read Data Cache.....	86
4.5	VIDEO & PALETTE.....	88
4.5.1	Pixel Data Path.....	88
4.5.2	VPMC.....	91
4.5.3	Pre Pixel Data Path.....	92
4.5.4	Page Block.....	93
4.5.5	Post Pixel Data Path.....	96
5	ELECTRICAL DATA.....	100
5.1	ABSOLUTE MAXIMUM RATING.....	100
5.2	DC CHARACTERISTICS.....	100
5.3	AC CHARACTERISTICS.....	101
5.3.1	System Clock Input Waveform.....	101
5.3.2	Local Memory(EDO DRAM) Interface.....	102
5.3.3	Local Memory(SDRAM) Interface.....	106
5.3.4	Local Memory(SRAM) Interface.....	108
6	PROGRAM GUIDE.....	110
6.1	SYSTEM INITIALIZE & SOFTWARE FLOW.....	110
6.1.1	System Initialize Flow.....	110
6.2	DMA.....	128
6.3	UART.....	129
6.4	TIMER.....	130
6.5	PULSE WIDTH MODULATION (PWM).....	131
6.6	PPM.....	132
6.7	VIDEO.....	133
6.7.1	TV- 방식 setting.....	133
6.7.2	NTSC-local, 720 x 480, pg0-8bit color, make lut0, fill-with-white-color example.....	134
6.8	BITBLT & SPRITE PROGRAMMING.....	135
6.8.1	Bit Block Transfer.....	135
6.8.2	SPRITE.....	137
6.9	RAMDAC & PLL.....	138
7	외부 메모리 연결 방법.....	140
7.1	LOCAL / EXPANSION ROM.....	140
7.2	LOCAL SRAM.....	141
7.3	LOCAL EDO/FP DRAM.....	141
7.4	LOCAL SDRAM.....	142
7.5	FRAME MEMORY (EDO/FP DRAM).....	143
7.6	FRAME MEMORY (SDRAM).....	143
8	VIDEO APPLICATION.....	139
8.1	BA7230 APPLICATION.....	139
8.2	KA2198 APPLICATION.....	157

FIGURES

FIGURE 2.1-1 INTERNAL BLOCK DIAGRAM.....	9
FIGURE 2.2-1 PIN DIAGRAM.....	10
FIGURE 2.3-1 PACKAGE DIMENSION.....	11
FIGURE 4.1-1 DMA PRIORITY DECISION METHOD & OPERATION	63
FIGURE 4.1-2 TIMING DIAGRAM OF DMA CONTROL LOGIC	64
FIGURE 4.2-1 UART DATA FORMAT.....	65
FIGURE 4.2-2 FUNCTIONAL BLOCK DIAGRAM OF UART	65
FIGURE 4.2-3 SAMPLING TIMING OF RXD	66
FIGURE 4.2-4 TIMING DIAGRAM OF RECEIVER CONTROL LOGIC.....	66
FIGURE 4.2-5 TIMING DIAGRAM OF TRANSMITTER CONTROL LOGIC	66
FIGURE 4.3-1: GENERAL INTERFACE PLL MODE.....	68
FIGURE 4.3-2 RGB ENCODER. INTERFACE AT NO PLL.....	69
FIGURE 4.3-3 BA7230 TYPE INTERFACE No PLL MODE	69
FIGURE 4.3-4 MC1378 INTERFACE	70
FIGURE 4.3-5 BA7230 TYPE INTERFACE PLL MODE.....	71
FIGURE 4.3-6 VIDEO CLOCK.....	72
FIGURE 4.3-7 FOUR-FIELD (M) NTSC FORMAT AND BURST BLANKING	77
FIGURE 4.3-8 HSYNC LINE (NTSC).....	78
FIGURE 4.3-9 H-LINE.....	78
FIGURE 4.3-10 HSYNC AT LOCAL & REMOTE MODE.....	79
FIGURE 4.3-11 ODD FIELD VS. EVEN FIELD	79
FIGURE 4.3-12 FIELD DETECTION WINDOW.....	80
FIGURE 4.3-13 V-LINE STRUCTURE.....	80
FIGURE 4.3-14 CBF (COLOR BURST FRAME)	81
FIGURE 4.3-15 CBF AT H-LINE.....	82
FIGURE 4.3-16 CHECKING HSYNC FROM CSYNC.....	82
FIGURE 4.3-17 EXTERNAL VIDEO SIGNAL CHECK	83
FIGURE 4.4-1 SOURCE / DESTINATION IMAGE의 지정.....	84
FIGURE 4.4-2 BIT BLOCK TRANSFER에서의 SOURCE와 DESTINATION IMAGE OVERLAP시 수행	85
FIGURE 4.4-3 OVERLAY & BITBLT SIMULATION EXAMPLE	87
FIGURE 4.5-1 PIXEL DATA PATH	88
FIGURE 4.5-2 LUT AT H-LINE	90
FIGURE 4.5-3 TYPICAL VIDEO MEMORY MAP	91
FIGURE 4.5-4 PRE-PIXEL DATA PATH	92
FIGURE 4.5-5 LUT ADDRESS	92
FIGURE 4.5-6 PAGE BLOCK DIAGRAM.....	93
FIGURE 4.5-7 PAGE BLOCK TIMMING DIAGRAM.....	93
FIGURE 4.5-8 LINE BACK PORCH.....	94
FIGURE 4.5-9 POST PIXEL DATA PATH	96
FIGURE 6.1-1 SOFTWARE FLOW.....	110
FIGURE 6.1-2 VISA3000 TEST PROGRAM FLOW	111
FIGURE 6.1-3 CYGWIN ENVIROMENT	112
FIGURE 6.1-4 ALLGCC.SH	113
FIGURE 6.1-5 PROGRAM 구조 및 COMPILE 과정.....	114
FIGURE 6.1-6 SE3208.VCT.....	116
FIGURE 6.1-7 CRT0.S.....	118
FIGURE 6.1-8 SE3208.C	120
FIGURE 6.1-9 SE3208.H.....	122
FIGURE 6.1-10 TEST.C	124
FIGURE 6.1-11 LOCAL MEMORY (RAM) CONTROL CONFIGURATION	124
FIGURE 6.1-12 LOCAL ROM, EXPANSION ROM AND SYSTEM CONTROL CONFIGURATION	125
FIGURE 6.1-13 CACHE CONFIGURATION PROGRAM	126
FIGURE 6.1-14 VIDEO FRAME MEMORY CONFIGURATION.....	127
FIGURE 6.1-15 INTERRUPT VECTOR 생성 관련 PROGRAM.....	128
FIGURE 6.2-1 DMA TEST PROGRAM.....	129
FIGURE 6.3-1 UART TEST PROGRAM	130

FIGURE 6.4-1 TIMER TEST PROGRAM	131
FIGURE 6.5-1 PWM TEST PROGRAM	132
FIGURE 6.6-1 TIMER TEST PROGRAM	133

TABLES

TABLE 3-1 VISA3000 MEMORY MAP	23
TABLE 3-2 VISA3000 REGISTER OFFSET ADDRESS	23
TABLE 3-3 PLL(AL2005H)의 P,M,S 값에 따른 주파수 TABLE	58
TABLE 4.3-1 VIDEO STANDARD	76
TABLE 4.3-2 NTSC & PAL 설정	78
TABLE 4.3-3 STANDARD	81
TABLE 4.3-4 START OF CBF	81
TABLE 4.3-5 LENGTH OF CBF	81
TABLE 4.5-1 LUT DATA FORMAT	88
TABLE 4.5-2 LUT ADDRESS GENERATION	89
TABLE 4.5-3 PAGE0 COLOR MODE & PIXEL DATA STRUCTURE	95
TABLE 4.5-4 PAGE1 COLOR MODE & PIXEL DATA STRUCTURE	95
TABLE 4.5-5 PAGE2 COLOR MODE & PIXEL DATA STRUCTURE	95
TABLE 4.5-6 PAGE3 COLOR MODE & PIXEL DATA STRUCTURE	95
TABLE 4.5-7 PRIORITY PAGE	96
TABLE 4.5-8 CPU ADDRESS 와 PALETTE RAM DATA 연결 관계	97
TABLE 4.5-9 COLOR MODE & R,G,B DATA	98
TABLE 4.5-10 OVEN# DELAY SETTING BY DCLK	98
TABLE 4.5-11 OVEN# DELAY SETTING BY VCLK	98

1 DESCRIPTIONS AND FEATURES

1.1 General Description

ADC's VISA3000 32bit SEISC microcontroller is designed to provide a cost-effective and high performance micro controller solution for Video/Graphic processor. The VISA3000 integrated microprocessor combines a 32bit SEISC(SE3208) processor core with several peripheral functions such as timer, serial interface and etc. To speed program execution, the on-chip cache SRAM provide one-cycle access to code and data. VISA3000 supports video format for NTSC/PAL display monitor, it can be in local mode only for internal video image and remote mode for external video signal overlay. VISA3000 have four video pages and four internal buffer memory to display overlay images on the background.

SE3208 is the family in EISC series and is optimized for Embedded Application and the general purpose 32bit and high-performance microprocessor. For all of this, SEISC makes code-density higher as taking the excellence of CISC and employs Simple Instruction Set to simplify hardware.

VISA3000 can provide the cost-effective and high performance system solution such as karaoke, video editor, photo sticker and etc.

1.2 Features

Built in 32bit CPU

- High Performance EISC Core SE3208
- 4Kbyte Unified Cache
- Max Speed 40Mhz

Video Processing

- Support NTSC / PAL(B,D,G,H,I,N,M, Comb-N) Display Monitor
- Support Internal Video Display Mode(Local Mode) and External Video & Overlay Mode(Remote Mode)
- Supports External Sync Detection

Graphic Processing

- Maximum 1024 pixel / Horizontal line
- Support 24bit, 16bit, 8bit, 4bit, 2bit, 1bit Color Mode
- Support 4 Video Page, 4 Level Superimpose
- 3 operand (source, destination, pattern), 256 ROPs
- Support Memory to Screen and Screen to Screen Bit Block Transfer with ROP
- Display Control By LUT(Line Look Up Table)
- Horizontal Line Expanding up to 4 Times
- Maximum 16Mbyte DRAM for Video Frame Memory
- Support 16/32 bit Video Memory Bus
- Support EDO-DRAM, Sync DRAM

Memory management

- LOCAL RAM : SRAM, EDO-DRAM, Sync DRAM(16/32bit) support
- Frame Memory : SRAM, EDO-DRAM, Sync DRAM support(16bit/32bit).
- External Chip Selectors with flexible data bus & Wait

Peripheral functions

- 2 channel DMA
- 26 Ch. Priority Interrupt controller
- 4 Ch. 16 bit Counter for timer
- 2 Ch. Pulse Period Measurements
- 2 Ch. UART with 16 * 8 bit FIFO
- 48 PIO (Peripheral Input Output)
- PWM

Integration

- Embedded RAMDAC
- Embedded PLL

Process

- 0.35um CMOS VLSI
- 3.3 Volt Operation
- 208 Pin LQFP Package

2 BLOCK DIAGRAM & PIN DESCRIPTION

2.1 Block Diagram

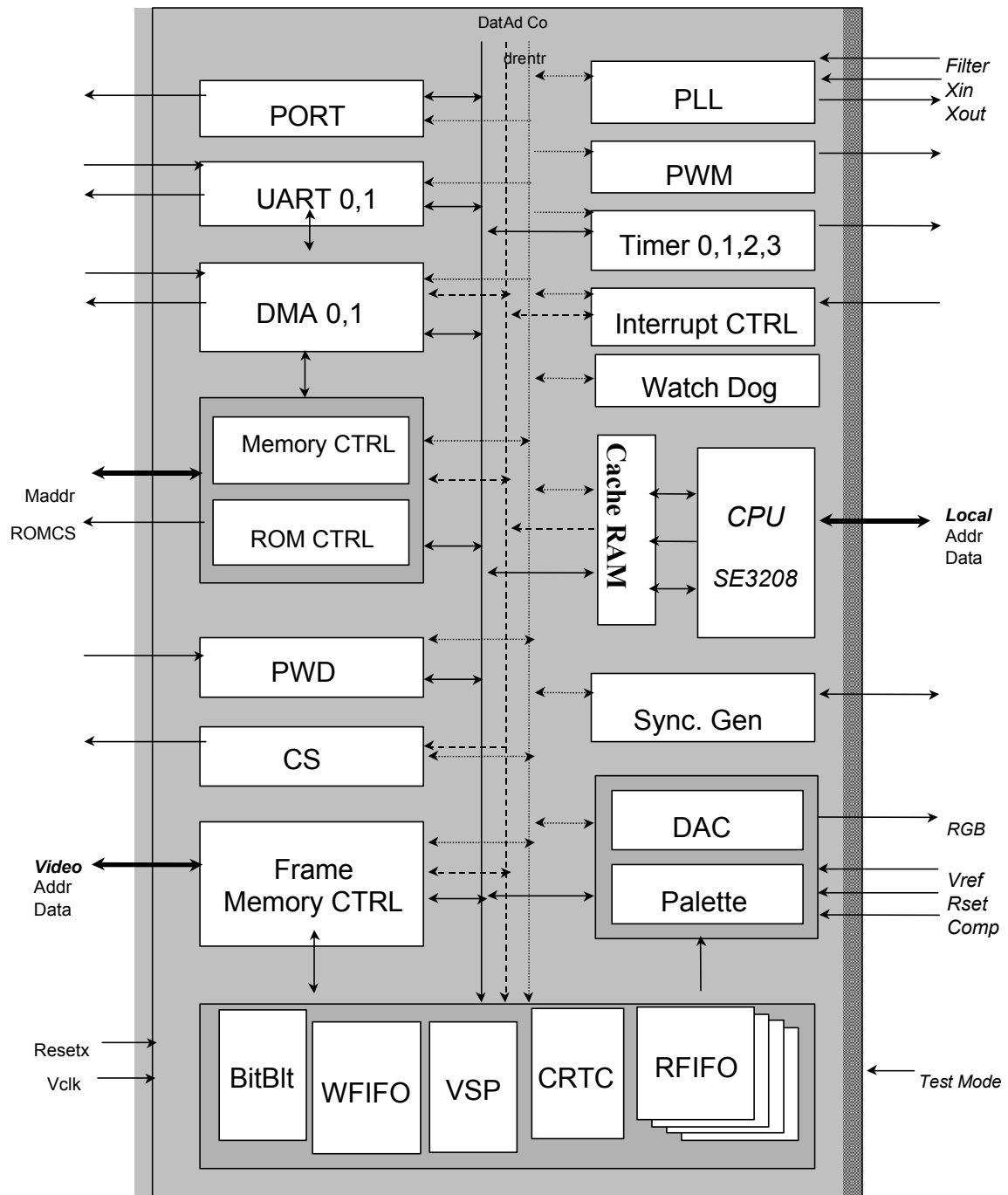


Figure 2.1-1 Internal Block Diagram

2.2 Package Diagram

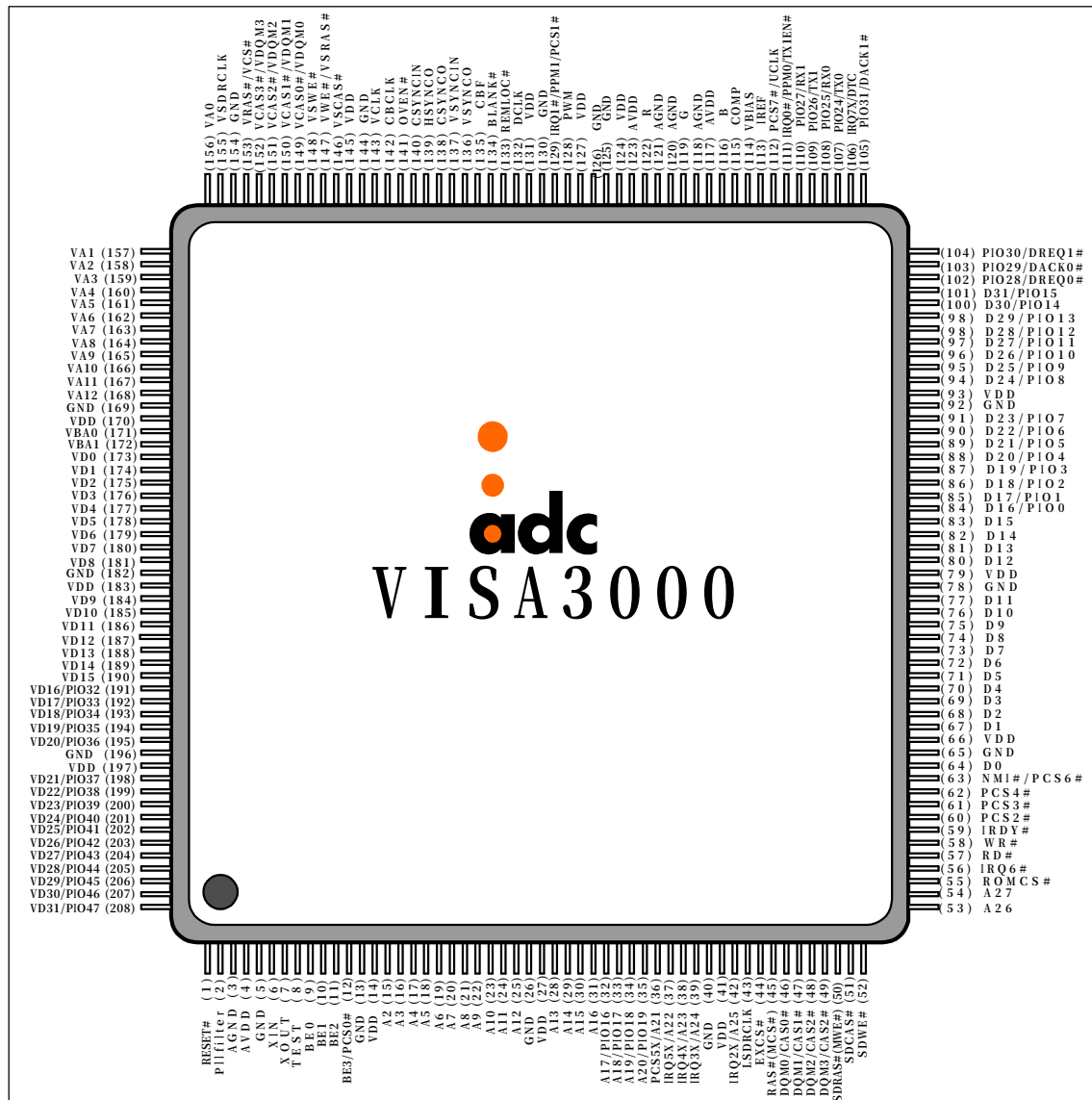


Figure 2.2-1 Pin Diagram

2.3 Package Dimension

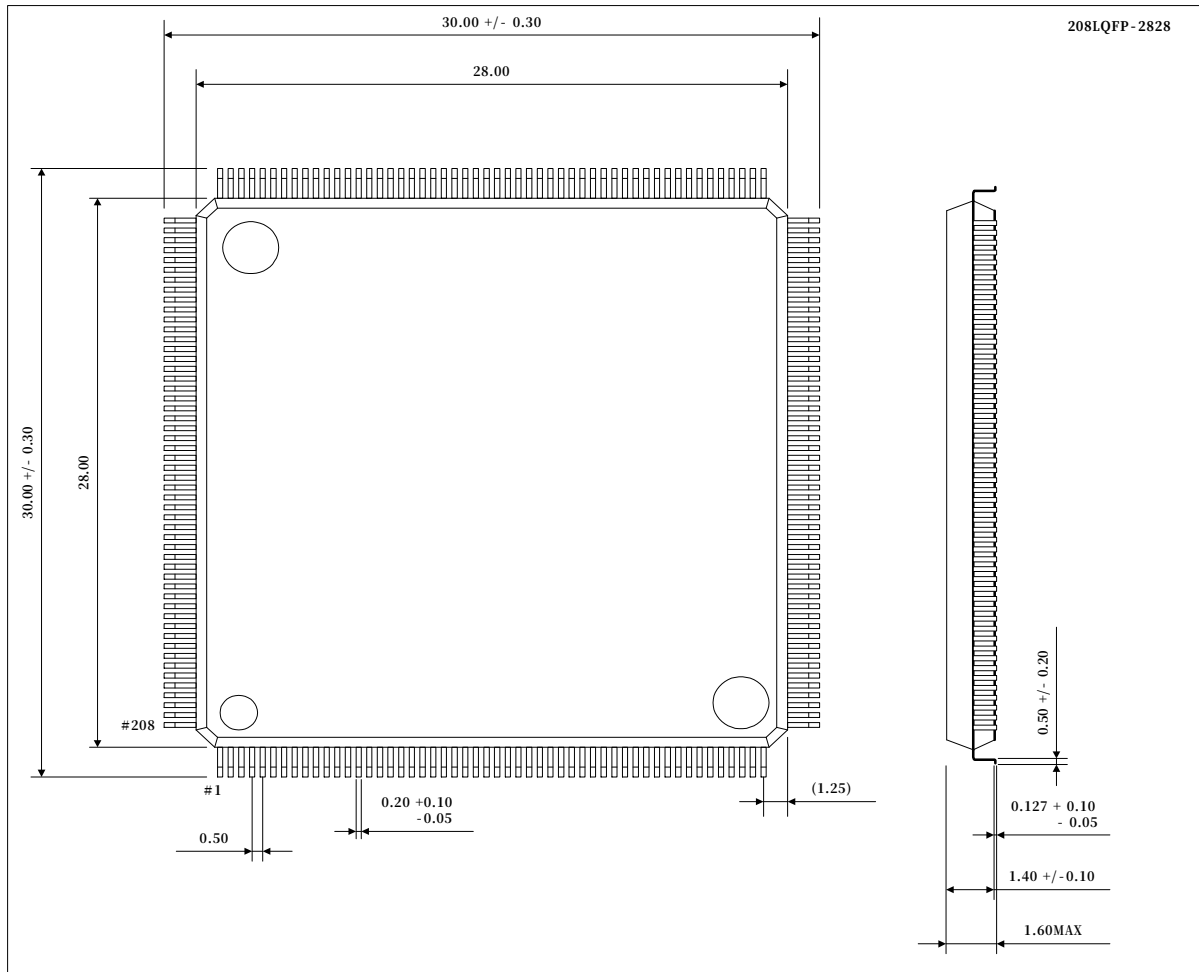


Figure 2.3-1 Package Dimension

2.4 Pin Information

I: Input pin O: Output pin B: Bi-directional pin

P: VDD pin G: VSS pin

CMOS: LVCMOS level, CMOS-S: LVCMOS level Schmitt trigger level, T: 5v-tolerant

Od : Open Drain type

Pkg pin No.	PIN Name	I/O	Entity name	Output drive current	Pull up/down	I/O interface
1	RESET#	I	PTIS	-	Null	CMOS-S, T
2	PLLFILTER	O	POAR50_BB	-	Null	Analog
3	AGND	G	VSSA,VBBA	-	-	-
4	AVDD	P	VDDA,VDDD	-	-	-
5	GND	G	VSSD	-	-	-
6	XIN	I	PSOSCM2	-	Null	Oscillator Pad
7	XOUT	O	PSOSCM2	-	Null	Oscillator Pad
8	TEST	I	PTIC	-	Null	CMOS, T
9	BEX[0]	O	POB4	4mA	Null	CMOS
10	BEX[1]	O	POB4	4mA	Null	CMOS
11	BEX[2]	O	POB4	4mA	Null	CMOS
12	BEX[3]/PCS0#	O	POB4	4mA	Null	CMOS
13	GND	G	VSSO	-	-	-
14	VDD	P	VDD3O	-	-	-
15	A[2]/RA[0]	O	POB4	4mA	Null	CMOS
16	A[3]/RA[1]	O	POB4	4mA	Null	CMOS
17	A[4]/RA[2]	O	POB4	4mA	Null	CMOS
18	A[5]/RA[3]	O	POB4	4mA	Null	CMOS
19	A[6]/RA[4]	O	POB4	4mA	Null	CMOS
20	A[7]/RA[5]	O	POB4	4mA	Null	CMOS
21	A[8]/RA[6]	O	POB4	4mA	Null	CMOS
22	A[9]/RA[7]	O	POB4	4mA	Null	CMOS
23	A[10]/RA[8]	O	POB4	4mA	Null	CMOS
24	A[11]/RA[9]	O	POB4	4mA	Null	CMOS
25	A[12]/RA[10]	O	POB4	4mA	Null	CMOS
26	GND	G	VSSP	-	-	-
27	VDD	P	VDD3P	-	-	-
28	A[13]/RA[11]	O	POB4	4mA	Null	CMOS
29	A[14]/RA[12]	O	POB4	4mA	Null	CMOS
30	A[15]/BA[0]	O	POB4	4mA	Null	CMOS
31	A[16]/BA[1]	O	POB4	4mA	Null	CMOS
32	A[17]/PIO16	B	PTBCT4	4mA	Null	CMOS,T
33	A[18]/PIO17	B	PTBCT4	4mA	Null	CMOS,T
34	A[19]/PIO18	B	PTBCT4	4mA	Null	CMOS,T
35	A[20]/PIO19	B	PTBCT4	4mA	Null	CMOS,T
36	PCS5#/A[21]/PIO20	B	PTBCT4	4mA	Null	CMOS,T
37	IRQ5#/A[22]/PIO21	B	PTBCUT4	4mA	UP	CMOS,T
38	IRQ4#/A[23]/PIO22	B	PTBCUT4	4mA	UP	CMOS,T
39	IRQ3#/A[24]/PIO23	B	PTBCUT4	4mA	UP	CMOS,T
40	GND	G	VSSO	-	-	-
41	VDD	P	VDD3O	-	-	-
42	IRQ2#/A[25]	B	PTBCUT4	4mA	UP	CMOS,T
43	LSDRCLK	O	POB4	4mA	Null	CMOS
44	EXCS#	O	POB4	4mA	Null	CMOS
45	MCS#(RAS#)	B	PBCT4	4mA	Null	CMOS
46	DQM0(SBE0#)/CAS0#/A[28]	O	POB4	4mA	Null	CMOS
47	DQM1(SBE1#)/CAS1#/A[29]	O	POB4	4mA	Null	CMOS
48	DQM2(SBE2#)/CAS2#/A[30]	O	POB4	4mA	Null	CMOS
49	DQM3(SBE3#)/CAS3#/A[31]	O	POB4	4mA	Null	CMOS
50	SDRAS#(MWE#)HOLD#	B	PBCT4	4mA	Null	CMOS
51	SDCAS#/HLDA#	O	POB4	4mA	Null	CMOS
52	SDWE#/INT#	B	PBCT4	4mA	Null	CMOS

53	A[26]/RAS0	B	PBCT4	4mA	Null	CMOS
54	A[27]	O	POB4	4mA	Null	CMOS
55	ROMCS#	O	POB4	4mA	Null	CMOS
56	IRQ6#/INTAX	B	PTBCUT4	4mA	UP	CMOS,T
57	RD#	O	POB4	4mA	Null	CMOS
58	WR#	O	POB4	4mA	Null	CMOS
59	IRDY#	I	PTICU	-	UP	CMOS,T
60	PCS2#/STAT[0]	O	POB4	4mA	Null	CMOS
61	PCS3#/STAT[1]	O	POB4	4mA	Null	CMOS
62	PCS4#/HALT#	O	POB4	4mA	Null	CMOS
63	PCS6#/NMI#	B	PBCT4	4mA	Null	CMOS
64	D[0]	B	PTBCUT4	4mA	UP	CMOS,T
65	GND	G	VSSIP	-	-	-
66	VDD	P	VDD3IP	-	-	-
67	D[1]	B	PTBCUT4	4mA	UP	CMOS,T
68	D[2]	B	PTBCUT4	4mA	UP	CMOS,T
69	D[3]	B	PTBCUT4	4mA	UP	CMOS,T
70	D[4]	B	PTBCUT4	4mA	UP	CMOS,T
71	D[5]	B	PTBCUT4	4mA	UP	CMOS,T
72	D[6]	B	PTBCUT4	4mA	UP	CMOS,T
73	D[7]	B	PTBCUT4	4mA	UP	CMOS,T
74	D[8]	B	PTBCUT4	4mA	UP	CMOS,T
75	D[9]	B	PTBCUT4	4mA	UP	CMOS,T
76	D[10]	B	PTBCUT4	4mA	UP	CMOS,T
77	D[11]	B	PTBCUT4	4mA	UP	CMOS,T
78	GND	G	VSSI	-	-	-
79	VDD	P	VDD3I	-	-	-
80	D[12]	B	PTBCUT4	4mA	UP	CMOS,T
81	D[13]	B	PTBCUT4	4mA	UP	CMOS,T
82	D[14]	B	PTBCUT4	4mA	UP	CMOS,T
83	D[15]	B	PTBCUT4	4mA	UP	CMOS,T
84	D[16]/PIO0	B	PTBCUT4	4mA	UP	CMOS,T
85	D[17]/PIO1	B	PTBCUT4	4mA	UP	CMOS,T
86	D[18]/PIO2	B	PTBCUT4	4mA	UP	CMOS,T
87	D[19]/PIO3	B	PTBCUT4	4mA	UP	CMOS,T
88	D[20]/PIO4	B	PTBCUT4	4mA	UP	CMOS,T
89	D[21]/PIO5	B	PTBCUT4	4mA	UP	CMOS,T
90	D[22]/PIO6	B	PTBCUT4	4mA	UP	CMOS,T
91	D[23]/PIO7	B	PTBCUT4	4mA	UP	CMOS,T
92	GND	G	VSSO	-	-	-
93	VDD	P	VDD3O	-	-	-
94	D[24]/PIO8	B	PTBCUT4	4mA	UP	CMOS,T
95	D[25]/PIO9	B	PTBCUT4	4mA	UP	CMOS,T
96	D[26]/PIO10	B	PTBCUT4	4mA	UP	CMOS,T
97	D[27]/PIO11	B	PTBCUT4	4mA	UP	CMOS,T
98	D[28]/PIO12	B	PTBCUT4	4mA	UP	CMOS,T
99	D[29]/PIO13	B	PTBCUT4	4mA	UP	CMOS,T
100	D[30]/PIO14	B	PTBCUT4	4mA	UP	CMOS,T
101	D[31]/PIO15	B	PTBCUT4	4mA	UP	CMOS,T
102	DREQ#[0]/PIO28	B	PTBST4	4mA	Null	CMOS-S,T
103	DACK#[0]/PIO29	B	PTBCT4	4mA	Null	CMOS,T
104	DREQ#[1]/PIO30	B	PTBST4	4mA	Null	CMOS-S,T
105	DACK#[1]/PIO31	B	PTBCT4	4mA	Null	CMOS,T
106	DTC/IRQ7#	B	PTBST4	4mA	Null	CMOS-S,T
107	TX[0]/PIO24	B	PTBCT4	4mA	Null	CMOS,T
108	RX[0]/PIO25	B	PTBCT4	4mA	Null	CMOS,T
109	TX[1]/PIO26	B	PTBCT4	4mA	Null	CMOS,T
110	RX[1]/PIO27	B	PTBCT4	4mA	Null	CMOS,T
111	TX1EN#/IRQ0#/PPM0	B	PTBST4	4mA	Null	CMOS-S,T
112	UCLK/PCS7#	B	PTBCT4	4mA	Null	CMOS,T
113	IREF	I	PIA_BB	-	Null	Analog
114	VBIAS	I	PIA_BB	-	Null	Analog
115	COMP	I	PIA_BB	-	Null	Analog
116	B	O	POA_BB	-	Null	Analog
117	AVDD	P	VDDA	-	-	Analog VDD

118	AGND	G	VSSA	-	-	Analog VSS
119	G	O	POA_BB	-	Null	Analog
120	AGND	G	VSSA	-	-	Analog VSS
121	AGND	G	VSSA	-	-	Analog VSS
122	R	O	POA_BB	-	Null	Analog
123	AVDD	P	VDDA	-	-	Analog VDD
124	VDD	P	VDDD	-	-	Digital VDD
125	GND	G	VSSD	-	-	Digital VSS
126	GND	G	VSSO	-	-	-
127	VDD	P	VDD3O	-	-	-
128	PWM	O	POB4	4mA	Null	CMOS
129	PCS1#/IRQ1#/PPM1	B	PTBST4	4mA	Null	CMOS-S,T
130	GND	G	VSSP	-	-	-
131	VDD	P	VDD3P	-	-	-
132	DCLK	O	POB4	4mA	Null	CMOS
133	REMLOC#	O	POB4	4mA	Null	CMOS
134	HVBLNK#	O	POB4	4mA	Null	CMOS
135	CBF	O	POB4	4mA	Null	CMOS
136	VSYN	O	POB4	4mA	Null	CMOS
137	VSYNIN	I	PTISU	-	UP	CMOS-S,T
138	CSYN	O	POB4	4mA	Null	CMOS
139	HSYN	O	POB4	4mA	Null	CMOS
140	CSYNIN	I	PTISU	-	UP	CMOS-S,T
141	OVEN#	O	POB4	4mA	Null	CMOS
142	CBCLK	O	POB4	4mA	Null	CMOS
143	VCLK	I	PTIC	-	Null	CMOS,T
144	GND	G	VSSO	-	-	-
145	VDD	P	VDD3O	-	-	-
146	VSCAS#	O	POB4	4mA	Null	CMOS
147	VSRAS#/VWE#	O	POB4	4mA	Null	CMOS
148	VSWE#/Cfg6	B	PBCT4	4mA	Null	CMOS
149	VDQM0/VCAS#[0]	O	POB4	4mA	Null	CMOS
150	VDQM1/VCAS#[1]	O	POB4	4mA	Null	CMOS
151	VDQM2/VCAS#[2]	O	POB4	4mA	Null	CMOS
152	VDQM3/VCAS#[3]	O	POB4	4mA	Null	CMOS
153	VCS#/VRAS#	O	POB4	4mA	Null	CMOS
154	GND	G	VSSP	-	-	-
155	VSDRCLK	B	PBCT4	4mA	Null	CMOS
156	VA[0]	O	POB4	4mA	Null	CMOS
157	VA[1]	O	POB4	4mA	Null	CMOS
158	VA[2]	O	POB4	4mA	Null	CMOS
159	VA[3]	O	POB4	4mA	Null	CMOS
160	VA[4]	O	POB4	4mA	Null	CMOS
161	VA[5]	O	POB4	4mA	Null	CMOS
162	VA[6]	O	POB4	4mA	Null	CMOS
163	VA[7]	O	POB4	4mA	Null	CMOS
164	VA[8]/Cfg7	B	PBCT4	4mA	-	CMOS
165	VA[9]/Cfg5	B	PBCT4	4mA	-	CMOS
166	VA[10]/Cfg4	B	PBCT4	4mA	-	CMOS
167	VA[11]/Cfg3	B	PBCT4	4mA	-	CMOS
168	VA[12]/Cfg2	B	PBCT4	4mA	Null	CMOS
169	GND	G	VSSP	-	-	-
170	VDD	P	VDD3P	-	-	-
171	VBA[0]/Cfg0	B	PBCT4	4mA	Null	CMOS
172	VBA[1]/Cfg1	B	PBCT4	4mA	Null	CMOS
173	VD[0]	B	PBCUT4	4mA	UP	CMOS
174	VD[1]	B	PBCUT4	4mA	UP	CMOS
175	VD[2]	B	PBCUT4	4mA	UP	CMOS
176	VD[3]	B	PBCUT4	4mA	UP	CMOS
177	VD[4]	B	PBCUT4	4mA	UP	CMOS
178	VD[5]	B	PBCUT4	4mA	UP	CMOS
179	VD[6]	B	PBCUT4	4mA	UP	CMOS
180	VD[7]	B	PBCUT4	4mA	UP	CMOS
181	VD[8]	B	PBCUT4	4mA	UP	CMOS
182	GND	G	VSSO	-	-	-

183	VDD	P	VDD3O	-	-	-
184	VD[9]	B	PBCUT4	4mA	UP	CMOS
185	VD[10]	B	PBCUT4	4mA	UP	CMOS
186	VD[11]	B	PBCUT4	4mA	UP	CMOS
187	VD[12]	B	PBCUT4	4mA	UP	CMOS
188	VD[13]	B	PBCUT4	4mA	UP	CMOS
189	VD[14]	B	PBCUT4	4mA	UP	CMOS
190	VD[15]	B	PBCUT4	4mA	UP	CMOS
191	VD[16]/PIO32	B	PBCUT4	4mA	UP	CMOS
192	VD[17]/PIO33	B	PBCUT4	4mA	UP	CMOS
193	VD[18]/PIO34	B	PBCUT4	4mA	UP	CMOS
194	VD[19]/PIO35	B	PBCUT4	4mA	UP	CMOS
195	VD[20]/PIO36	B	PBCUT4	4mA	UP	CMOS
196	GND	G	VSSI	-	-	-
197	VDD	P	VDD3I	-	-	-
198	VD[21]/PIO37	B	PBCUT4	4mA	UP	CMOS
199	VD[22]/PIO38	B	PBCUT4	4mA	UP	CMOS
200	VD[23]/PIO39	B	PBCUT4	4mA	UP	CMOS
201	VD[24]/PIO40	B	PBCUT4	4mA	UP	CMOS
202	VD[25]/PIO41	B	PBCUT4	4mA	UP	CMOS
203	VD[26]/PIO42	B	PBCUT4	4mA	UP	CMOS
204	VD[27]/PIO43	B	PBCUT4	4mA	UP	CMOS
205	VD[28]/PIO44	B	PBCUT4	4mA	UP	CMOS
206	VD[29]/PIO45	B	PBCUT4	4mA	UP	CMOS
207	VD[30]/PIO46	B	PBCUT4	4mA	UP	CMOS
208	VD[31]/PIO47	B	PBCUT4	4mA	UP	CMOS

* PAD TYPE DESCRIPTION

1. INPUT PAD Type

PIC : CMOS Level Normal Input pad

PTIC: CMOS Level Input pad(5V-tolerant)

PTICU: CMOS Level Input pad(5V-tolerant) with Pull-up

PTIS: Schmitt trigger Level Input pad(5V-tolerant)

PTISU: Schmitt trigger Level Input pad(5V-tolerant) with Pull-up

2. OUTPUT PAD Type

POB4: Normal Output pad(4mA Drive)

3. INOUT PAD Type

PBCT4 : CMOS level Tri-state I/O pad(4mA Drive)

PBCUT4 : CMOS level Tri-state I/O pad(4mA Drive) with Pull-up

PTBCT4 : CMOS level Tri-state I/O pad(4mA Drive/ 5V-tolerant)

PTBCUT4 : CMOS level Tri-state I/O pad(4mA Drive/ 5V-tolerant) with Pull-up

PTBST4 : Schmitt trigger level Tri-state I/O pad(4mA Drive/ 5V-tolerant)

PTBSUT4 : Schmitt trigger level Tri-state I/O pad(4mA Drive/ 5V-tolerant) with Pull-up

4. Oscillator PAD Type

PSOSCM2 : 2mA drive oscillator pad(10Mhz ~40Mhz)

5. Analog PAD Type

PIA_BB : Analog Normal Input Pad with separate Bulk Bias

POB_BB : Analog Normal output pad with separate Bulk Bias

POAR50_BB : Analog Normal Output Pad with Register 50ohm and separate Bulk Bias

- total 208pin = signal pin 173pin + power/ground 35pin = input(10) + output(65) + I/O(98) + power/gnd(35)

VISA3000 Pin Assign

2001.11.21 kykim

BOTTOM		RIGHT		TOP		LEFT	
No	Pin Name	No	Pin Name	No	Pin Name	No	Pin Name
1(I)	RESETX(t,s)	53(B)	A26/ctwrx	105(B)	DACK1X/PIO31(t)	157(O)	VA1
2(O)	PLLFILTER	54(O)	A27	106(B)	DTC/IRQ7X(t,s)	158(O)	VA2
3	AGND	55(O)	ROMCSX	107(B)	TX0/PIO24(t)	159(O)	VA3
4	AVDD	56(B)	IRQ6X/INTAX(t,s,u)	108(B)	RX0/PIO25(t)	160(O)	VA4
5	GND	57(O)	RDX	109(B)	TX1/PIO26(t)	161(O)	VA5
6(I)	XIN	58(O)	WRX	110(B)	RX1/PIO27(t)	162(O)	VA6
7(O)	XOUT	59(I)	IRDYX(t,u)	111(B)	TX1EN/IRQ0X/PPM0(t,s)	163(O)	VA7
8(I)	TEST	60(O)	PCS2X/STAT0	112(B)	UCLK/PCS7X(t)	164(B)	VA8/Cfg7
9(O)	BE0X(A0) / DC_OUT	61(O)	PCS3X/STAT1	113(I)	IREF	165(B)	VA9/Cfg5
10(O)	BE1X / AC_OUT	62(O)	PCS4X/HALTX	114(I)	VBIAS	166(B)	VA10/Cfg3
11(O)	BE2X(A1)	63(B)	PCS6X/NMIX	115(O)	COMP	167(B)	VA11/Cfg4
12(O)	BE3X/PCS0X	64(B)	D0 (t,u)	116(O)	Blue	168(B)	VA12/Cfg2
13	GND	65	GND	117	AVDD	169	GND
14	VDDO	66	VDDIP	118	AGND	170	VDDIP
15(O)	RA0(A2)	67(B)	D1 (t,u)	119(O)	Green	171(B)	VBA0/Cfg1
16(O)	RA1(A3)	68(B)	D2 (t,u)	120	AGND	172(B)	VBA1/Cfg0
17(O)	RA2(A4)	69(B)	D3 (t,u)	121	AGND	173(B)	VD0/dactdb0(u)
18(O)	RA3(A5)	70(B)	D4 (t,u)	122(O)	Red	174(B)	VD1/dactdb1(u)
19(O)	RA4(A6)	71(B)	D5 (t,u)	123	AVDD	175(B)	VD2/dactdb2(u)
20(O)	RA5(A7)	72(B)	D6 (t,u)	124	VDDD	176(B)	VD3/dactdb3(u)
21(O)	RA6(A8)	73(B)	D7 (t,u)	125	VSSD	177(B)	VD4/dactdb4(u)
22(O)	RA7(A9)	74(B)	D8 (t,u)	126	GND	178(B)	VD5/dactdb5(u)
23(O)	RA8(A10)	75(B)	D9 (t,u)	127	VDDO	179(B)	VD6/dactdb6(u)
24(O)	RA9(A11)	76(B)	D10(t,u)	128(O)	PWM/Compare	180(B)	VD7/dactdb7(u)
25(O)	RA10(A12)	77(B)	D11(t,u)	129(B)	IRQ1X/PCS1X/PPM1(t,s)	181(B)	VD8/dactdg0(u)
26	GND	78	GND	130	GND	182	GND
27	VDDP	79	VDDI	131	VDDP	183	VDDO
28(O)	RA11(A13)	80(B)	D12(t,u)	132(O)	DCLK	184(B)	VD9/dactdg1(u)
29(O)	RA12(A14)	81(B)	D13(t,u)	133(O)	REMLOCX	185(B)	VD10/dactdg2(u)
30(O)	BA0(A15)	82(B)	D14(t,u)	134(O)	HVBLANKX	186(B)	VD11/dactdg3(u)
31(O)	BA1(A16)	83(B)	D15(t,u)	135(O)	CBF	187(B)	VD12/dactdg4(u)
32(B)	A17/PIO16/pixel16/cta0(t)	84(B)	D16 /PIO0/pixel0(t,u)	136(O)	VSXNC	188(B)	VD13/dactdg5(u)
33(B)	A18/PIO17/pixel17/cta1(t)	85(B)	D17 /PIO1/pixel1(t,u)	137(I)	VSXNCIN(t,s,u)	189(B)	VD14/dactdg6(u)
34(B)	A19/PIO18/pixel18/cta2(t)	86(B)	D18 /PIO2/pixel2(t,u)	138(O)	CSXNC	190(B)	VD15/dactdg7(u)
35(B)	A20/PIO19/pixel19/cta3(t)	87(B)	D19 /PIO3/pixel3(t,u)	139(O)	HSXNC	191(B)	VD16/PIO32/dactdr0(u)
36(B)	A21/PIO20/PCS5X/pixel20/cta4(t,s)	88(B)	D20 /PIO4/pixel4(t,u)	140(I)	CSXNCIN(t,s,u)	192(B)	VD17/PIO33/dactdr1(u)
37(B)	A22/PIO21/IRQ5X/pixel21/cta5(t,s,u)	89(B)	D21 /PIO5/pixel5(t,u)	141(O)	OVENX	193(B)	VD18/PIO34/dactdr2(u)
38(B)	A23/PIO22/IRQ4X/pixel22/cta6(t,s,u)	90(B)	D22 /PIO6/pixel6(t,u)	142(O)	CBCLK	194(B)	VD19/PIO35/dactdr3(u)
39(B)	A24/PIO23/IRQ3X/pixel23/cta7(t,s,u)	91(B)	D23 /PIO7/pixel7(t,u)	143(I)	VLCLK(t)	195(B)	VD20/PIO36/dactdr4(u)
40	GND	92	GND	144	GND	196	GND
41	VDDO	93	VDDO	145	VDDO	197	VDDI
42(B)	A25/IRQ2X/cta8(t,s,u)	94(B)	D24 /PIO8/pixel8(t,u)	146(O)	VSCASX	198(B)	VD21/PIO37/dactdr5(u)
43(O)	LSDRCLK	95(B)	D25 /PIO9/pixel9(t,u)	147(O)	VWEX/VSRSX	199(B)	VD22/PIO38/dactdr6(u)
44(O)	EXCSX	96(B)	D26 /PIO10/pixel10(t,u)	148(B)	VSXWEX/Cfg6	200(B)	VD23/PIO39/dactdr7(u)
45(B)	MCSX(RASX)/ctrdx	97(B)	D27 /PIO11/pixel11(t,u)	149(O)	VCASX0/VDQM0	201(B)	VD24/PIO40/dactpd(u)
46(O)	A28/DQM0(SBE0X)/CAS0X	98(B)	D28 /PIO12/pixel12(t,u)	150(O)	VCASX1/VDQM1	202(B)	VD25/PIO41/dactclk(u)
47(O)	A29/DQM1(SBE1X)/CAS1X	99(B)	D29 /PIO13/pixel13(t,u)	151(O)	VCASX2/VDQM2	203(B)	VD26/PIO42(u)
48(O)	A30/DQM2(SBE2X)/CAS2X	100(B)	D30 /PIO14/pixel14(t,u)	152(O)	VCASX3/VDQM3	204(B)	VD27/PIO43(u)
49(O)	A31/DQM3(SBE3X)/CAS3X	101(B)	D31 /PIO15/pixel15(t,u)	153(O)	VRASX/VCSX	205(B)	VD28/PIO44(u)
50(B)	SDRASX(MWEX)/HOLDX	102(B)	DREQ0X/PIO28(t,s)	154	GND	206(B)	VD29/PIO45(u)
51(O)	SDCASX/HLDEX	103(B)	DACK0X/PIO29(t)	155(O)	VSDRCLK	207(B)	VD30/PIO46(u)
52(B)	SDWEX/INTX	104(B)	DREQ1X/PIO30(t,s)	156(O)	VA0	208(B)	VD31/PIO47(u)

Nand Tree Test : A17(start point) -->TEST(end point) --> BE0X(DC_OUT monitor) , BE1X(AC_OUT monitor)

(s)표시는 Schmitt PAD임

(t)표시는 Tolerant PAD임

(u)표시는 Pull-Up PAD임

빨간색은 Pin mux register에 의해서 Default로 설정되는 pin type임.

VSDRCLK는 PAD는 Bidirection pad를 사용했으나 항상 output으로 사용됨.

2.5 Pin Description

Pin name	I/O	Pin No.	Description
RESET#	I	1	System reset. Low active signal.
PLLFILTER	O	2	Internal PLL의 filter output
XIN	I	6	Internal/External Clock의 Source이다. Internal PLL을 사용할 때는 14.318MHz를 입력하고 External Clock을 사용할 경우는 CPU Clock의 2배 frequency를 입력한다.
XOUT	O	7	OSC Cell의 Output. XIN이 OSC PAD에서 바로 feedback 되어 output된다.
TEST	I	8	Test Mode를 enable/disable한다. 이 signal은 chip test를 위해 사용되고 Normal하게는 ground로 tie하여 Normal mode로 동작하도록 한다. Test Mode로 하려면 TEST PIN을 High로 하면서 다른 Configuration Pin의 Pullup/Pulldown으로 조정한다.
BE0# (A0)	O	9	CPU byte enable bit 0. ROM 또는 IO의 data bus width가 8bit으로 구성될 경우 system address bit 0에 해당하고, 16/32 bit로 구성될 때는 byte enable 0에 해당한다. DC Test(NandTree)는 Reset#를 Low로 계속 유지하면서 test할 수 있다. 이 pin은 DC_OUT으로 monitoring pin이다
BE1#	O	10	CPU byte enable bit 1. ROM 또는IO의 data bus width가 16bit / 32 bit로 구성될 경우 system의 byte enable 1에 해당한다.
BE2# (A1)	O	11	CPU byte enable bit 2. ROM 또는 IO의 data bus width가 16bit 또는 8bit으로 구성될 경우 system address bit 1에 해당하고, 32bit로 구성 될 때는 byte enable 2에 해당한다.
BE3# / PCS0#	O	12	CPU byte enable bit 3 또는 ROM 또는 IO의 data bus width가 32bit로 구성 될 경우 byte enable 3에 해당한다. Default는 BE3#으로 사용된다.
A[16:2] (BA[1:0], RA[12:0])	O	31~28, 25~15,	Local address bit [16:2]에 해당하며 BA[1:0]는 SDRAM에서 Bank select address로 사용되며, 특히 BA[0]는 FP/EDO DRAM의 Address[13]으로도 사용된다. RA[12:0]는 Fast Page DRAM, EDO DRAM, SDRAM의 Local Memory address로 사용된다.
A[20:17] / PIO[19:16]/ (cta[3:0])	B	35~32	Local address bit [20:17] 또는 Programmable IO port로 사용될 수 있다. Pin mux control register의 해당 bit를 프로그래밍하여 핀의 용도를 정할 수 있다. Pin mux control register2 bit[15:12]에 의하여 변경이 가능하다. DAC으로 input되는 pixel data[19:16]를 register(DPCON[1])를 setting함에 따라서 monitoring할 수 있다. Cache Test Mode시에는 cache를 위한 address[3:0]로 input으로 사용된다. Default는 A[20:17]으로 사용된다.
A21/PIO20/ PCS5#/ (cta[4])	B	36	Local address bit 21 또는 Programmable IO port20으로 사용될 수 있다. 그리고 Peripheral chip select 0로 사용되며, Pin mux control register2 bit[17:16]에 의하여 변경이 가능하다. DAC으로 input되는 pixel data[20]를 register(DPCON[1])를 setting함에 따라서 monitoring할 수 있다. Cache Test Mode시에는 cache를 위한 address[4]로 input으로 사용된다. Default는 PCS5#으로 사용된다.

A22/PIO21/ IRQ5#/ (cta5)	B	37	Local address bit 22 또는 Programmable IO port21으로 사용될 수 있다. 그리고 External Interrupt Request5로 사용되며, Pin mux control register2 bit[19:18]에 의하여 변경이 가능하다. DAC으로 input되는 pixel data[21]를 register(DPCON[1])를 setting함에 따라서 monitoring할 수 있다. Cache Test Mode시에는 cache를 위한 address[5]로 input으로 사용된다. Default 는 IRQ5#으로 사용된다.
A23/PIO22/ IRQ4#/ (cta6)	B	38	Local address bit 23 또는 Programmable IO port22으로 사용될 수 있다. 그리고 External Interrupt Request4로 사용되며, Pin mux control register2 bit[21:20]에 의하여 변경이 가능하다. DAC으로 input되는 pixel data[22]를 register(DPCON[1])를 setting함에 따라서 monitoring할 수 있다. Cache Test Mode시에는 cache를 위한 address[6]로 input으로 사용된다. Default 는 IRQ4#으로 사용된다.
A24/PIO23/ IRQ3#/ (cta7)	B	39	Local address bit 24 또는 Programmable IO port23으로 사용될 수 있다. 그리고 External Interrupt Request3로 사용되며, Pin mux control register2 bit[23:22]에 의하여 변경이 가능하다. DAC으로 input되는 pixel data[23]를 register(DPCON[1])를 setting함에 따라서 monitoring할 수 있다. Cache Test Mode시에는 cache를 위한 address[7]로 input으로 사용된다. Default 는 IRQ3#으로 사용된다.
A25/IRQ2#/ (cta8)	B	42	Local address bit 25또는 External Interrupt Request2으로 사용될 수 있다. Pin mux control register2 bit24에 의하여 변경이 가능하다. Cache Test Mode시에는 cache를 위한 address[8]로 input으로 사용된다. Default 는 IRQ2#으로 사용된다
A26 / ctwr _x	O (B)	53	Local address bit 26 Cache Test Mode시에는 cache를 위한 write command로 input으로 사용된다.
A27	O	54	Local address bit 27
EXCS#	O	44	외부 expansion ROM을 위한 chip select signal 이다. Address 0x0400_0000 부터 0xFFFF_FFFF까지의 16Mbyte * 12 영역에서 active 된다.
MCS#(RAS#) / ctrd _x	B	45	Local memory가 SDRAM/SRAM mode 일 경우 memory chip select 또는 Local memory가 EDO-DRAM mode일 경우 row address strobe signal로 사용된다. Cache Test Mode시에는 cache를 위한 read command로 input으로 사용된다.
D[15:0]	B	83~80, 77~67	CPU data bus lower 16 bit
D[31:16] / PIO[15:0]	B	101~94, 91~84,	CPU data bus upper 16 bit 또는 Programmable IO port로 사용된다. 외부 device를 위한 IO port로 이 bit를 사용할 경우 local memory data bus width는 data conflict를 피하기 위해 32bit bus로 사용할수 없다. Local memory bus width가 power on configuration에 따라 16bit으로 설정되면 이 핀들은 자동으로 PIO[15:0] 용도가 변경된다. DAC으로 input되는 pixel data[15:0]를 register(DPCON[1])를 setting함에 따라서 monitoring할 수 있다. Default 는 D[31:16] 으로 사용된다.
SBE[3:0]#/ DQM[3:0]/ CAS[3:0]# (A[31:28])	O	49~46	Local memory가 SRAM mode일 경우 SRAM data byte enable 신호로 사용한다. SRAM mode에서 data bus width 가 16 bit 이면 sbe[1] 은 SRAM address[1]에 해당하며, 32 bit 인 경우는 모두 byte enable로 사용된다. SDRAM mode일 우 DQM 신호로 동작하며, EDO-DRAM mode일 경우 column address strobe 신호로 동작한다. CPU test 시에 Local address bit[31:28] Default 는 SBE[3:0]#으로 사용된다.

SDRAS#/ MWE#/ HOLD#	B	50	Local memory가 SDRAM mode 일 우 row address strobe 신호로 Edo-DRAM/SRAM mode일 우 memory write enable 신호로 사용된다. Pin mux control register2 bit11에 의하여 변경이 가능하다 CPU의 Hold Request signal로 사용된다. Default 는 SDRAS#으로 사용된다.
SDCAS#/ HLDA#	B	51	Local memory로 SDRAM을 사용할 경우 column address strobe 신호로 사용된다. Pin mux control register2 bit11에 의하여 변경이 가능하다 CPU의 HOLD Acknowledge signal로 사용된다. Default 는 SDCAS#으로 사용된다.
SDWE#/ INT#	B	52	Local memory로 SDRAM을 사용할 경우 memory write enable 신호로 사용된다. Pin mux control register2 bit9에 의하여 변경이 가능하다. CPU의 Interrupt Request signal로 사용된다. Default 는 SDWE#으로 사용된다.
IRQ6#/ INTA#	B	56	External Interrupt Request6로 사용되며, Pin mux control register2 bit8에 의하여 변경이 가능하다. CPU의 Interrupt에 대한 acknowledge signal로 사용된다. Default 는 IRQ6#으로 사용된다.
NMI#/ PCS6#	B	63	CPU로의 Non maskable Interrupt이거나 Peripheral Chip Select6로 사용 가능하다. Pin mux control register2 bit10에 의하여 변경이 가능하다. Default 는 PCS6#으로 사용된다.
LSDRCLK	O	43	Local memory SDRAM clock.
ROMCS#	O	55	Local boot ROM chip select.
PCS1# / PPM1/ IRQ1	B	129	Peripheral chip select 신호 중 하나로써 output pin 또는 pulse period measurement channel 1 input pin으로 사용되기도 하고, External interrupt request 신호로 외부 device에서 입력되는 신호이다. Pin mux control register2 bit[4:3]에 의하여 select가 가능하다. Default 는 PCS1#으로 사용된다.
PCS2#/ STAT0	O	60	Peripheral chip select 신호이다. VISA3000은 총 8개의 chip select를 가지며 0x0120_0000 ~ 0x012F_FFFF address 영역에서 PCS2#신호가 active 된다. CPU의 Status를 monitoring하기 위한 STAT0로 사용된다. Pin mux control register2 bit[7]에 의하여 select가 가능하다. Default 는 PCS2#으로 사용된다.
PCS3#/ STAT1	O	61	Peripheral chip select 신호이다. VISA3000은 총 8개의 chip select를 가지며 0x0130_0000 ~ 0x013F_FFFF address 영역에서 PCS3#신호가 active 된다. CPU의 Status를 monitoring하기 위한 STAT1로 사용된다. Pin mux control register2 bit[6]에 의하여 select가 가능하다. Default 는 PCS3#으로 사용된다.
PCS4#/ HALT#	O	62	Peripheral chip select 신호이다. VISA3000은 총 8개의 chip select를 가지며 0x0140_0000 ~ 0x014F_FFFF address 영역에서 PCS4#신호가 active 된다. CPU의 Halt state를 나타내는 HALT# signal로 사용된다. Pin mux control register2 bit[5]에 의하여 select가 가능하다. Default 는 PCS4#으로 사용된다.
RD#	O	57	Local ROM, Expansion ROM, IO Access 할 때 Read Command Signal.
WR#	O	58	Local ROM, Expansion ROM, IO Access 할 때 Write Command Signal.
IRDY#	I	59	Peripheral Device (IO)에서 발생되는 wait 신호로서 active low로 동작한다. 이 핀의 사용여부는 PCS Control Register bit 15의 Set으로 이루어지며, 이 핀을 사용할 때의 실제 wait는 programmable 한 wait와 이 핀이 low인 구간의 wait 합으로 만들어진다. 즉, 이 핀이 high로 assert되기 전까지는 계속 wait 상태를 유지한다.
PWM	O	128	Pulse width modulation output 신호로 사용된다.

DREQ0# /PIO28	B	102	DMA channel 0를 위한 request 신호이다. VISA3000은 외부로부터 두개까지 DMA request를 받아들일 수 있다. Programmable IO port bit 28으로 사용된다. Pin mux control register1 bit[4]로 제어된다. Default 는 DREQ0#으로 사용된다.
DACK0# /PIO29	B	103	DMA channel 0 acknowledge 신호이다. Programmable IO port bit 29으로 사용된다. Pin mux control register1 bit[3]로 제어된다. Default 는 DACK0#으로 사용된다.
DREQ1# /PIO30	B	104	DMA channel 1을 위한 request 신호이다. Programmable IO port bit 30으로 사용된다. Pin mux control register1 bit[2]로 제어된다. Default 는 DREQ1#으로 사용된다.
DACK1# /PIO31	B	105	DMA channel 1 acknowledge 신호이다. Programmable IO port로 bit 31로 사용된다. Pin mux control register1 bit[1]로 제어된다. Default 는 DACK1# 으로 사용된다.
DTC /IRQ7	B	106	DMA terminal count signal로 즉, DMA data를 전송이 모두 완료되면 active된다. Pin mux control register의 해당 bit를 프로그래밍하여 External interrupt request 신호로 사용 할수 있으며, 외부 device에서 입력되는 신호이다. Default 는 DTC# 으로 사용된다.
TX0 /PIO24	B	107	UART channel 0 transmit data. Programmable IO port bit 24으로 사용된다. Pin mux control register1 bit[12]로 제어된다. Default 는 TX0으로 사용된다.
RX0 /PIO25	B	108	UART channel 0 receive data. Programmable IO port bit 25으로 사용된다. Pin mux control register1 bit[11]로 제어된다. Default 는 RX0으로 사용된다.
TX1 /PIO26	B	109	UART channel 1 transmit data. Programmable IO port bit 26으로 사용된다. Pin mux control register1 bit[10]로 제어된다. Default 는 TX1으로 사용된다.
RX1 /PIO27	B	110	UART channel 1 receive data. Programmable IO port bit 27으로 사용된다. Pin mux control register1 bit[9]로 제어된다. Default 는 RX1으로 사용된다.
TX1EN /IRQ0# /PPM0	B	111	UART channel 1 transmit data enable. Data 전송 중에 active 된다. External interrupt request 신호로 사용 할 수 있으며, 외부 device에서 입력되는 신호이다. 그리고 Pulse period measurement channel 0 input pin으로 사용된다. Pin mux control register1 bit[7:6]로 제어된다. Default 는 TX1EN으로 사용된다.
UCLK /PCS7#	B	112	The external UART clock input. 내부 UART의 operation clock을 외부에서 받아서 사용 할 수 있게 한다. Peripheral chip select 신호이다. Pin mux control register로 제어된다. Default 는 UCLK으로 사용된다.
VSCAS#	O	146	SDRAM을 Video Memory로 사용 시에는 CAS로 사용된다.
VWE# /VSRAS#	O	147	DRAM을 Video Memory로 사용 시에는 Write Command SDRAM을 Video Memory로 사용 시에는 RAS로 사용된다.

VSWEX/ Cfg6	B	148	SDRAM을 Video Memory로 사용 시 Write Command Power On Configuration시에는 Pullup/Down에서 입력이 되어 Test Pin과 함 께 Test Mode를 enable/disable한다. 자세한 내용은 CFGR Register를 참고
VCAS#[0]/ VDQM0	O	149	DRAM을 Video Memory로 사용 시에는 CAS0 SDRAM을 Video Memory로 사용 시에는 DQM0.
VCAS#[1]/ VDQM1	O	150	DRAM을 Video Memory로 사용 시에는 CAS1 SDRAM을 Video Memory로 사용 시에는 DQM1
VCAS#[2]/ VDQM2	O	151	DRAM을 Video Memory로 사용 시에는 CAS2 SDRAM을 Video Memory로 사용 시에는 DQM2
VCAS#[3]/ VDQM3	O	152	DRAM을 Video Memory로 사용 시에는 CAS3 SDRAM을 Video Memory로 사용 시에는 DQM3
VRAS#/ VCS#	O	153	DRAM을 Video Memory로 사용 시에는 RAS SDRAM을 Video Memory로 사용 시에는 Chip Select
VSDRCLK	B	155	Video RAM(SDRAM)으로의 Clock SDRAM Controller과 SDRAM의 clock의 skew를 줄이기 위해 I/O PAD를 사용 하여 feedback clock을 만들어 SDRAM Controller에서 사용하여 Video Memory access를 speed-up시킴.
VA[7:0]	O	163~156	Video RAM으로의 Address [7:0]
VA[8]/Cfg7	B	164	Video RAM으로의 Address [8] Power On Configuration시에는 Pullup/Down에서 입력이 되어 Test Pin과 함 께 Test Mode를 enable/disable한다. PLL Test Mode 시에는 Pull-down하고, {vswe#,va[9]}도 모두 pull-up으로 해야 한다. 자세한 내용은 CFGR Register를 참고
VA[9]/ Cfg5	B	165	Video RAM으로의 Address[9] Power On Configuration시에는 Pullup/Down에서 입력이 되어 Main Clock의 Main Clock의 Source를 결정한다. Pull-down : External Clock(XIN)을 Main Clock으로 사용한다. Pull_up : Internal Clock(PLL)을 Main Clock으로 사용한다. 이때 Xin은 14.318Mhz를 사용한다.
VA[10]/ Cfg4	B	166	Video RAM으로의 Address[10] Power On Configuration시에는 Pullup/Down에서 입력이 되어 Test Pin, VSWEX 과 함께 Test Mode를 결정한다. 자세한 내용은 CFGR Register를 참고
VA[11]/ Cfg3	B	167	Video RAM으로의 Address[11] Power On Configuration시에는 Pullup/Down에서 입력이 되어 Test Pin, VSWEX 과 함께 Test Mode를 결정한다. 자세한 내용은 CFGR Register를 참고
VA[12]/ Cfg2	B	168	Video RAM으로의 Address[12] Power On Configuration시에는 Pullup/Down에서 입력이 되어 BA[0]와 함께 Local ROM Data Bus Width를 결정한다.
VBA[1]/ Cfg1	B	171	Video RAM(SDRAM)으로의 Bank Address[1] Power On Configuration시에는 Pullup/Down에서 입력이 되어 Local Memory Data Bus Width를 결정한다.
VBA[0]/ Cfg0	B	170	Video RAM에서 SDRAM 사용 시 Bank Address[0]으로 사용되며, FP/EDO DRAM으로 사용 시 Address[13]으로 사용된다. Power On Configuration시에는 Pullup/Down에서 입력이 되어 VA[12]와 함께 Local ROM Data Bus Width를 결정한다.

VD[31:0] {DACTCLK DACTPD DACTDR[7:0] DACTDG[7:0] DACTDB[7:0]}	B	208~198 195~184 181~173	Video RAM Data Bus DAC Test Mode시에는 DAC Input들이 이 pin을 통해서 할당 된다. VD[25] = DACTCLK VD[24] = DACTPD VD[23:16] = DACTDR[7:0] VD[15:8] = DACTDG[7:0] VD[7:0] = DACTDB[7:0]
DCLK	O	132	Video Output을 위한 Dot Clock.
REMLOC#	O	133	Remote/Local Mode select Remote Mode는 외부Video가 있음을 나타낸다. 1: Remote Mode 0: Local Mode(default)
HVBLNK#	O	134	Horizontal/Vertical Blank
CBF	O	135	Color burst frame.
VSYNC	O	136	Vertical Sync output
VSYNCIN	I	137	Vertical Sync Input
CSYNC	O	138	Composite Sync 이 동기신호에 맞추어 video data를 화면에 display한다.
HSYNC	O	139	Horizontal Sync Video data의 수평동기 신호이다. 이 신호가 발생하면 화면의 왼쪽부터 새로운 line이 display된다.
CSYNCIN	I	140	Composite Sync Input 외부조합 영상신호를 동기 분류기로 통과시켜 얻은 신호이다. 이 신호는 외부 video의 존재여부를 나타낸다.
OVEN#	O	141	Overlay Enable. 외부 video data와 VISA3000의 video data를 화면에 선택적으로 display 하기위한 select 신호이다.
CBCLK	O	142	Color Burst Clock. Video data의 color 동기 신호를 나타낸다. Color 동기가 맞지 않으면 두 영상의 신호를 Overlay할 때마다 색상이 변한다.
VCLK	I	143	Video Clock으로서 Video Data를 화면에 display하는 기본 clock이다.
R	O	122	DAC Analog Red Data
G	O	119	DAC Analog Green Data
B	O	116	DAC Analog Blue Data
IREF	I	113	DAC의 Reference Voltage.
VBIAS	I	114	DAC의 Bias Voltage
COMP	I	115	DAC의 Compare Output
AVDD	P	4,117,123	Analog Power(3pin)
AGND	G	3,118,120,121	Analog Ground(4pin)
VDD	P	14,27,41,66,79,93,124,127,131,145,170,183,197	Power(13 pin)
GND	G	5,13,26,40,65,78,92,125,126,130,144,154,169,182,196	Ground(15pin)

3 REGISTER DESCRIPTIONS

VISA3000에 내장된 CPU SE3208은 32bit의 Address를 가지며 총 4Gbyte의 Address 영역을 Direct Access 할 수 있다. VISA3000의 메모리 영역은 아래의 표와 같이 할당되어 있다.

Address Range	Description	Remark
0000 0000h ~ 00FF FFFFh	Local ROM	ROMCS#
0100 0000h ~ 01FF FFFFh	Peripheral Device & Registers	
0100 0000h ~ 010F FFFFh	Peripheral Device 0	PCS0#
0110 0000h ~ 011F FFFFh	Peripheral Device 1	PCS1#
0120 0000h ~ 012F FFFFh	Peripheral Device 2	PCS2#
0130 0000h ~ 013F FFFFh	Peripheral Device 3	PCS3#
0140 0000h ~ 014F FFFFh	Peripheral Device 4	PCS4#
0150 0000h ~ 015F FFFFh	Peripheral Device 5	PCS5#
0160 0000h ~ 016F FFFFh	Peripheral Device 6	PCS6#
0170 0000h ~ 017F FFFFh	Peripheral Device 7	PCS7#
0180 0000h ~ 01FF FFFFh	VISA3000 Registers	
0200 0000h ~ 02FF FFFFh	Local DRAM/SRAM	
0300 0000h ~ 03FF FFFFh	Video Frame Memory	
0400 0000h ~ 0FFF FFFFh	Expansion ROM	EXCS# (A27 – A0)
1000 0000h ~ FFFF FFFFh	Reserved for Feature Use	

Table 3-1 VISA3000 Memory Map

VISA3000의 Register 영역은 01800000h부터 존재하며 각 기능 Block당 1Kbyte 씩 할당 되어 있다. Register들은 Memory mapped I/O의 형태로 자세한 내용은 아래와 같다.

Offset Address	Block	Remark
0180 0000h	System / General	
0180 0400h	Local Memory Controller	ROM, DRAM, SRAM
0180 0800h	DMA	2 Channel
0180 0C00h	Interrupt Controller	24 Channel
0180 1000h	UART	2 Channel
0180 1400h	Timer & Counter	4 Channel
0180 1800h	Pulse Width Modulation	1 Channel
0180 1C00h	Pulse Period Measurement	2 Channel
0180 2000h	PIO (PORT)	48 PORT
0180 2400h	Peripheral Chip Select	8 Chip Select
0180 2800h	Reserved	
0180 2C00h	Reserved	
0180 3000h	Video Memory Controller	
0180 3400h	CRT Controller	
0180 3800h	Graphic Controller	
0180 3C00h	Video Controller	
0180 4000h	RAMDAC & PLL	Tripple 10bit DAC
0180 4400h	Palette	Built in Memory Addressing
0180 4800h		
0180 4C00h	Reserved	
0180 5000h	Reserved	
~	~	
0180 FFFFh	Reserved	

Table 3-2 VISA3000 Register Offset Address

- The address boundary is 400h(1024Byte).

3.1 System / General Register

3.1.1 System ID Register (SYSID)

Address : 0180 0000h

Bit	R/W	Description	Default Value
31:16	R	Reserved.	00h
15: 8	R	Device ID	04h
7 : 0	R	Revision Number	00h

3.1.2 Configuration Register (CFGR)

Power On시 외부 Configuration Pin으로부터 설정된 값을 읽어온다. 해당 Pin은 반드시 Pull-up 또는 Pull-down Register를 연결시켜야 한다. Configuration Pin으로 사용되는 Pin은 VBA[1:0], VA[12:8], VSWE#로 총 8개이다.

Configuration Pin중 일부 Pin들은 Test를 위한 초기 값으로 사용되기 때문에 TEST Pin이 active되지 않는 상황에서는 pull-up/pull-down 어떤 상태에 있어도 무관하다.

Address : 0180 0004h

Bit	R/W	Description	Assigned Pin
31: 8	R	Reserved.	-
7	hidden	이 bit는 bit[6:5]=11 과 함께 PLL test Mode Enable bit이다. 0: PLL Test Mode 1: Normal Mode	VA[8]
6	hidden	이 configuration bit가 0이고 TEST pin이 1일 경우, 아래 bit[4:3]의 mode 설정이 가능. 이 configuration bit가 1이고 TEST pin이 1일 경우 내부 Counter들의 test 동작이 가능하다. 정상Mode에서는 TEST Pin은 0, configuration은 1이 되어야 한다.	VSWE#
5	hidden	Main Clock Select 0 : External Clock (Disable PLL) 1 : Internal PLL Clock 내부 PLL의 출력을 사용하지 않고 Chip외부의 clock source를 사용할 경우, Pin VA[9]에 Pull-down resister를 연결하면 Pin Xin을 통하여 외부 Clock을 사용할 수 있다.	VA[9]
4 : 3	hidden	Select Test Mode (When TEST pin is active state.) 00 : CPU Stand Alone 01 : Cache Tag Memory 10 : Internal DAC 11 : Cache Data Memory	VA[11:10]
2 : 1	R	Local ROM Data Bus Width 00 : 8Bit 01 : 16Bit 1x : 32bit	VA[12], VBA[0]
0	R	Local Memory Bus Width 0 : 16Bit 1 : 32Bit	VBA[1]

3.1.3 System Control Register (SYSCON)

Address : 0180 0008h

Bit	R/W	Description	Default Value
31 : 10	R	Reserved	0
9 : 8	RW	Expansion ROM Data Bus Width 00 : 8Bit 01 : 16Bit 1x : 32bit	0
7 : 6	RW	Frame Memory Internal Command Cycle Wait Control 00 : 2 Clk 01 : 3 Clk 10 : 4 Clk 11 : 5 Clk	11
5 : 4	RW	Register Internal Command Cycle Wait Control 00 : 2 Clk 01 : 3 Clk 10 : 4 Clk 11 : 5 Clk	11
3	R	Reserved	0
2 : 0	RW	Expansion ROM Read/Write Wait state 000 : 2 Clk 100 : 10 Clk 001 : 4 Clk 101 : 12 Clk 010 : 6 Clk 110 : 14 Clk 011 : 8 Clk 111 : 16 Clk * 1Clk는 main clock의 2분주 된 clock을 의미한다. 따라서 PLL output이 50Mhz일 경우, 내부 Bus clock은 25Mhz 이다.	111

3.1.4 Watch Dog Timer Control Register (WDCON)

Address : 0180 0010h

Bit	R/W	Description	Default Value
31 : 4	R	Reserved.	0
3	R	Watch Dog Reset Status. WDCON Read시 Clear된다. 0 : Normal 1 : Watch Dog에 의한 Reset발생	0
2 : 1	RW	Prescale Factor Selection 00 : 32 cycle 01 : 64 cycle 10 : 128 cycle 11 : 256 cycle Prescaler counter의 기준 Cycle에 대한 분주 된 Clock을 선택 할 수 있다. 최대 256 Cycle 주기를 갖는 Clock으로 WDCNT를 Count 가능.	11
0	RW	Watch Dog Timer Enable / Disable 0 : Disable Timer 1 : Enable Timer	0

3.1.5 Watch Dog Timer Count Register (WDCNT)

Address : 0180 0014h

Bit	R/W	Description	Default Value
31:16	R	Reserved.	0
15: 0	RW	Watch Dog Timer Count Value. Watch dog timer는 여기에 쓰여진 값에서부터 다운 카운팅을 하게 되며, 0에 도달하였을 때 Reset을 발생 시킨다. 이 Register 값은 항상 Watch dog counter의 값을 반영하고 있으므로 현재 카운터의 값을 읽어 볼 수 있다. Watch Dog에 의한 Reset의 폭은 선택된 Prescale Factor Selection 값에 따른다.	0xFFFF

3.1.6 Pin Mux Control Register 1 (PINMUX1)

Address : 0180 0018h

Bit	R/W	Description	Default Value
31 : 16	RW	Reserved.	0
15		Reserved.	
14		Reserved.	
13		Reserved.	
12		0 : BE3# 1 : PCS0#	
11		0 : TX0 1 : PIO24	
10		0 : RX0 1 : PIO25	
9		0 : TX1 1 : PIO26	
8		0 : RX1 1 : PIO27	
7 : 6		0x : TX1EN 10 : IRQ0# 11 : PPM0	
5		0 : UCLK 1 : PCS7#	
4		0 : DREQ0# 1 : PIO28	
3		0 : DACK0# 1 : PIO29	
2		0 : DREQ1# 1 : PIO30	
1		0 : DACK1# 1 : PIO31	
0		0 : DTC 1 : IRQ7	

Configuration Register의 Local Memory Bus Width의 설정에 따라 외부 Data Bus Pin인 D[31:16]은 PIO Pin으로 사용될 수 있다. 즉, Local Memory Data Bus로 16Bit을 사용할 경우에만 PIO0~PIO15로 사용할 수 있다.

3.1.7 Pin Mux Control Register 2 (PINMUX2)

Address : 0180 001Ch

Bit	R/W	Description	Default Value
31 : 25	R	Reserved.	0
24	RW	0 : A25	0
23:22		00 : IRQ3 1x : A24	
21 : 20		00 : IRQ4 1x : A23	
19 : 18		00 : IRQ5 1x : A22	
17 : 16		00 : PCS5# 1x : A21	
15		0 : A20	
14		0 : A19	
13		0 : A18	
12		0 : A17	
11		0 : MWE# / SDRAS# SDCAS#	
10		0 : PCS6#	
9		0 : SDWE#	
8		0 : IRQ6#	
7		0 : PCS2#	
6		0 : PCS3#	
5		0 : PCS4#	
4 : 3		0x : PCS1# 10 : IRQ1#	
2 : 0		Reserved.	

내부 CPU Core를 access하지 않는 case (normal case)에서는 Pin Mux register2[11:5]는 Zero를 유지하도록 한다.

3.2 Local Memory Control Registers

3.2.1 Local ROM Control Register (LROMC)

Address : 0180 0400h

Bit	R/W	Description	Default Value
31 : 8	R	Reserved.	0
7	RW	Pin LROMCS decode 방법 설정. 0 : Decode address and RD# 1 : Decode address only 이것은 Local ROM 영역을 decode 하는데 있어서 사용된 신호를 구분하여, LROMCS 신호를 출력시키도록 한다.	1
6 : 3	R	Reserved.	0
2 : 0	RW	LROMCS wait state 000 : 2 Clk 100 : 10 Clk 001 : 4 Clk 101 : 12 Clk 010 : 6 Clk 110 : 14 Clk 011 : 8 Clk 111 : 16 Clk 외부 device의 access 속도에 따라 programmable하게 set함으로써 최적의 상태를 갖도록 한다.	111

3.2.2 Local DRAM Control Register (LDCON)

Address : 0180 0408h

Bit	R/W	Description	Default Value
31 : 25	R	Reserved	0
24	RW	Local Memory Control Reset 0 : Active 1 : Software Reset 이 값이 1인 동안 Memory state machine은 reset되어지며 다시 active되어 질 때, SDRAM의 경우 mode setting cycle이 발생한다.	0
23 : 20	R	Reserved.	0000
19 : 18	RW	Row Address Line Number (SDRAM Only) 00 : 11 01 : 12 10 : 13 11 : Reserved.	0
17 : 16	RW	Column Address Line Number (FP,EDO DRAM / SDRAM) 00 : 8 01 : 9 10 : 10 11 : 11 SDRAM Mode 인 경우는 11 은 무시.	0
15 : 14	R	Reserved.	0
13	RW	Internal Read Data Latch Timing Control for FP DRAM 0 : Normal 1 : 1 Clk Delay	0
12 : 11	RW	RAS to CAS Delay. 00 : 1 Clk 01 : 2 Clk 1x : 3 Clk	01
10 : 9	RW	RAS Precharge Time 00 : 1 Clk 01 : 2 Clk 10 : 3 Clk 11 : 4 Clk	11
8	RW	SDRAM Mode : CAS Latency 0 : 2 Clk 1 : 3 Clk EDO/FP DRAM Mode : CAS Precharge Time 0 : 1 Clk 1 : 2 Clk	1
7 : 6	RW	EDO/FP DRAM Mode : CAS Active Pulse Width Time 0x : 1 Clk 1x : 2 Clk SRAM Mode : SRAM Command Width Time 00 : 1 Clk 01 : 2 Clk 10 : 3 Clk 11 : 4 Clk	11
5 : 4	RW	SDRAM Mode : RAS Cycle Time after Auto-refresh Command 00 : 3 Clk 01 : 4 Clk 10 : 5 Clk 11 : 6 Clk EDO/FP DRAM Mode : RAS Pulse Width for CBR Refresh 00 : 2 Clk 01 : 3 Clk 10 : 4 Clk 11 : 5 Clk	11
3	RW	Refresh Cycle Period 0 : 15.6 usec 1 : 31.2 usec	0
2	RW	Number of Refresh Cycle / Period 0 : 1 Cycle 1 : 2 Cycle	1
1 : 0	RW	Local Memory Type. 00 : SRAM 01 : EDO DRAM 10 : Fast Page DRAM 11 : Sync DRAM	0

3.3 DMA

3.3.1 DMA0 Control Register (DMAC0)

Address : 0180 0800h

Bit	R/W	Description	Default Value
31:11	R	Reserved.	0
10	RW	DMA0 Enable 0 : Disable 1 : Enable DMA Operation	0
9	RW	DMA0 Request Polarity. 0 : Active High 1 : Active Low	0
8	RW	DMASA0, DMADA0, DMATC0 Register and Internal Counter Simultaneous Write Enable 0 : Register only 1 : Register and counter	1
7:6	RW	DMA0 Transfer Mode 0x : Single Transfer 10 : Repeat Data Transfer with Counter Reloading 11 : Repeat Data Transfer with Counter and Address Reloading	0
5	RW	Source Address Hold 0 : Increase/Decrease Address 1 : Fix Address	0
4	RW	Source Address Direction 0 : Increase Address 1 : Decrease Address	0
3	RW	Destination Address Hold 0 : Increase/Decrease Address 1 : Fix Address	0
2	RW	Destination Address Direction 0 : Increase Address 1 : Decrease Address	0
1:0	RW	Data Transfer Width 00 : 8bit 01 : 16bit 1x : 32bit	0

DMAC0[8]

0: Register only 는 reload mode에서 다음 전송 시 다른 주소나 전송회수를 위해 사용되며, 단일 전송 모드나 재전송 모드의 처음에서는 반드시 1: Register and counter로 설정해야 한다.

3.3.2 DMA0 Source Address Register (DMASA0)

Address : 0180 0804h

Bit	R/W	Description	Default Value
31:0	RW	DMA0 Source Address A[31:0]	0

3.3.3 DMA0 Destination Address Register (DMADA0)

Address : 0180 0808h

Bit	R/W	Description	Default Value
31:0	RW	DMA0 Destination Address A[31:0]	0

3.3.4 DMA0 Transfer Count Register (DMATC0)

Address : 0180 080Ch

Bit	R/W	Description	Default Value
31:24	R	Reserved.	0
23: 0	RW	DMA0 Transfer Count Register. 이 값은 Data Width와 무관하게 DMA Operation이 수행될 때마다 1씩 감소한다.	0

DMA1 Control Register (DMAC1)

Address : 0180 0810h

Bit	R/W	Description	Default Value
31 : 11	R	Reserved.	0
10	RW	DMA1 Enable 0 : Disable 1 : Enable DMA Operation	0
9	RW	DMA1 Request Polarity. 0 : Active High 1 : Active Low	0
8	RW	DMASA1, DMADA1, DMATC1 Register and Internal Counter Simultaneous Write Enable 0 : Register only 1 : Register and counter	1
7 : 6	RW	DMA1 Transfer Mode 0x : Single Transfer 10 : Repeat Data Transfer with Counter Reloading 11 : Repeat Data Transfer with Counter and Address Reloading	0
5	RW	Source Address Hold 0 : Increase/Decrease Address 1 : Fix Address	0
4	RW	Source Address Direction 0 : Increase Address 1 : Decrease Address	0
3	RW	Destination Address Hold 0 : Increase/Decrease Address 1 : Fix Address	0
2	RW	Destination Address Direction 0 : Increase Address 1 : Decrease Address	0
1 : 0	RW	Data Transfer Width 00 : 8bit 01 : 16bit 1x : 32bit	0

3.3.6 DMA1 Source Address Register (DMASA1)

Address : 0180 0814h

Bit	R/W	Description	Default Value
31:0	RW	DMA1 Source Address A[31:0]	0

3.3.7 DMA1 Destination Address Register (DMADA1)

Address : 0180 0818h

Bit	R/W	Description	Default Value
31:0	RW	DMA1 Destination Address A[31:0]	0

3.3.8 DMA1 Transfer Count Register (DMATC1)

Address : 0180 081Ch

Bit	R/W	Description	Default Value
31:24	R	Reserved.	0
23: 0	RW	DMA1 Transfer Count Register. 이 값은 Data Width와 무관하게 DMA Operation이 수행될 때마다 1씩 감소한다.	0

3.4 Interrupt Controller

3.4.1 Interrupt Mode Register (INTMOD)

Address : 0180 0C00h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 8	RW	External IRQ7 ~ IRQ0 Trigger Mode 0 : Level Trigger 1 : Edge Trigger	0xFF
7 : 0	RW	External IRQ7 ~ IRQ0 Active State 0 : Active Low 1 : Active High	0xFF

* 외부 interrupt 입력 Pin을 Pulse Period Measurement를 위한 입력 핀으로 사용할 때는 이 register의 bit[9:8], bit[1:0]는 high로 반드시 setting해야 한다.

3.4.2 Interrupt Vector Register (INTVEC)

Interrupt의 우선순위는 아래 표와 같이 Vector 값이 작은 것이 우선순위를 갖는다. 따라서 여러 개의 Interrupt가 내부적으로 발생할 경우, 우선순위에 따라 Interrupt Vector를 발생시킨다.

Address : 0180 0C04h

Bit	R/W	Description	Default Value
31 : 11	R	Reserved.	0
10 : 8	RW	Interrupt Vector High 3 Bit. Programmable Bits.	0
7 : 5	R	Reserved.	0
4 : 0	W	Interrupt Clear Vector Value. 내부에서 발생된 Interrupt의 우선순위에 의하여 Interrupt Service Routine처리 후 다시 그 Code 값을 Write함으로써 해당 Interrupt Bit을 Clear 시킬 수 있다.	0

Vector No.	Description	Remarks
31:26	Reserved	Interrupt의 발생은 우선순위에 의하여 순차적으로 발생하며, CPU의 INTA#의 Active 구간에서 Vector 값을 Data Bus로 출력한다.
25	Sprite Interrupt	
24	BiTBLT Interrupt	
23	PWM Interrupt	
22	PPM Overflow Interrupt	
21	Video Vertical Blank Interrupt	
20	Video External Sync. Change Interrupt	
19	External IRQ7	
18	External IRQ6	
17	External IRQ5	
16	External IRQ4	
15	UART Channel 1 Transmit Interrupt	
14	UART Channel 1 Receive Interrupt	
13	UART Channel 1 Error Interrupt	
12	UART Channel 0 Transmit Interrupt	
11	UART Channel 0 Receive Interrupt	
10	UART Channel 0 Error Interrupt	
9	External IRQ3	
8	External IRQ2	
7	Timer 3 Interrupt	
6	Timer 2 Interrupt	
5	DMA 1 Interrupt	
4	DMA 0 Interrupt	
3	External IRQ1 / Pulse Period Measure 1 Interrupt	
2	External IRQ0 / Pulse Period Measure 0 Interrupt	
1	Timer 1 Interrupt	
0	Timer 0 Interrupt (Highest Priority)	

Table 1. Interrupt Vector & Priority

3.4.3 Interrupt Enable Register (INTEN)

Interrupt Source에 대한 Mask Bit로 Interrupt Enable/Disable 기능을 한다. Mask Bit가 0일 때 Interrupt는 Disable되고 1일 때 Enable 된다.

Address : 0180 0C08h

Bit	R/W	Description	Default Value
31:26	R	Reserved.	0
25	RW	Sprite Interrupt Enable	
24	RW	BiTBLT Interrupt Enable	
23	RW	PWM Interrupt Enable	
22	RW	PPM Overflow Interrupt Enable	
21	RW	Video Vertical Blank Interrupt Enable	
20	RW	Video External Sync. Change Interrupt Enable	
19	RW	External IRQ7 Enable	
18	RW	External IRQ6 Enable	
17	RW	External IRQ5 Enable	
16	RW	External IRQ4 Enable	
15	RW	UART Channel 1 Transmit Interrupt Enable	
14	RW	UART Channel 1 Receive Interrupt Enable	
13	RW	UART Channel 1 Error Interrupt Enable	
12	RW	UART Channel 0 Transmit Interrupt Enable	
11	RW	UART Channel 0 Receive Interrupt Enable	
10	RW	UART Channel 0 Error Interrupt Enable	
9	RW	External IRQ3 Enable	
8	RW	External IRQ2 Enable	
7	RW	Timer 3 Interrupt Enable	
6	RW	Timer 2 Interrupt Enable	
5	RW	DMA 1 Interrupt Enable	
4	RW	DMA 0 Interrupt Enable	
3	RW	External IRQ1 Enable / Pulse Period Measure 1 Interrupt	
2	RW	External IRQ0 Enable / Pulse Period Measure 0 Interrupt	
1	RW	Timer 1 Interrupt Enable	
0	RW	Timer 0 Interrupt Enable	

3.5 UART

3.5.1 UART Channel 0/1 Control Register (UCON0 / UCON1)

Address : 0180 1000h / 0180 1020h

Bit	R/W	Description	Default Value
31: 5	R	Reserved.	0
7	RW	Loop Back Test. 0 : Normal 1 : Loopback Mode for Test	0
6	RW	Send Break. Break는 한 Frame 기간 이상 Transmit Data 출력을 연속적으로 Low Level로 출력시키는 것임. 0 : Not Send Break 1 : Send Break	0
5	RW	Enable Receive Status Interrupt. Data 수신 중 발생한 Break, Error에 대한 Interrupt 발생을 Enable 또는 Disable 한다. 0 : Do not generate receive status interrupt 1 : Generate receive status interrupt	0
4	RW	Serial Clock Selection. 0 : Internal Clock 1 : External Clock (UCLK)	0
3 : 2	RW	Parity Mode. 0x : No Parity 10 : Even Parity 11 : Odd Parity	0
1	RW	Number of Stop Bit. 0 : 1 Bit per Frame 1 : 2 Bit per Frame	0
0	RW	Word Length. Frame당 전송 또는 수신될 Data Bit 수. 0 : 7 Bit 1 : 8 Bit	1

3.5.2 UART Channel 0/1 Status Register (USTAT0 / USTAT1)

Address : 0180 1004h / 0180 1024h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 12	R	Reserved.	0
11 : 8	R	Receive-FIFO Level. 16 Level FIFO. Receive Interrupt는 Receive FIFO의 반이 채워졌을 때 마다 발생한다.	0
7	R	Reserved	0
6	R	Transmitter Empty. 이 Bit는 Transmit Buffer에 전송할 Data가 없고 Tx Shift Register가 Empty 일 때 0로 Clear 된다. 0 : Transmitter is empty. 1 : Transmitter has some data. Tx is in progress.	0
5	R	Transmit FIFO Holding Data. 이 Bit는 UTXB0(UTXB1) register에 Write 함으로써 자동으로 Set 된다. 0 : Tx Buffer is Empty. 1 : Tx Buffer has some data.	0
4	R	Receive FIFO Data Ready. 이 Bit는 URXB0(URXB0) register를 읽어 FIFO가 empty이면 Clear된다. 0 : Receive FIFO is empty. 1 : Receive FIFO has some data.	0
3	R	Break Detect. 이 Bit는 USTAT0(USTAT1) register를 읽을 때 자동으로 Clear 된다. 0 : Normal Operation 1 : Break Sequence Detection	0
2	R	Frame Error. 이 Bit는 USTAT0(USTAT1) register를 읽을 때 자동으로 Clear 된다. 0 : No Error 1 : Error	0
1	R	Parity Error. 이 Bit는 USTAT0(USTAT1) register를 읽을 때 자동으로 Clear 된다. 0 : No Error 1 : Error	0
0	R	Overrun Error. 이 Bit는 USTAT0(USTAT1) register를 읽을 때 자동으로 Clear 된다. 0 : No Error 1 : Error	0

3.5.3 UART Channel 0/1 Transmit Buffer Register (UTXB0 / UTXB1)**Address : 0180 1008h / 0180 1028h**

Bit	R/W	Description	Default Value
31: 8	W	Reserved.	0
7: 0	W	Transmit Data for UART0 / 1	

3.5.4 UART Channel 0/1 Receive Buffer Register (URXB0 / URXB1)**Address : 0180 100Ch / 0180 102Ch**

Bit	R/W	Description	Default Value
31: 8	R	Reserved.	0
7: 0	R	Receive Data for UART0 / 1	

3.5.5 UART Channel 0/1 Baud Rate Divisor Register (UBDR0 / UBDR1)**Address : 0180 1010h / 0180 1030h**

Bit	R/W	Description	Default Value
31: 16	R	Reserved.	0
15: 0	RW	Baud Rate Divisor Value.	

TX/RX Baud Rate은 아래 식으로 계산된다. Clock Source는 Internal 또는 External Clock(UCLK)이 된다.

$$\text{Baud Rate} = \text{Clock Source} / (\text{Divisor Value} * 16)$$

3.6 Timer

3.6.1 Timer Control Register

Register Name	Address	Description
TMCON0	0180 1400h	Timer 0 Control Register
TMCON1	0180 1408h	Timer 1 Control Register
TMCON2	0180 1410h	Timer 2 Control Register
TMCON3	0180 1418h	Timer 3 Control Register

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 8	RW	Timer <i>n</i> Prescaler Divide Count Value(PD) 이 값은 1 이상이어야 한다.	0xFF
7 : 5	R	Reserved for test mode.	0
4 : 2	R	Reserved.	0
1	RW	Timer <i>n</i> Operation Mode 0 : Single (One Shot Count) 1 : Continuous (Periodic Count)	0
0	RW	Timer <i>n</i> Enable 0 : Stop 1 : Run	0

3.6.2 Timer Count Register

Register Name	Address	Description
TMCNT0	0180 1404h	Timer 0 Count Register
TMCNT1	0180 140Ch	Timer 1 Count Register
TMCNT2	0180 1414h	Timer 2 Count Register
TMCNT3	0180 141Ch	Timer 3 Count Register

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 0	RW	Timer Count Value.(TCV) Timer Up-counter 값은 이 Register 값과 비교 되어 진다.	0x000F

Timer를 이용하여 원하는 주파수를 구하는 식은 아래와 같다.

$$\text{Expect (Hz)} = \text{Bus_Clock} / ((\text{PD}+1) * (\text{TCV}+1))$$

3.7 Pulse Width Modulation

3.7.1 PWM Control Register (PMCON)

Address : 0180 1800h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 8	RW	PWM Prescaler Divide Count 8 Bit Value	0xFF
7 : 2		Reserved	
1	RW	PWM Output Polarity 0 : Negative (Start Level is High) 1 : Positive (Start Level is Low)	0
0	RW	PWM Enable 0 : Stop 1 : Run	0

3.7.2 PWM Duty Register (PMDTY)

Address : 0180 1804h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 0	RW	PWM Count Value A. PWM Up-counter 값은 제일 먼저 이 Register 값과 비교 되어 진다. 따라서 Counter값이 Register 값에 도달하였을 때 내부 출력은 1이 된다. 이는 Low Pulse의 Duty를 결정한다.	0x000F

3.7.3 PWM Period Register (PMPRD)

Address : 0180 1808h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 0	RW	PWM Count Value B.(PCNT) PWM Up-counter 값은 이 Register 값과 비교 되어 진다. 이 값은 전체 주기에 해당하는 값으로, Counter값이 Register 값에 도달 하였을 때 내부 출력은 0이 된다. 최종 출력의 Polarity는 PMCON Bit1에 의하여 결정된다.	0x000F

PWM를 이용하여 원하는 주파수를 구하는 식은 아래와 같다.

$$\text{Expect (Hz)} = \text{Bus_Clock} / ((\text{PD}+1) * (\text{PCNT}+1))$$

3.7.4 Pulse Count Register (PULCNT)

Address : 0180 180Ch

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 0	RW	Number of Pulse – 1	0x000F

3.8 Pulse Period Measurement

3.8.1 PPM 0 Control Register (PDCON0)

Address : 0180 1C00h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 8	RW	Input Pulse Sampling Period Count Value. 이 값은 1 이상이어야 한다.	0xFF
7 : 5	R	Reserved for test mode.	0
4	R	PPM 0 Pulse Width Count Overflow Status 이 Bit는 PDCON Read시 Clear 된다. 0 : Normal 1 : Overflow	0
3	RW	Pulse Measurement시 Interrupt 발생 위치. 0 : Trigger시 마다 발생. 1 : Active pulse 마지막 Trigger에서 발생.	0
2	RW	Measurement Mode 0 : Period 1 : Pulse	0
1	RW	Input Pulse Polarity. IRQ0 으로 입력된다. 0 : Negative (Pulse falling edge에서 Interrupt 발생) 1 : Positive (Pulse rising edge에서 Interrupt 발생)	0
0	RW	PPM 0 Enable 0 : Stop 1 : Run	0

3.8.2 PPM 0 Pulse Width Register (PDPW0)

Address : 0100 1C04h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 0	R	PDCON0 의 Sampling 주기로 Counting한 결과 값.	0

Pulse period measurement 기능을 사용하기 위해서는 먼저 PPM0와 PPM1 Pin으로 Pulse를 입력 받기 위하여 Pin mux control register (PINMUX1, PINMUX2)를 통하여 해당 기능 핀을 위한 프로그램이 필요하다. 현재 이 핀은 다른 기능을 위해 multiplex하여 사용하고 있기때문이다. 그리고 외부에서 들어오는 Pulse 의 active 상태를 그대로 받기 위하여 Interrupt Mode Register (INTMOD) bit [1:0]와 bit[9:8]은 1로 set되어야 한다. 만일 PPM0 만 사용한다면 bit[0]와 bit[8]만 1로 set되면된다.

3.8.3 PPM 1 Control Register (PDCON1)

Address : 0180 1C08h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 8	RW	Input Pulse Sampling Period Count Value. 이 값은 1 이상이어야 한다.	0xFF
7 : 5	R	Reserved for test mode.	0
4	R	PPM 1 Pulse Width Count Overflow Status 이 Bit는 PDCON Read시 Clear 된다. 0 : Normal 1 : Overflow	0
3	RW	Pulse Measurement시 Interrupt 발생 위치. 0 : Trigger시 마다 발생. 1 : Active pulse 마지막 Trigger에서 발생.	0
2	RW	Measurement Mode 0 : Period 1 : Pulse	0
1	RW	Input Pulse Polarity. IRQ1 으로 입력된다. 0 : Negative (Pulse falling edge에서 Interrupt 발생) 1 : Positive (Pulse rising edge에서 Interrupt 발생)	0
0	RW	PPM 1 Enable 0 : Stop 1 : Run	0

3.8.4 PPM 1 Pulse Width Register (PDPW1)

Address : 0180 1C1Ch

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15 : 0	R	PDCON1 의 Sampling 주기로 Counting한 결과 값.	0

3.9 PIO

3.9.1 PIO Mode Register (PIOMOD)

PIO Pin을 사용하기 위하여 먼저 Pin Mux Control Register를 설정하여야 한다.

Address : 0180 2000h

Bit	R/W	Description	Default Value
31	RW	PIO 31 Pin의 Mode Control.	0
30		PIO 30 Pin의 Mode Control.	
29		PIO 29 Pin의 Mode Control.	
28		PIO 28 Pin의 Mode Control.	
27		PIO 27 Pin의 Mode Control.	
26		PIO 26 Pin의 Mode Control.	
25		PIO 25 Pin의 Mode Control.	
24		PIO 24 Pin의 Mode Control.	
23		PIO 23 Pin의 Mode Control.	
22		PIO 22 Pin의 Mode Control.	
21		PIO 21 Pin의 Mode Control.	
20		PIO 20 Pin의 Mode Control.	
19		PIO 19 Pin의 Mode Control.	
18		PIO 18 Pin의 Mode Control.	
17		PIO 17 Pin의 Mode Control.	
16		PIO 16 Pin의 Mode Control.	
15		PIO 15 Pin의 Mode Control.	
14		PIO 14 Pin의 Mode Control.	
13		PIO 13 Pin의 Mode Control.	
12		PIO 12 Pin의 Mode Control.	
11		PIO 11 Pin의 Mode Control.	
10		PIO 10 Pin의 Mode Control.	
9		PIO 9 Pin의 Mode Control.	
8		PIO 8 Pin의 Mode Control.	
7		PIO 7 Pin의 Mode Control.	
6		PIO 6 Pin의 Mode Control.	
5		PIO 5 Pin의 Mode Control.	
4		PIO 4 Pin의 Mode Control.	
3		PIO 3 Pin의 Mode Control.	
2		PIO 2 Pin의 Mode Control.	
1		PIO 1 Pin의 Mode Control.	
0		PIO 0 Pin의 Mode Control. 0 : Totempole Output Mode 1 : Open Collector Output Mode	

PIO Port를 Input Port로만 사용할 경우에는 Mode Control Register를 1로 Set(Open Collector Output Mode)하고 PIOLDAT Register의 해당 Port Bit를 0으로 Set시킴으로써 항상 Input으로 사용이 가능하다.

PIO Port 0 ~ 15는 System Data Bus D[31:16]와 공유하고 있기 때문에, System Configuration Pin 설정에 따라 사용이 제한된다. Local Memory Bus Width로 16bit이 설정 되었을 경우에만, Pin D[31:16]은 PIO 로써 사용될 수 있다.

3.9.2 PIO Latched Output Data Register (PIOLDAT)

Address : 0180 2004h

Bit	R/W	Description	Default Value
31 : 0	RW	PIO31 ~ PIO0 Totempole Output Mode시 외부로 출력될 Data 값을 의미한다. Open Collector Output Mode시 0은 Output Disable 상태로 출력이 High-Z 상태가 되고, 1은 Output Enable 상태로 0을 출력한다. Write한 값은 Latch 되어지고 이 Register를 Read시 Write 했던 Data를 읽어 볼 수 있다.	0

3.9.3 PIO External Data Register (PIOEDAT)

Address : 0180 2008h

Bit	R/W	Description	Default Value
31 : 0	R	PIO31 ~ PIO0 PIO Pin으로부터 입력된 Data이다.	-

3.9.4 PIO Mode Register (PIOMOD2)

Address : 0180 2010h

Bit	R/W	Description	Default Value
31:16	R/W	Reserved.	0
15		PIO 47 Pin의 Mode Control.	
14		PIO 46 Pin의 Mode Control.	
13		PIO 45 Pin의 Mode Control.	
12		PIO 44 Pin의 Mode Control.	
11		PIO 43 Pin의 Mode Control.	
10		PIO 42 Pin의 Mode Control.	
9		PIO 41 Pin의 Mode Control.	
8		PIO 40 Pin의 Mode Control.	
7		PIO 39 Pin의 Mode Control.	
6		PIO 38 Pin의 Mode Control.	
5		PIO 37 Pin의 Mode Control.	
4		PIO 36 Pin의 Mode Control.	
3		PIO 35 Pin의 Mode Control.	
2		PIO 34 Pin의 Mode Control.	
1		PIO 33 Pin의 Mode Control.	
0		PIO 32 Pin의 Mode Control. 0 : Totempole Output Mode 1 : Open Collector Output Mode	

3.9.5 PIO Latched Output Data Register2 (PIOLDAT2)

Address : 0180 2014h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved	0
15 : 0	RW	PIO47 ~ PIO32 Totempole Output Mode시 외부로 출력될 Data 값을 의미한다. Open Collector Output Mode시 0은 Output Disable 상태로 출력이 High-Z 상태가 되고, 1은 Output Enable 상태로 0을 출력한다. Write한 값은 Latch 되어지고 이 Register를 Read시 Write 했던 Data를 읽어 볼 수 있다.	0

3.9.6 PIO External Data Register2 (PIOEDAT2)

Address : 0180 2018h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved	0
15 : 0	R	PIO47 ~ PIO32 PIO Pin으로부터 입력된 Data이다.	-

3.10 Peripheral Device Chip Select

3.10.1 PCS Control Register

Register Name	Address	Description	Pin Name
CS0CON	0180 2400h	PCS0 Control Register	
CS1CON	0180 2404h	PCS1 Control Register	
CS2CON	0180 2408h	PCS2 Control Register	
CS3CON	0180 240Ch	PCS3 Control Register	
CS4CON	0180 2410h	PCS4 Control Register	
CS5CON	0180 2414h	PCS5 Control Register	
CS6CON	0180 2418h	PCS6 Control Register	
CS7CON	0180 241Ch	PCS7 Control Register	

Bit	R/W	Description	Default Value
31 : 16	R	Reserved.	0
15	RW	External Ready Enable 0 : Not Use External Ready 1 : Use External Ready	0
14 : 13	R	Reserved.	0
12 : 8	RW	The Number of Wait State 0에서 31 Cycle까지의 Wait state를 추가하여 Chip Select의 Pulse 폭을 조정할 수 있다.	10000
7	RW	PCs Enable 0 : Disable 1 : Enable	0
6 : 4	R	Reserved.	0
3 : 2	RW	Data Bus Width 00 : 8Bit 01 : 16Bit 1x : 32Bit	0
1 : 0	RW	Decode Mode 0x : Decode Address Only 10 : Decode with RD# 11 : Decode with WR#	0

3.11 Video Frame Memory Control

3.11.1 Video Memory Configuration Register (VMCFG)

Address : 0180 3000h

Bit	R/W	Description	Default Value
31 : 10	R	Reserved.	0
9	RW	Display 구간에서 Pixel read cycle 만 허가한다. 이 Bit는 Overlay Page, Color mode등으로 인해 Video Refresh를 위한 Access Time이 부족할 때 사용할 수 있다. 0 : Normal 1 : Pixel Read Cycle Only for Video Refresh	0
8	RW	Video Memory Control Reset 0 : Active 1 : Software Reset 이 값이 1인 동안 Memory state machine은 reset되어지며 다시 active되어질 때, SDRAM의 경우 mode setting cycle이 발생한다.	1
7	R	Reserved.	0
6 : 5	RW	Row Address Line Number (SDRAM Only) 00 : 11 01 : 12 10 : 13 11 : Reserved.	0
4 : 3	RW	Column Address Line Number (FP,EDO DRAM / SDRAM) 00 : 8 01 : 9 10 : 10 11 : 11 SDRAM인 경우 11 은 무시됨.	0
2	RW	Video Memory Bus Width . 0 : 16Bit 1 : 32Bit	0
1 : 0	RW	Video Memory Type. 0x : Sync DRAM 10 : EDO DRAM 11 : Fast Page DRAM	0

3.11.2 Video Memory Control Register (VMCON)

Address : 0180 3004h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved	0
15 : 12	RW	Frame Memory SDRAM Clock Control (0x03000000 ~ 0x03ffffff) 이 값에 따라 약 1nsec 정도의 Delay를 줄 수 있다. Bit 15의 값이 set 될 경우 원래 clock의 inversion 상태이다.	0
11 : 10	RW	RAS to CAS Delay. 00 : 1 Clk 01 : 2 Clk 1x : 3 Clk	01
9 : 8	RW	RAS Precharge Time 00 : 1 Clk 01 : 2 Clk 10 : 3 Clk 11 : 4 Clk	11
7	R	Reserved.	0
6	RW	SDRAM Mode : CAS Latency 0 : 2 Clk 1 : 3 Clk EDO/FP DRAM Mode : CAS Precharge Time 0 : 1 Clk 1 : 2 Clk	1
5	RW	EDO/FP DRAM Mode : CAS Active Pulse Width Time 0 : 1 Clk 1 : 2 Clk	1
4 : 3	RW	SDRAM Mode : RAS Cycle Time after Auto-refresh Command 00 : 3 Clk 01 : 4 Clk 10 : 5 Clk 11 : 6 Clk EDO/FP DRAM Mode : RAS Pulse Width for CBR Refresh 00 : 2 Clk 01 : 3 Clk 10 : 4 Clk 11 : 5 Clk	11
2 : 1	RW	Refresh Cycle per Horizontal Scan Line. 00 : 2 Cycle 01 : 3 Cycle 10 : 4 Cycle 11 : 5 Cycle	11
0	RW	Internal Read Data Latch Timing Control for FP DRAM 0 : Normal 1 : 1 Clk Delay	1

3.12 CRT Controller

3.12.1 CRTC Status / Mode Register (CRTMOD)

Address : 0180 3400h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved	0
15	R	Horizontal Sync. Status. Sync Active 구간에서 0로 읽힌다.	-
14	R	Vertical Display Enable Status Vertical Display Enable 구간에서 1로 읽힌다.	-
13	R	Horizontal & Vertical Blank Period. Blank 구간에서 0이 읽힌다.	-
12	R	External Video Signal Status 0 : Invalid State 1 : Valid State	-
11 : 10	R	Reserved.	0
9	RW	Screen Blank Enable 0 : Normal 1 : Blank Screen	0
8	RW	Write Protect CRTC Registers except for CRTMOD Register bit [15:8] 0 : Write 가능. 1 : Write 불가능.	0
7	RW	Horizontal Scan Line Number 0 : 525 Line 1 : 625 Line	0
6	RW	Color Burst Frequency 0 : 3.58 Mhz 1 : 4.43 Mhz	0
5 : 4	RW	Local / Remote Mode 00 : Local Mode 01 : Remote Mode (외부 PLL 사용 안 하는 경우) 10 : Remote Mode (외부 PLL 사용 하는 경우) 11 : Remote Mode (MC1378을 사용 하는 경우) 이때 Csync는 High-z 상태로 간다.	0
3	RW	Line LUT Lower Address 0 : Odd Field (NTSC), Even Field (PAL) 1 : Even Field (NTSC), Odd Field (PAL)	0
2	RW	VSYNC Select at Remote Mode 0 : VSYNCIN (External Pin) 1 : Internal block (CsyncToVH)	0
1 : 0	RW	Vertical Sync Width Generation 00 : Serration Only 01 : Pre-equalization and Serration 10 : Post-equalization and Serration 11 : Pre/Post-equalization and Serration	0

3.12.2 CRTC Timing Control Register (CRTTIM)

Address : 0180 3404h

Bit	R/W	Description	Default Value
31 : 13	R	Reserved	0
15 : 14	RW	OVEN# Delay Write되는 값만큼 Video Clock (VCLK) Delay 발생. Dot clock delay 조정 후 미세 조정을 위해 사용한다.	0
13	RW	Fsc4clk Select.(Sub carrier 주파수를 선택한다) 0 : CPCLK Pin 1 : 14.318Mhz (External clock Xin)	0
12	RW	CBF Output Polarity 0 : Low Active Signal 1 : High Active Signal	0
11	RW	OVEN# Output Polarity 0 : Low Active Signal 1 : High Active Signal	0
10 : 8	RW	OVEN# Delay Write되는 값만큼 Dot Clock Delay 발생.	0
7	RW	Video Block으로 입력되는 Video Clock (VCLK) 내부적으로 2채배 허용. 0 : Normal 1 : 2채배	0
6 : 4	RW	CBCLK 생성. Video Block으로 입력되는 Video Clock을 3.58Mhz(NTSC)/4.43Mhz(PAL-I)가 되도록 분주한다. 000 : 금지 001 ~ 111 : 2 ~ 8분주	000
3	RW	VCLK (Video Clock) Select. 0 : VCLK Pin 1 : 14.318Mhz	0
2 : 0	RW	DCLK 생성. 외부 Video Encoder에서 입력되는 Video Clock을 분주하여 Dot Clock을 발생시킨다. 000 : 금지 001 ~ 111 : 2 ~ 8분주	000

3.12.3 Horizontal Sync Width / Back Porch Register (HSWBP)

Address : 0180 3408h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved	0
15 : 8	RW	Horizontal Back Porch - 1	0
7 : 0	RW	Horizontal Sync Width - 1	0

3.12.4 Horizontal Display Total Register (HDISP)

Address : 0180 340Ch

Bit	R/W	Description	Default Value
31 : 10	R	Reserved	0
9 : 0	RW	Horizontal Display Total - 1	0

3.12.5 Horizontal Sync Front Porch Register (HSFP)

Address : 0180 3410h

Bit	R/W	Description	Default Value
31 : 9	R	Reserved	0
7 : 0	RW	Horizontal Front Porch - 1 Local Video Mode시 이 register 값은 의미가 없다. Remote Mode의 경우에 사용한다.	0

3.12.6 Field Window Bound Register (FWINB)

Address : 0180 3414h

Bit	R/W	Description	Default Value
31 : 15	R	Reserved	0
14	RW	Even Field Select 0 : From Hsync block 1 : From CynvToVH	0
13 : 8	RW	Odd/Even Field Window Upper Bound by 16 pixel	0
7 : 6	R	Reserved.	0
5 : 0	RW	Odd/Even Field Window Lower Bound by 16 pixel	0

3.12.7 Vertical Sync Back Porch Register (VSBP)

Address : 0180 3418h

Bit	R/W	Description	Default Value
31 : 8	R	Reserved	0
7 : 0	RW	Vertical Sync Back Porch / 2 - 1	0

3.12.8 Vertical Display Total Register (VDISP)

Address : 0180 341Ch

Bit	R/W	Description	Default Value
31 : 9	R	Reserved	0
8 : 0	RW	Vertical Display Total	0

3.12.9 Horizontal Total Register (HTOT)

Address : 0180 3420h

Bit	R/W	Description	Default Value
31 : 12	R	Reserved	0
11	RW	Csync width is 2cplk for test.	0
10	RW	Enable this register bit[9:0] programming.	0
9 : 0	RW	(Horizontal Total / 2) - 1	0

3.12.10 Vertical Total Register (VTOT)

Address : 0180 3424h

Bit	R/W	Description	Default Value
31 : 11	R	Reserved	0
10	RW	Enable this register bit[9:0] programming.	0
9 : 0	RW	NTSC : (Vertical Total - 9) * 2 PAL : (Vertical Total - 9) * 2 - 1	0

* 정상적인 경우 HTOT와 VTOT Register는 Program 할 필요가 없다. 내부적으로 자동 Setting됨.

3.13 Graphic Controller

3.13.1 Video Page Base Address Register (VBASE)

Address : 0180 3800h

Bit	R/W	Description	Default Value
31 : 24	RW	Video Page 3 Base Address [23:16]	0
23 : 16	RW	Video Page 2 Base Address [23:16]	0
15 : 8	RW	Video Page 1 Base Address [23:16]	0
7 : 0	RW	Video Page 0 Base Address [23:16]	0

3.13.2 Horizontal Resolution Register (HRES)

Address : 0180 3804h

Bit	R/W	Description	Default Value
31 : 9	R	Reserved.	0
8 : 6	RW	Display Horizontal Resolution (HR2)	0
5 : 3	RW	Display Horizontal Resolution (HR1)	0
2 : 0	RW	Display Horizontal Resolution (HR0)	0

Block Transfer를 위한 Horizontal Resolution은 HR0, 1, 2에 의하여 지정된다. 각 3Bit에 대한 내용은 다음과 같다.

Field Value	Add-up Value	Can be specified in Field
000	0	HR0, HR1, HR2
001	32	HR0
010	64	HR0, HR1
011	128	HR0, HR1, HR2
100	256	HR0, HR1, HR2
101	512	HR0, HR1, HR2
110	1024	HR1, HR2
111	2048	HR2

640 수평 해상도를 위한 HR Field 설정 예. HR0 = 010b, HR1 = 010b, HR2 = 101b

화면의 설정과 실제 메모리상의 화면 크기 설정이 다를 경우, 이 레지스터 값은 메모리 상의 화면 크기와 같아야 한다. 이 레지스터에 설정된 값이 Linear Address를 구하는데 사용되므로 Horizontal Resolution은 메모리에 할당된 Dot의 수이다. 실제 640 x 480 해상도의 CRT Control Register를 Set하고 메모리 할당은 512 x 480 x 16bit color의 크기로 사용할 수 있다. 이 경우 HRES Reg.에 적용할 값은 512이다.

3.13.3 Raster Operation Value (ROPV)

Address : 0180 3808h

Bit	R/W	Description	Default Value
31 : 8	R	Reserved	0
7 : 0	RW	Raster Operation Value. 3 Operand에 대한 Logical Operation 값을 기술한다. - Pattern Data : Pattern Register Data - Source Data : Source X,Y로 지정된 Data (Bitblt Operation이 아닐 경우는 Frame memory write Data) - Destination Data : Destination X,Y로 지정된 Data (Bitblt Operation이 아닐 경우는 최근에 Read한 Frame memory read data) Operation은 32bit data 각 bit에 적용된다. Default 값은 항상 Source Data move의 형태, 즉 CPU Write Data를 Frame Memory로 직접 쓰기를 수행한다. 그러므로 Bitblt function을 수행 후 일반적인 CPU write operation (ROP operation을 하지 않는 쓰기 동작)을 하기 위해서는 항상 이 register 값을 0xCC 값으로 만들어야 원하는 값으로 write 되는 것을 막을 수 있다.	0xCC

<Example> Source ^ Destination 을 위한 ROPV register 값은 0x66 가 된다.
Pattern register 값 Move를 위해서는 0xF0, Destination Move는 0xAA이다.

Pattern Source Destination ROPV (for Src ^ Dst)

0	0	0	0 : ROPV bit[0]
0	0	1	1 : ROPV bit[1]
0	1	0	1 : ROPV bit[2]
0	1	1	0 : ROPV bit[3]
1	0	0	0 : ROPV bit[4]
1	0	1	1 : ROPV bit[5]
1	1	0	1 : ROPV bit[6]
1	1	1	0 : ROPV bit[7]

Frame memory로의 CPU write를 수행할 경우 write할 data는 Pattern Data와 최근에 read한 data와 ROP를 수행할 수 있다. 여기서 Source data는 current write cpu data이고 Destination data는 최근에 read한 data이다. Read한 data는 내부의 destination buffer (32bit x 4)에 저장되어 있어 원하는 pattern을 임의로 지정할 수 있다. (미리 Frame memory를 read 해 놓음으로써) Write할 data의 하위 address 1, 0에 따라 Raster operation되어질 destination buffer의 data가 자동으로 선택된다.

내부의 Destination buffer는 Block transfer를 수행하지 않을 때는 Frame memory read cache buffer로 사용된다.

<Example> Write data와 Pattern register 값과 Exclusive-or를 할 경우, ROPV는 0x3C 이다.
Write data와 Read한 4word data pattern과 And를 할 경우, ROPV는 0x88 이다.

3.13.4 Pattern Data Register (PATD)

Address : 0180 380Ch

Bit	R/W	Description	Default Value
31 : 0	RW	ROP를 위한 Pattern Data	0

256 color mode일 경우에는 4 pixel에 대한 Pattern Data, 64K color mode 일 경우에는 2 pixel에 대한 Pattern data를 의미한다.

16M Color mode (24bpp)에는 4 page 중 3 page를 각각 RGB로 할당하므로 Block transfer가 RGB 각각 3 page에 대하여 행해져야 한다. 그러므로 3번 transfer를 하도록 program을 하는 것을 제외하고는 256 color mode와 동일한 동작을 하게 되므로, Pattern data도 4 pixel을 의미한다.

3.13.5 Source Top-Left Coordinate Register (SRCXY)

Address : 0180 3810h

Bit	R/W	Description	Default Value
31 : 27	R	Reserved.	0
26 : 16	RW	Source Top-Left X Coordinate	0
15 : 11	R	Reserved.	0
10 : 0	RW	Source Top-Left Y Coordinate	0

3.13.6 Destination Top-Left Coordinate Register (DSTXY)

Address : 0180 3814h

Bit	R/W	Description	Default Value
31 : 27	R	Reserved.	0
26 : 16	RW	Destination Top-Left X Coordinate	0
15 : 11	R	Reserved.	0
10 : 0	RW	Destination Top-Left Y Coordinate	0

3.13.7 Window Size Register (WINSZ)

Address : 0180 3818h

Bit	R/W	Description	Default Value
31 : 27	R	Reserved.	0
26 : 16	RW	Transfer Window Width (Minimum 4)	0
15 : 11	R	Reserved.	0
10 : 0	RW	Transfer Window Height (Minimum 1)	0

3.13.8 Block Transfer Control Register (BITBLT)

Address : 0180 381Ch

Bit	R/W	Description	Default Value
31 : 4	R	Reserved.	0
3	RW	Video Page 3 Block Transfer Active	0
2	RW	Video Page 2 Block Transfer Active	0
1	RW	Video Page 1 Block Transfer Active	0
0	RW	Video Page 0 Block Transfer Active 1 : 지정된 block의 전송을 시작한다. 전송 완료 후 이 Bit는 자동으로 Reset 되어진다.	0

Bit block transfer를 위한 parameter를 설정 후, 해당 Page의 Block transfer를 active 시킨다. 동일한 위치의 동일한 크기의 block을 각 Page에 대하여 Transfer를 하고자 할 경우, Active bit을 동시에 Set 할 수 있다. 이 때 Transfer의 우선 순위는 Page 우선 순위와 같다. (Page3 > Page2 > Page1 > Page0) Transfer가 완료 되는 Page의 Active bit는 순서적으로 Clear되며, 마지막 Page의 전송이 완료 되었을 때 Interrupt가 발생된다.

Active bit이 set되어 transfer가 진행중일 때는 Graphic Controller register 내용을 갱신할 수 없다.

3.13.9 Image Sprite Control Register (SPRITE)

Address : 0180 3820h

Bit	R/W	Description	Default Value
31 : 24	RW	Global ROP value. Function은 ROPV와 동일하다. Sprite 동작 중에 ROPV 대신에 사용된다.	0
23 : 5	RW	Sprite Descriptor Table Start Address bit [23:5]	0
4 : 2	R	Reserved.	0
1	RW	Global ROP value Enable 1 : Global ROP value is used for BitBLT 0 : Global ROP value is ignored. ROPV is used for BitBLT	0
0	RW	Active Sprite. 이 bit가 1로 set되면 Bit block transfer가 시작되며, sprite table의 모든 block의 전송이 완료 되었을 때, 자동으로 clear 된다. Sprite 기능을 active시킨 후 종료 Interrupt 발생 전에 이 bit를 0으로 clear 할 경우, 그 시점의 진행중인 block의 전송을 마치고 종료 interrupt를 발생한다.	0

Sprite Descriptor Table 기본 구성

Bit 31	Bit 0
0	ROPV
1	PATD
2	SRCXY
3	DSTXY
4	WINSZ
5	BITBLT
6	EOT
7	Reserved

Descriptor Table은 8 Dword 단위로 Alignment되어야 한다. Descriptor는 왼쪽 그림과 같이 8 Dword 단위로 구성된다. Block Transfer를 위한 6개의 Parameter Value를 포함하며, Sprite가 Active 되면 Frame Memory로부터 읽어온 Table 값이 해당 Register에 Setting되고 Block Transfer를 수행한다. End of Table을 가리키기 위한 Key를 Check하여 Sprite를 종료한다. 그렇지 않을 경우, 다음 Descriptor를 읽어와 수행을 계속한다.

EOT (End of Table) 값은 7번째 Dword의 Bit[31:16]가 0x55AA 일 때이다.

Frame Memory

Descriptor 0	8 Dword
Descriptor 1	8 Dword
Descriptor 2	8 Dword
Descriptor 3	8 Dword
Descriptor 4	8 Dword
Descriptor 5	8 Dword [EOT]

<- 0x0376_4408 : Start Address

그림에서 Sprite Descriptor Table Start Address는 0376_4408h이고, 6개의 Block Transfer를 마친 후 Interrupt를 발생한다.

3.13.10 Reserved for Test Mode

Address : 0180 3824h

Bit	R/W	Description	Default Value
31 : 24	R	Reserved.	0
23 : 3	R	Sprite Address 가 Load된 Counter 값 (Bit4,3 = 00)	0
2 : 0	R	Internal Counter Test (10bit / 10bit / 9 bit counter)	0

3.14 Video Controller

3.14.1 Video Page Control Register

Register Name	Address	Description
VPCON0	0180 3C00h	Video Page 0 Control Register
VPCON1	0180 3C10h	Video Page 1 Control Register
VPCON2	0180 3C20h	Video Page 2 Control Register
VPCON3	0180 3C30h	Video Page 3 Control Register

Bit	R/W	Description	Default Value
31 : 6	R	Reserved	0
7	RW	Enable Page Display 0 : Disable 1 : Enable	0
6 : 4	R	Reserved	0
3	RW	Palette Bank Select 0 : Palette 0 1 : Palette 1	0
2 : 0	RW	Color Mode 000 : 1Bit 001 : 2Bit 010 : 4Bit 011 : 8Bit 10x : 16Bit 11x : 24Bit	011

3.14.2 Video Page LUT Base Address Register

Register Name	Address	Description
VPLUB0	0180 3C04h	Video Page 0 LUT Base Address Register
VPLUB1	0180 3C14h	Video Page 1 LUT Base Address Register
VPLUB2	0180 3C24h	Video Page 2 LUT Base Address Register
VPLUB3	0180 3C34h	Video Page 3 LUT Base Address Register

Bit	R/W	Description	Default Value
31 : 12	R	Reserved	0
11 : 0	RW	LUT Base Address A[23:12]	-

Video Page Line Look-Up Table 구성

Bit[31:22]	Bit[21:3]	Bit[2]	Bit[1:0]
Line Back Porch 10 Bit	Pixel Data Start Address A[23:5]	Line Enable / Disable	Horizontal Pixel Expansion

3.15 RAMDAC and Frequency Synthesizer

3.15.1 RAMDAC & PLL Control Register (DPCON)

Address : 0180 4000h

Bit	R/W	Description	Default Value
31 : 10	R	Reserved	0
9	RW	Power Saving Mode for DAC 0 : Normal 1 : Power Saving Enable	0
8	RW	Palette CPU Access Enable. 0 : Can not Access Palette Memory by CPU 1 : Can Access	0
7 : 3	R	Reserved.	0
2	RW	Write Protect PLL Program Register (PLPGM) 0 : Write 불가능. 1 : Write 가능.	0
1	RW	24Bit Palette Output 0 : Normal Mode 1 : Output to PIO[23:0] 이 Bit를 1로 Set할 경우, Pin mux Control Register와 상관없이 Palette의 R[7:0], G[7:0], B[7:0] 24bit 출력이 PIO Pin [23:0]로 출력되게 된다. 따라서 Local memory는 16bit Interface로 동작시켜야 하고, local address인 A[31:17]은 사용할 수 없다. 이 mode는 단지 test를 위한 mode이다.	0
0	RW	Power Saving Mode for PLL 0 : Normal 1 : Power Saving Enable 외부 Clock을 직접 사용 시, 내부의 PLL은 자동으로 Power Saving 모드로 전환된다.	0

3.15.2 PLL Program Register (PLPGM)

Address : 0180 4004h

Bit	R/W	Description	Default Value
31 : 16	R	Reserved	0
15 : 8	RW	Main Divider 8 Bit (M)	0x57
7 : 2	RW	Pre-divider 6 Bit (P)	001111
1 : 0	RW	Post Scaler 2 Bit (S)	10

Frequency Equation : $F_{out} = ((M + 8) * F_{in}) / ((P + 2) * 2^S)$

Reference input frequency : $F_{in} = 14.318\text{MHz}$

Example : $M = 0x3E$, $P = 001000$, $S = 01$ (50MHz)

Default frequency output is 20MHz.

OUTPUT FREQUENCY EQUATION & TABLE

Frequency Equation:
$$F_{OUT} = \frac{(m+8)}{(p+2) \times 2^s} \times F_{IN}$$

Table 1. Example of Divider Ratio

M[7]	M[6]	M[5]	M[4]	M[3]	M[2]	M[1]	M[0]	m	M	P[6]	P[5]	P[4]	P[3]	P[2]	P[1]	P[0]	p	P	S[1]	S[0]	2 ^s
0	1	0	1	0	1	0	1	85	$\frac{(m+8)}{93}$	0	1	0	1	0	0	1	41	$\frac{(p+2)}{43}$	0	0	1

Table 2. Sample Frequency Coefficient(Reference Input Frequency=14.318MHz)

Fout	P	M	S	Fout	P	M	S	Fout	P	M	S	Fout	P	M	S	Fout	P	M	S
10	15	87	3	41.6	19	144	1	69	9	98	1	100.76	25	182	0	137	5	59	0
14.318	8	72	3	42	13	80	1	70	16	80	0	101	17	126	0	138	23	233	0
15	19	168	3	43	8	52	1	71	22	111	0	102	22	163	0	139	22	225	0
16	15	68	2	44	25	158	1	72	18	93	0	103	19	143	0	140	7	80	0
17	10	49	2	44.3	14	91	1	73	18	94	0	104	17	140	0	141	11	120	0
18	27	138	2	44.74	10	67	1	74	10	54	0	105	10	80	0	142	10	111	0
19	11	61	2	44.9	9	61	1	75	15	81	0	106	8	66	0	143.18	7	82	0
20	15	87	2	45.5	12	81	1	76	11	61	0	107	17	134	0	144	15	163	0
21	13	80	2	46	24	159	1	77	14	78	0	108	22	173	0	145	6	73	0
22	12	78	2	47	21	143	1	77.25	21	116	0	109	16	129	0	146	3	43	0
23	24	159	2	48	15	106	1	78	18	101	0	110	20	161	0	147	13	146	0
24	25	173	2	49	11	81	1	79	27	152	0	111	14	116	0	148	1	23	0
25.175	23	168	2	49.2	22	157	1	80	15	87	0	112	15	125	0	149	20	221	0
25.5	14	106	2	49.5	10	75	1	81	30	173	0	113	17	142	0	150	19	212	0
26	17	130	2	50	8	62	1	82	9	55	0	114	22	183	0	151	9	108	0
27	22	173	2	50.35	23	168	1	83.04	13	79	0	115	22	185	0	152	11	130	0
28.322	21	174	2	51	22	163	1	84	13	80	0	116	8	73	0	153	17	195	0
29	8	73	2	52	17	130	1	85	14	87	0	117	21	180	0	154	2	35	0
30	19	168	2	53	8	66	1	86	8	52	0	118	23	196	0	155	21	241	0
30.25	18	161	2	54	9	75	1	87	11	71	0	119	14	125	0	156	17	199	0
31	7	70	2	55	20	161	1	88	25	158	0	120	19	168	0	157	22	255	0
31.5	8	80	2	56	15	125	1	89	12	79	0	121	18	161	0	158.12	21	246	0
32	15	144	2	56.644	21	174	1	89.8	9	61	0	122	21	188	0	159	17	203	0
32.514	17	79	1	57	22	183	1	91	12	81	0	123	20	181	0	160	21	249	0
33	21	98	1	58	8	73	1	92	24	159	0	124	7	70	0	161	2	37	0
34	10	49	1	59	19	165	1	93	8	57	0	125	9	88	0	162	17	207	0
35	36	80	1	60	19	168	1	93.4	17	116	0	126	8	80	0	163	11	140	0
35.5	22	111	1	61	21	188	1	94.5	8	58	0	127	13	125	0	164	9	118	0
36	18	93	1	62	13	122	1	95	9	65	0	128	14	135	0	165	19	234	0
37	10	54	1	62.5	9	88	1	95.5	7	52	0	129	8	82	0	166	20	247	0
37.5	15	81	1	63	8	80	1	96	15	106	0	130	11	110	0	167	7	97	0
38	11	61	1	64	15	144	1	97	20	141	0	131	18	175	0	168	13	168	0
39	27	150	1	65	11	110	1	98	11	181	0	132	7	75	0	169	13	169	0
39.5	27	152	1	66	12	121	1	99	21	151	0	133	19	187	0	170	6	127	0
40	15	87	1	66.6	12	123	1	99.7	25	180	0	134	12	123	0				
41	9	55	1	68	12	125	1	100.	8	62	0	135	12	124	0				

[Note] If you want to used anyother divider ratios, Please you contact to SEC engineers

Table 3-3 PLL(AL2005H)의 P,M,S 값에 따른 주파수 table

3.15.3 Pixel Data Register (PIXEL)

Address : 0180 400Ch

Bit	R/W	Description	Default Value
31: 24	R	Reserved	0
23: 0	R	Pixel Data Register DAC 입력 전의 Pixel 값을 읽을 수 있다.	-

3.16 Palette Memory

3.16.1 Palette Memory Bank 0 (PAL0)

Address : 01800 4400h ~ 0180 47FFh

Bit	R/W	Description	Default Value
31 : 24	R	Reserved	0
23 : 0	RW	Red (Bit[23:16]), Green(Bit[15:8]), Blue(Bit[7:0])	0

3.16.2 Palette Memory Bank 1 (PAL1)

Address : 0180 4800h ~ 0180 4BFFh

Bit	R/W	Description	Default Value
31 : 24	R	Reserved	0
23 : 0	RW	Red (Bit[23:16]), Green(Bit[15:8]), Blue(Bit[7:0])	0

VISA3000에서는 4 Page의 Graphic Data의 출력을 내장된 두개의 Palette로 선택적으로 Mapping 시킬 수 있다. Palette Memory Bank 0는 2/4/16/256 Color에 대한 Palette를 Programming 함으로써 Foreground Color 용 Palette로 사용하고, Palette Memory Bank 1은 256 Color의 Background Color용 Palette로 전용될 수 있다. 각 Palette의 Detail Address Map은 아래와 같다.

0x0180 4400 ~ 0x0180 47FF 256 Color background palette (PAL0)

0x0180 4800 ~ 0x0180 BFFF 256 Color foreground palette (PAL1)

0x0180 BFEA ~ 0x0180 BFEB 2 Color
 0x0180 BFEC ~ 0x0180 BFEF 4 Color
 0x0180 BFF0 ~ 0x0180 BFFF 16 Color

3.17 SE3208 System Co-processor

3.17.1 General Descriptions

Embedded 32bit EISC (Extendable Instruction Set Computer)에 적용되는 Coprocessor는 4Kbyte Unified Cache를 가지며 Write Through/Write Back 모드를 모두 지원한다. Cache 가능한 영역으로는 Local ROM, Local RAM, Expansion ROM영역이 해당된다.

- Cache Feature

- 4Kbyte Unified 2 Way Set Associative Cache

- Write Through / Write Back

- 16 Byte / Line

- LRU Replacement

- Memory Map for Caching

- 0000_0000h ~ 00FF_FFFFh : Local ROM Area

- 0200_0000h ~ 02FF_FFFFh : Local DRAM, SRAM Area

- 0400_0000h ~ 0FFF_FFFFh : Expansion ROM Area

3.17.2 System Co-processor Register Descriptions

3.17.2.1 Master Command Register

Co-processor #0 – R7

Bit	R/W	Description	Default Value
31 : 13	R	Reserved	0
12	R	Always 1.	1
11	W	Local ROM, Local RAM Function Unit Control 0 : Disable Cache Unit 1 : Enable Cache Unit	0
10	W	Expansion ROM Function Unit Control 0 : Disable Cache Unit 1 : Enable Cache Unit	0
9 : 8	R	Function Unit (Cache)	01
7 : 1	R	Reserved.	0
0	R/W	Co-processor wait option 0: normal(1clock) 1: 1 Wait (2clock)	

3.17.2.2 Status Register

Co-processor #0 – R6

Bit	R/W	Description	Default Value
31	R	Privileged co-processor.	1
30 : 28	R	Type	0
27 : 25	R	Sub type	0
23	R	Local ROM, Local RAM Cache Status 0 : Disable 1 : Enable	0
22	R	Expansion ROM Cache Status 0 : Disable 1 : Enable	0
21	R	Write Buffer (FiFo) Enable	1
20	R	Invalid Status 0 : Not Invalidate Processing 1 : Invalidate Processing	0
19 : 16	R	Reserved.	0
15 : 14	R	Debugger Configuration 2 Channel Debugger, Break only at Instruction Fetch.	01
13 : 12	R	Reserved. (No TLB)	0
11	R	Write Buffer Configuration 4 Word Deep Write Buffer	1
10 : 8	R	Cache Configuration Unified 1Kbyte, 2 Way Set Associative, Write Through/Write Back	011
7 : 1	R	Reserved.	0
0	R	Always 1	1

3.17.2.3 Invalidate Cache without Copy Back Register**Co-processor #0 – R5**

Bit	R/W	Description	Default Value
31	R	Cache Type 0 : Unified Cache	0
30 : 1	R	Reserved.	0
0	W	Cache Tag 상태를 모두 Invalidate 상태로 Set시킴. 0 : Disable Invalid (No Action) 1 : Enable Invalid R15 Master Command Register에서 Cache가 Disable되었을 경우에만 유효하다.	0

3.17.2.4 Memory Bank Data Register**Co-processor #0 – R3**

Bit	R/W	Description	Default Value
31 : 8	R	Reserved.	0
7	R	Always 1.	1
6	RW	Memory Bank (Local ROM / RAM, Expansion ROM)의 Cache Configuration을 설정한다. 0 : Write Through Cache 1 : Write Back Cache	0
5 : 0	R	Reserved.	0

4 FUNCTION DESCRIPTION

4.1 DMA

I/O 장치와 메모리간의 데이터 전송에 있어 프로그램에 의한 방법을 사용하게 되면 데이터 전송률이 낮아지게 되고, CPU는 거의 모든 시간을 I/O와 데이터 전송에 관여해야만 하므로 시간적인 낭비를 가질 수 있다. 데이터 전송률을 높이고 시스템의 효율을 높이기 위해서 데이터 전송을 전담하는 제2의 CPU인 DMA를 두어서 직접적으로 I/O와 메모리간 또는 메모리와 메모리 사이의 대용량의 데이터 전송을 처리할 수 있다. 다시 말해서, DMA는 데이터의 입출력 동작속도를 빠르게 하고 CPU가 담당할 입출력 동작을 대신 수행하기 위해서 사용된다.

사용되는 예로서는 Hard Disk와 Memory사이의 데이터 전송 및 CRT 입출력을 위한 스캐닝 동작 등이 있다.

다음은 본 VISA3000의 DMA에 관한 주요 feature 이다.

- 2 Channel DMA
- Priority Decision
- Bus Clock 사용 (Main clock / 2)
- 최대 128Mbyte Addressing
- Cycle Steal Mode
- Transfer Width: 8, 16, 32 bit
- Source, Destination Address Control Programmable: Decrease, Increase, No change

참고로, DMA와 Interrupt의 차이는 machine cycle이 끝나야만 break point를 얻을 수 있는 것이 Interrupt이고, 매 machine cycle마다 cycle stealing과 같은 방식으로 break point를 얻을 수 있는 것이 DMA이다.

4.1.1 Timing Diagram

4.1.1.1 DMA Priority Decision Method & Operation Timing Diagram

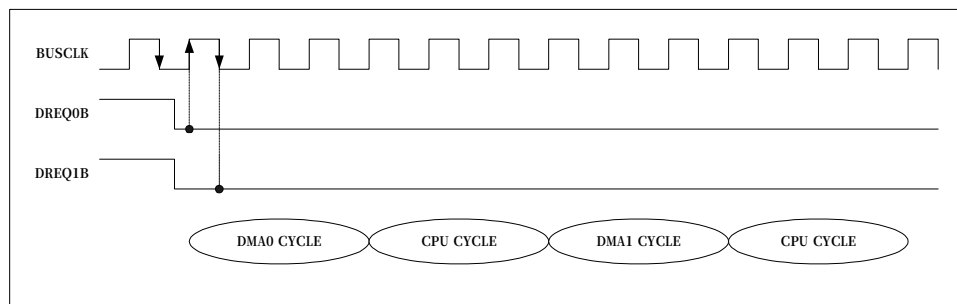


Figure 4.1-1 DMA Priority Decision Method & Operation

위의 그림과 같이 DMA Request 가 동시에 발생했을 경우 DMA Channel 0는 Bus Clock 의 Rising Edge에서 DREQ0B를 검사하고, DMA Channel 1은 Bus Clock 의 Falling Edge에서 DREQ1B를 검사한다. 따라서, 먼저 검출된 DMA Request (여기서는 DREQ0B)에 해당하는 DMA Channel 이 동작하게 된다. 그리고, DMA Channel 0를 먼저 수행한 후, DMA Channel 0에 대한 내부 동작 신호를 Disable 시켜 CPU Cycle 후에 DMA1 Cycle 을 수행하도록 한다. 그러므로 위의 그림과 같은 DMA Request 신호가 발생하게 된다면 DMA와 CPU의 동작 사이클은 (DMA0 → CPU → DMA1 → CPU)의 순서가 된다.

4.1.1.2 DMA Priority Decision Method & Operation Timing Diagram

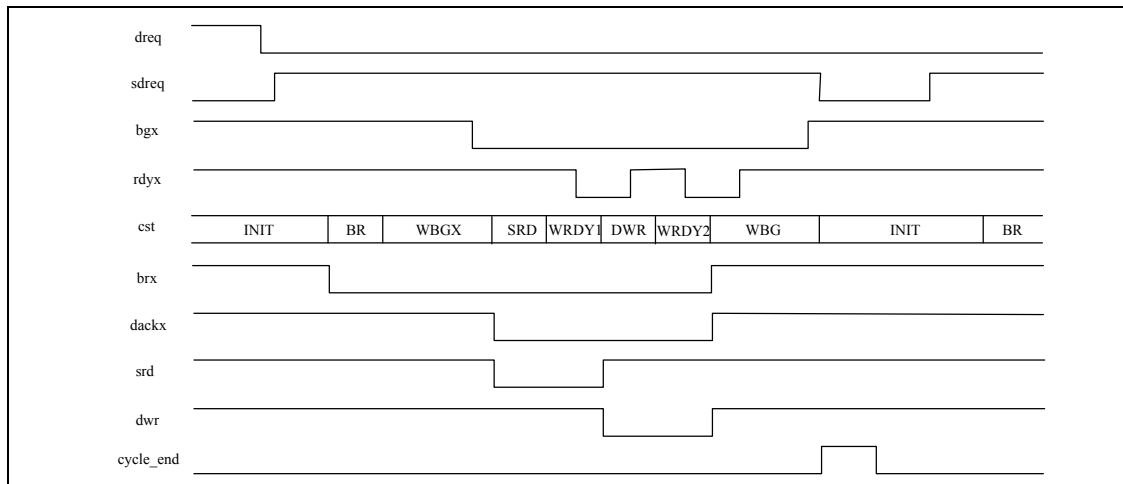


Figure 4.1-2 Timing diagram of DMA control logic block

4.1.2 DMA Mode

4.1.2.1 One Shot Transfer Mode

레지스터에 설정한 Transfer Counter 만큼 전송되고, DMA Channel이 Disable 되는 모드이다.

4.1.2.2 Auto Reloading Transfer Mode

One Shot Transfer Mode에서 Transfer Counter가 0에 도달하면 전송이 종료되는 반면에 Auto Reloading Transfer Mode는 레지스터에서 Counter로 Reloading되어 전송이 다시 계속된다. 그러므로 Register에 Reloading이 일어나기 전에 원하는 새로운 값들(Transfer Counter, Source Address, Destination Address)이 설정되어 있어야 한다. 다음은 address auto-reloading의 사용 예이다. 2000H ~ 2FFFH, 3000H ~ 3FFFH 번지의 메모리 두 개 블록의 데이터를 번갈아가며 5000H라는 주소로 전달하는 경우에 먼저 control register DMAC[8]을 '1'로 설정하여 첫번째 블록의 시작 주소 값(2000H)을 source address register (DMASA) 입력하고 이 값이 source address counter에도 로드되도록 한다. 다음 DMAC[8]을 '0'으로 하여 두번째 블록의 시작 주소 값(3000H)을 DMASA 레지스터에만 입력한다. DMASA의 값이 source address counter로 로드되는 것은 DMASA 입력할 때와 auto-reload mode에서 전송을 마치는 순간에만 일어나므로 DMA 설정 처음 단계에서 반드시 DMA[8]을 '1'로 해주고 DMASA에 주소 값을 입력해야 처음 전송이 제대로 된다. DMA enable 되어있고 DREQ가 active 되면 2000H 번지부터 차례로 5000H 번지로 데이터 이동이 시작되며 정해진 개수가 다 전송되면 DMASA의 값 3000H가 source address counter로 로드되어 다시 3000H부터 전송이 시작된다. 첫번째 블록의 전송이 마치면 인터럽트가 일어나고 사용자는 인터럽트 루틴에 2000H를 DMASA에 입력하는 내용을 작성하면 두번째 블록의 전송이 끝나고 다시 첫번째 블록의 전송을 할 수 있게 된다.

4.2 UART

호스트 컴퓨터 또는 기타 주변장치와의 비동기식 직렬 통신을 위한 회로로 다음과 같은 특성을 갖도록 설계되었다.

- Transfer rate: 110 bps to 76800 bps
- Full-duplex communication
- Data length: 7 or 8 bits
- Stop bit length: 1 or 2 bits
- Parity bit: even, odd, or none
- Receive error detection: parity, frame, and overrun errors

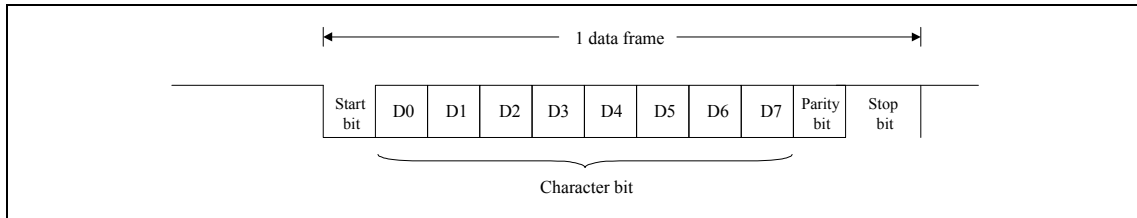


Figure 4.2-1 UART data format

4.2.1 Function Block Diagram

Baud clock generator 블록은 외부 클럭 uclk, 내부 클럭 uartclk, bclk 중 하나를 선택하여 UBDR 레지스터의 값에 따라 분주하여 보드 클럭인 baud_clk보다 16배 빠른 bd_clk16을 발생시킨다. baud_clk는 송신단의 구동 클럭으로, bd_clk16은 receiver단의 구동 클럭으로 사용된다. rxd 포트로 입력되는 신호는 RSR 레지스터에서 쉬프트되어 통신 모드에 따라 7비트, 8비트마다 RX FIFO로 로드된다. 이때 parity, frame error 비트가 추가되므로 FIFO width는 10비트이다. rx interrupt는 RSR에서 RXFIFO로 데이터 로드가 일어나는 때 8번째마다 일어나며, rx error interrupt는 parity, frame, overrun error의 경우에 발생한다. txd 포트를 통한 데이터 전송은 TXFIFO에 데이터가 있으면 TSR 쉬프트 레지스터에 데이터가 로드되어 시작한다. parity와 stop bit는 control logic 블록에서 계산되어 TSR에 데이터 비트와 같이 로드된다. txd 신호는 loopback test를 위하여 rxd 외부 신호대신 RSR에 입력될 수 있다.

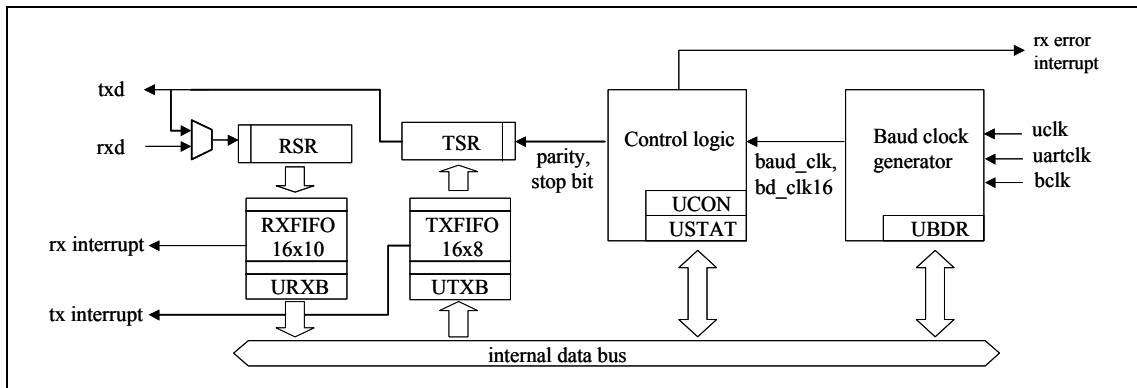


Figure 4.2-2 Functional block diagram of UART

4.2.2 Timing Diagram

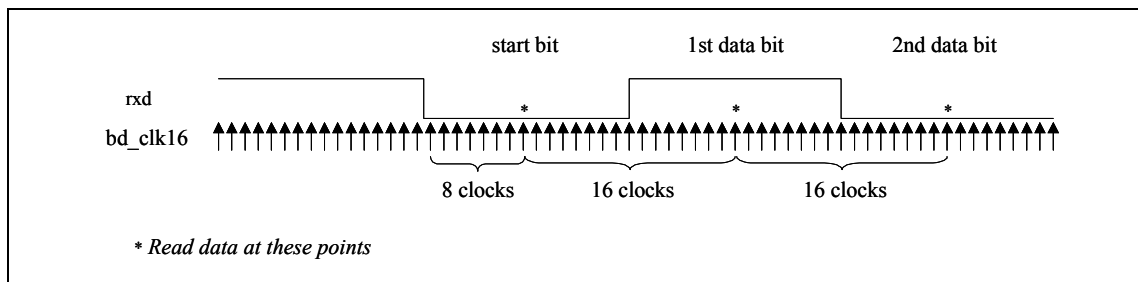


Figure 4.2-3 Sampling timing of rx

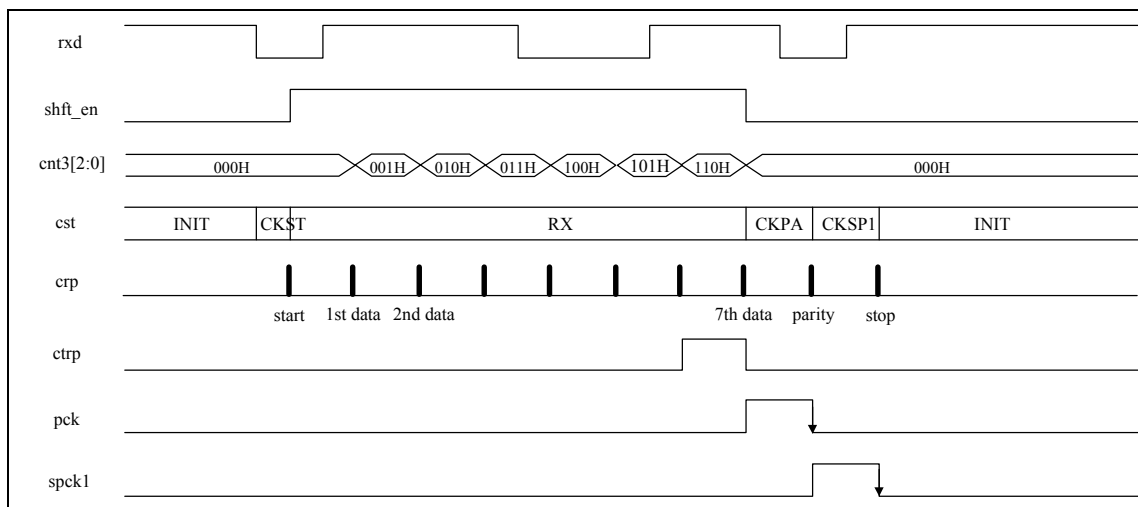


Figure 4.2-4 Timing diagram of receiver control logic

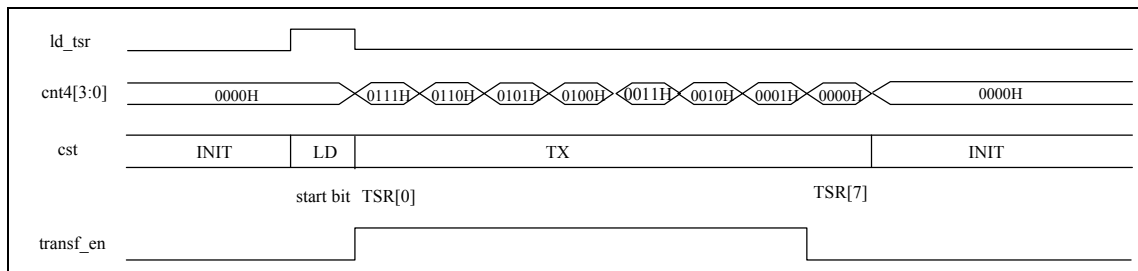


Figure 4.2-5 Timing diagram of transmitter control logic

4.2.3Baud generation

bclk, uartclk, uclk 세 클럭 중 하나를 분주하여 baud clock을 만들어 내며, 분주 시 사용하는 카운터는 16비트의 크기를 갖는다. UART baud rate divisor register에 설정된 값을 N이라 했을 때, baud rate B는 다음과 같이 정해진다.

$$B = f / (16*(N+1)) \quad (4.6.1)$$

f : selected clock frequency

4.3 Video Sync와 Clock

CRT를 제어하는 신호 즉 video 신호 중 video sync 신호를 발생하는 블록이다.

4.3.1 Video Sync

VISA3000의 video sync generation 은 application 에 따라 다양한 기능을 지원한다. video sync를 스스로 발생할 수도 있고 외부의 composite sync에서 각 요소를 분리 할 수도 있다.

VISA3000의 Video 출력은 local mode 와 remote mode를 갖으며 remote mode 는 3가지 경우로 구분되어진다.

Local mode와 remote mode를 나누는 기준은 다음과 같다.

Local : video sync를 스스로 만들.

Remote : video sync 는 외부의 것에 맞추어 진다.

remote mode 1 : no pll mode

→ CsynToVH에 의지하는 모드 외부에 PLL이 불필요. 외부 비디오 sync 와 완벽하게 맞추어 지지는 않는다. 그러므로 화면 pixel이 수평 방향으로 1 pixel 미만으로 흔들 릴 수도 있다.

remote mode 2 : pll mode

→ -> 외부에 PLL을 사용하는 mode로 외부 비디오 sync 와 완벽하게 맞추어 진다. 임의의 해상도를 만들 수 있다.

remote mode 3 : MC1378 mode

-> video overlay encoder로 MC1378을 사용함.

visa3000 의 sync generator는 다음과 같이 5가지 application에 target을 두고 설계된 것으로 어떠한 analog video encoder도 interface 할 수 있다.

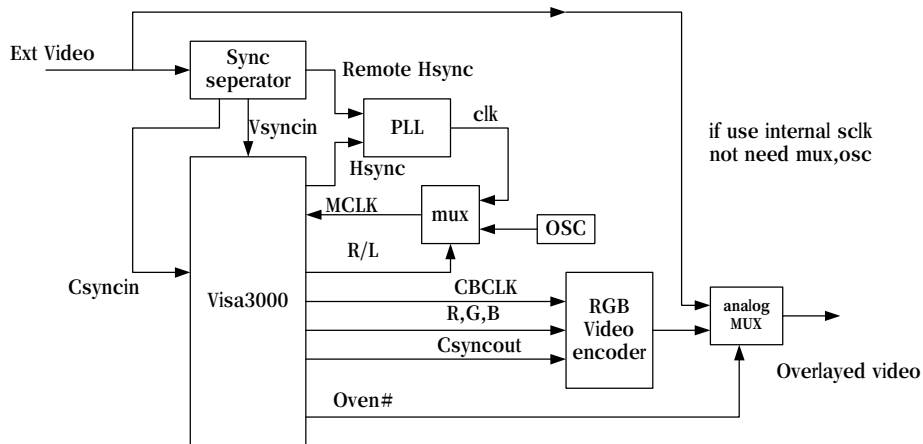


Figure 4.3-1: general interface PLL mode

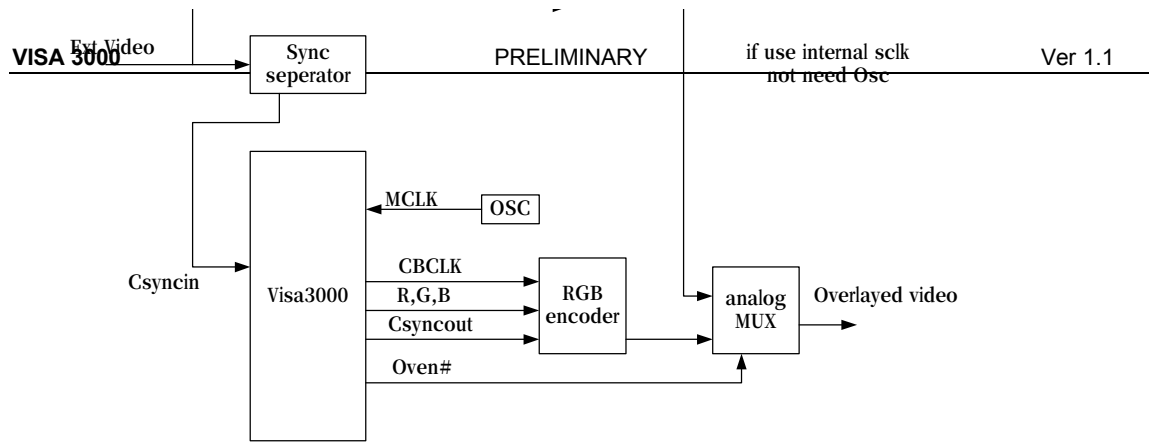


Figure 4.3-2 RGB encoder. interface at No PLL

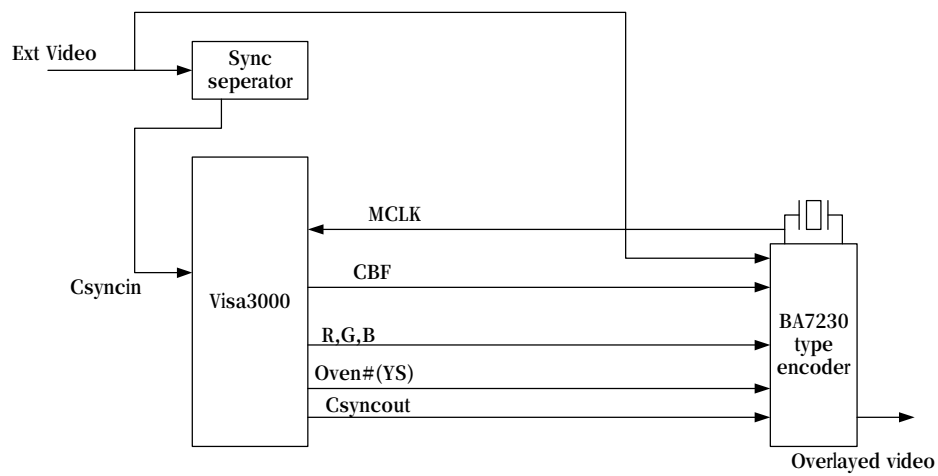


Figure 4.3-3 BA7230 type interface No PLL mode

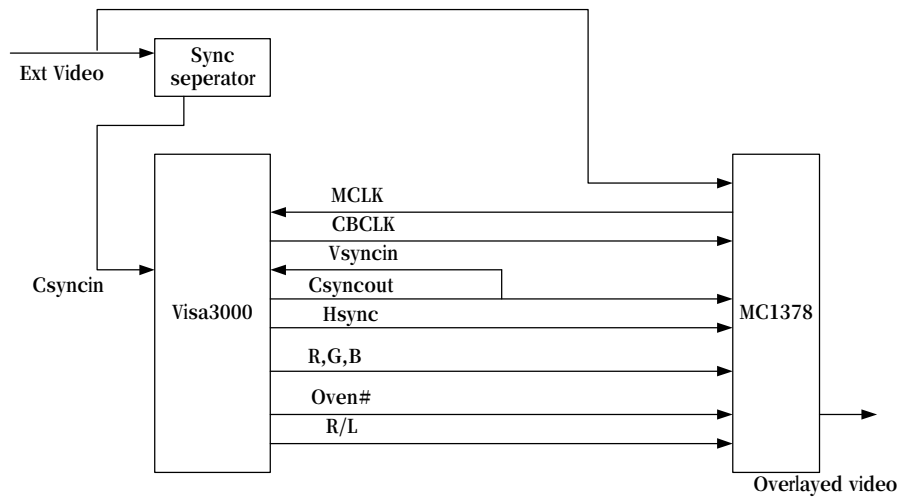


Figure 4.3-4 MC1378 interface

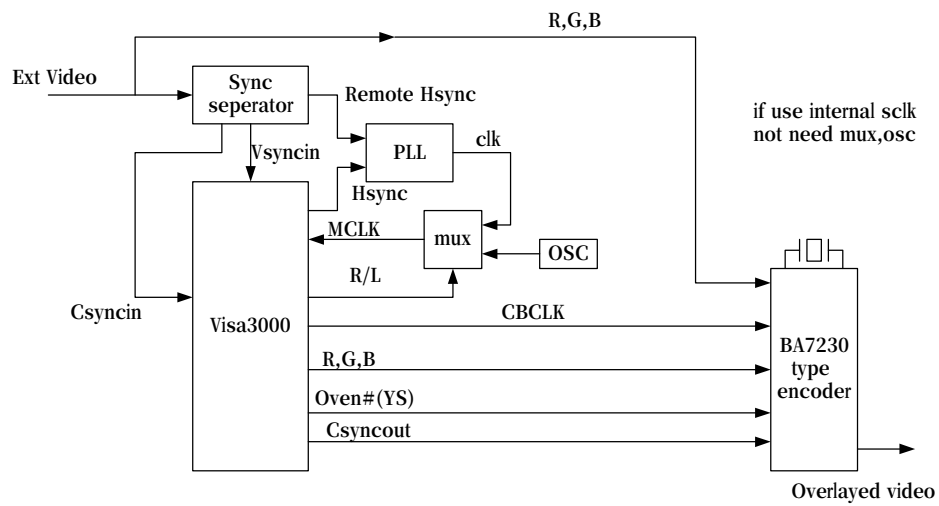


Figure 4.3-5 BA7230 type interface PLL mode

4.3.2 Video Clock

Video의 내부 동기신호와 pixel data 흐름의 동기화는 모두 VCLK에서 시작된다. 내부에서는 VCLK를 분주하여 CBCLK와 DCLK를 만든다.

CPCLK는 내부에서 CSync를 만드는 clock으로 사용되며 또한 이것의 4분주 clock CBCLK는 chip 외부로 출력되므로 MC1378 mode에서는 VCLK, CBCLK 그리고 MC1378이 연결되어 PLL를 형성한다. 특히, DCLK 주파수는 pixel data의 전송속도를 결정하므로 monitor의 수평 해상도를 결정한다.

4.3.2.1 Block Diagram

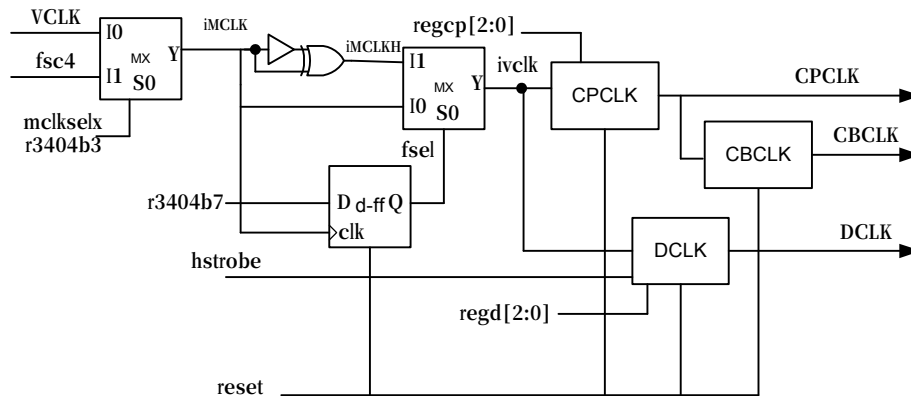


Figure 4.3-6 video clock

iMCLK는 2개(I0,I1)의경로를 거친다. I1는 2체배된 clock을 나타내고, I0은 원 상태의 iMCLK을 통과시킨다.

d-ff를 둔 것은 r3404b7 변화시 ivclk에 글리치가 발생하지 않도록 하기 위해서이다.

입력되는 MCLK는 50%의 duty 비율을 가져야 한다.

CPCLK 발생기와 DCLK발생기는 각각 다음과 같은 식에 준하여 분주한다.

$$F_{cpclk} = F_{ivclk} / (regcp + 1) \quad \text{단 } regcp = 1, 2, 3, 4, 5, 6, 7$$

$$F_{dclk} = F_{ivclk} / (regd + 1) \quad \text{단 } regd = 1, 2, 3, 4, 5, 6, 7$$

$$F_{cbclk} = F_{cpclk} / 4$$

regcp, regd 값이 0인 경우 cpclk, dclk 1일 때와 같다.

4.3.2.2 Clock Generation

MCLK과 MCLK 2체배 clock을 만들어 mux된 clock이 ivclk이다.

ivclk으로 cpclk와 dclk를 만든다. cpclk는 composite sync를 만드는 사용되는 clock으로 ntsc인 경우 반드시 14.3Mhz이고 PAL인 경우 이고 17.7Mhz이다. cpclk를 4분주 하면cbclk가 얻어진다. regd는 dclk를 만드는 분주 값이고 regcp는 cpclk를 만드는 분주 값이다.

clock 을 만들 때 다음과 같은 순서로 고려를 합니다.

1. $F_{ivclk} = (R3404b3 == 1) ? F_{xin_pin} : F_{velk_pin} \Rightarrow$ 어느 핀을 clock source로 사용할 것인가?
2. $F_{iivclk} = (R3404b7 == 1) ? F_{ivclk} * 2 : F_{ivclk}$
3. $F_{epclk} = F_{iivclk} / (R3404b[6:4] + 1) \Rightarrow$ 이 값은 local mode에서 반드시 14.318Mhz 가 되어 합니다.
 $\Rightarrow 3.58\text{Mhz}$ 가 나옵니다.
4. $F_{dclk} = F_{iivclk} / (R3404b[2:0] + 1) \Rightarrow$ 원하는 해상도를 맞춥니다.
5. $F_{sc4clk} = (R3404b[13] == 1) ? F_{xin_pin} : F_{epclk} \Rightarrow$ Csync2VH & CBF_generate block에서 사용

Hline 성분을 보면

$H_{line} = hsync + hsync_back-porch + hdisply + hsync_pront_porch$

표준에서 $H_{line} = 4.7 + 4.7 + 52.6 + 1.5 = 63.5 \text{ us}$

여기서 화면에 보이는 부분 $hdisply = 52.6 \text{ us}$

그러나 실제 보이는 부분은 더 좁고 TV마다 다르다. (보통 48 us 정도로 잡으면 좋을 듯 함)

그러므로

horizontal 해상도 = $48 \times F_{dclk}$

*NTSC local mode에서 다양한 clk 의 예제

Fivclk [Mhz]	Fiivclk [Mhz] (R3404b7)	Fcpclk [Mhz] (r3404b[6:4])	Fdclk [Mhz] (r3404b[2:0])	수평 해상도 by 48 us	수평 해상도 by 52.6 us
14.3	28.6 (1)	14.3 (1)	14.318 (1)	687	753
			9.5454 (2)	458	502
			7.159 (3)	344	377
			5.727 (4)	275	301
			4.773 (5)	229	251
			4.091 (6)	196	215
			3.579 (7)	172	188

Fivclk [Mhz]	Fiivclk [Mhz] (R3404b7)	Fcpclk [Mhz] (r3404b[6:4])	Fdclk [Mhz] (r3404b[2:0])	수평 해상도 by 48 us	수평 해상도 by 52.6 us
21.4773	42.9545 (1)	14.3 (2)	21.4773 (1)	1031	1129
			14.3181 (2)	687	753
			10.7386 (3)	515	565
			8.5909 (4)	412	452
			7.1590 (5)	344	377
			6.1363 (6)	295	323
			5.3693 (7)	258	282

Fivclk [Mhz]	Fiivclk [Mhz] (R3404b7)	Fcpclk [Mhz] (r3404b[6:4])	Fdclk [Mhz] (r3404b[2:0])	수평 해상도 by 48 us	수평 해상도 by 52.6 us
28.6363	28.6363 (0)	14.3181 (1)	14.318 (1)	687	753
			9.5454 (2)	458	502
			7.159 (3)	344	377
			5.727 (4)	275	301
			4.773 (5)	229	251
			4.091 (6)	196	215
			3.579 (7)	172	188
	57.2726(1)	14.3181 (3)	28.6363(1)	1375	1506
			19.0908(2)	916	1004
			14.3181(3)	687	753
			11.4545(4)	550	603
			9.5454(5)	458	502
			8.1818(6)	393	430
			7.1590(7)	344	377

Fivclk [Mhz]	Fiivclk [Mhz] (R3404b7)	Fcpclk [Mhz] (r3404b[6:4])	Fdclk [Mhz] (r3404b[2:0])	수평 해상도 by 48 us	수평 해상도 by 52.6 us
35.7596	71.1591(1)	14.3181 (4)	35.7596 (1)	1716	1881
			23.7197 (2)	1139	1248
			17.7897 (3)	854	936

			14.2318 (4)	683	749
			11.8599 (5)	569	624
			10.1655 (6)	488	534
			8.8949 (7)	427	468

Fivelk [Mhz]	Fiiivelk [Mhz] (R3404b7)	Fcpclk [Mhz] (r3404b[6:4])	Fdclk [Mhz] (r3404b[2:0])	수평 해상도 by 48 us	수평 해상도 by 52.6 us
42.9545	42.9545 (0)	14.3 (2)	21.4773 (1)	1031	1130
			14.3181 (2)	687	753
			10.7386 (3)	515	565
			8.5909 (4)	412	452
			7.1590 (5)	344	377
			6.1363 (6)	295	323
			5.3693 (7)	258	282
	85.909 (1)	14.3(5)	42.9545 (1)	2062	2259
			28.6363(2)	1375	1506
			21.4773(3)	1031	1130
			17.1818(4)	825	904
			14.3181(5)	687	753
			12.2727(6)	589	646
			10.7386(7)	515	565

Fivelk [Mhz]	Fiiivelk [Mhz] (R3404b7)	Fcpclk [Mhz] (r3404b[6:4])	Fdclk [Mhz] (r3404b[2:0])	수평 해상도 by 48 us	수평 해상도 by 52.6 us
50.1136	100.2272(1)	14.3181 (6)	50.1136 (1)	2405	2636
			33.4240 (2)	1604	1758
			25.0680 (3)	1203	1319
			20.0544 (4)	963	1055
			16.712 (5)	802	879
			14.3181 (6)	687	753
			12.534 (7)	602	659

Fivelk [Mhz]	Fiiivelk [Mhz] (R3404b7)	Fcpclk [Mhz] (r3404b[6:4])	Fdclk [Mhz] (r3404b[2:0])	수평 해상도 by 48 us	수평 해상도 by 52.6 us
57.2727	57.2727(0)	14.3181 (3)	28.6363(1)	1375	1506
			19.0908(2)	916	1004
			14.3181(3)	687	753
			11.4545(4)	550	603
			9.5454(5)	458	502
			8.1818(6)	393	430
			7.1590(7)	344	377
	114.5454(1)	14.3181 (7)	57.2727(1)	2749	3013
			38.1818(2)	1833	2008
			28.6363(3)	1376	1506
			22.9090(4)	1100	1205
			19.0909(5)	916	1004
			16.3636(6)	785	861
			14.3181(7)	687	753

예를 들어 510 pixel 에 근접한 해상도를 얻으려면 다음과 같은 경우의 수가 가능합니다.

Fivelk [Mhz]	Fiiivelk [Mhz] (R3404b7)	Fcpclk [Mhz] (r3404b[6:4])	Fdclk [Mhz] (r3404b[2:0])	수평 해상도 by 48 us	수평 해상도 by 52.6 us
14.3	28.6 (1)	14.3 (1)	9.5454 (2)	458	502
21.4773	42.9545 (1)	14.3 (2)	10.7386 (3)	515	565
28.6363	28.6363 (0)	14.3181 (1)	9.5454 (2)	458	502
28.6363	57.2726(1)	14.3181 (3)	11.4545(4)	550	603
35.7596	71.1591(1)	14.3181 (4)	11.8599 (5)	569	624
35.7596	71.1591(1)	14.3181 (4)	10.1655 (6)	488	534
42.9545	42.9545 (0)	14.3 (2)	10.7386 (3)	515	565
42.9545	85.909 (1)	14.3(5)	12.2727(6)	589	646
42.9545	85.909 (1)	14.3(5)	10.7386(7)	515	565
50.1136	100.2272(1)	14.3181 (6)	12.534 (7)	602	659
57.2727	57.2727(0)	14.3181 (3)	11.4545(4)	550	603

57.2727	57.2727(0)	14.3181 (3)	9.5454(5)	458	502
---------	------------	-------------	-----------	-----	-----

그러나 주파수가 높으면 전력 소비가 증가하므로 주파수가 가능한 낮은 쪽에서 찾아야 할 것입니다.
 그래서 35.7596 Mhz 이상은 제외시키면
 14.3181 Mhz 일 때 1 가지
 21.4773 Mhz 일 때 1 가지
 28.6363 Mhz 일 때 2 가지 가능합니다. 이 중에
 21.4773 Mhz 을 추천합니다.

4.3.3Csync Generation

VISA3000의 Csync generator는 각 표준 TV 신호 sync를 발생한다.
 NTSC뿐만 아니라 PAL(B,D,G,H,I,N), PAL(M), PAL (Combi-N)방식도 지원한다.
 이 부분은 remote mode에서는 전혀 필요 없는 부분으로 local mode에서 만 필요 하다. remote mode에서는 외부
 영상에서 동기를 추출하기 때문에 필요가 없는 것이다. 그러나 외부 영상이 없는 환경은 반드시 발생하고 이 때는
 스스로 video sync신호를 발생해서 비디오를 발생 할 수 있어야 한다.
 Csync generator에서는composite_sync와Vsync그리고 Hsync 발생을 위한 Hstrobe 신호를 발생한다.

다음은 국제 TV 신호에 대하여 각 방식에 대한 신호 규격을 간단히 요약한 것이다.

	NTSC	PAL		
		B,D,G,H,I,N	M	Combi - N
Hsync Width	4.7 usec	4.7 usec	4.7 usec	5.0 usec
Front Porch	1.5 usec	1.5 usec	1.75 usec	1.9 usec
Back Porch	4.7 usec	5.8 usec	4.55 usec	4.0 usec
H. Display	52.6 usec	52.0 usec	52.5 usec	53.1 usec
H. total	63.556 usec / 15,734 Hz	64.0 usec / 15,625 Hz	63.556 usec / 15,734 Hz	64.0 usec / 15,625 Hz
H. line/frame	525	625	525	625
Field freq.	59.94 Hz	50 Hz	59.94 Hz	50 Hz
Video Bandwidth	4.2 MHz	I = 5.5MHz B,G,H = 5.0MHz N = 4.2MHz D = 6.0MHz	4.2 MHz	4.2 MHz
Color Burst Frequency	3.579545 MHz	4.433618 MHz	3.575611 MHz	3.582056 MHz
Color Burst	2.51 usec (9 cycle)	2.26 usec (10 cycle)	2.52 usec (9 cycle)	2.51 usec (9 cycle)
CB porch	1.00 usec	1.00 usec	1.00 usec	1.00 usec
Pre-equalize	6 pulses	5 pulses	6 pulses	5 pulses
Low width	2.35 usec	2.35 usec	2.35 usec	2.35 usec
High width	29.43 usec	29.65 usec	29.43 usec	29.65 usec
Serration	6 pulses	5 pulses	6 pulses	5 pulses
Low width	27.1 usec	27.3 usec	27.1 usec	27.3 usec
High width	4.7 usec	4.7 usec	4.7 usec	4.7 usec
Post-equalize	6 pulses	5 pulses	6 pulses	5 pulses
Low width	2.35 usec	2.35 usec	2.35 usec	2.35 usec
High width	29.43 usec	29.65 usec	29.43 usec	29.65 usec

Table 4.3-4 Video Standard

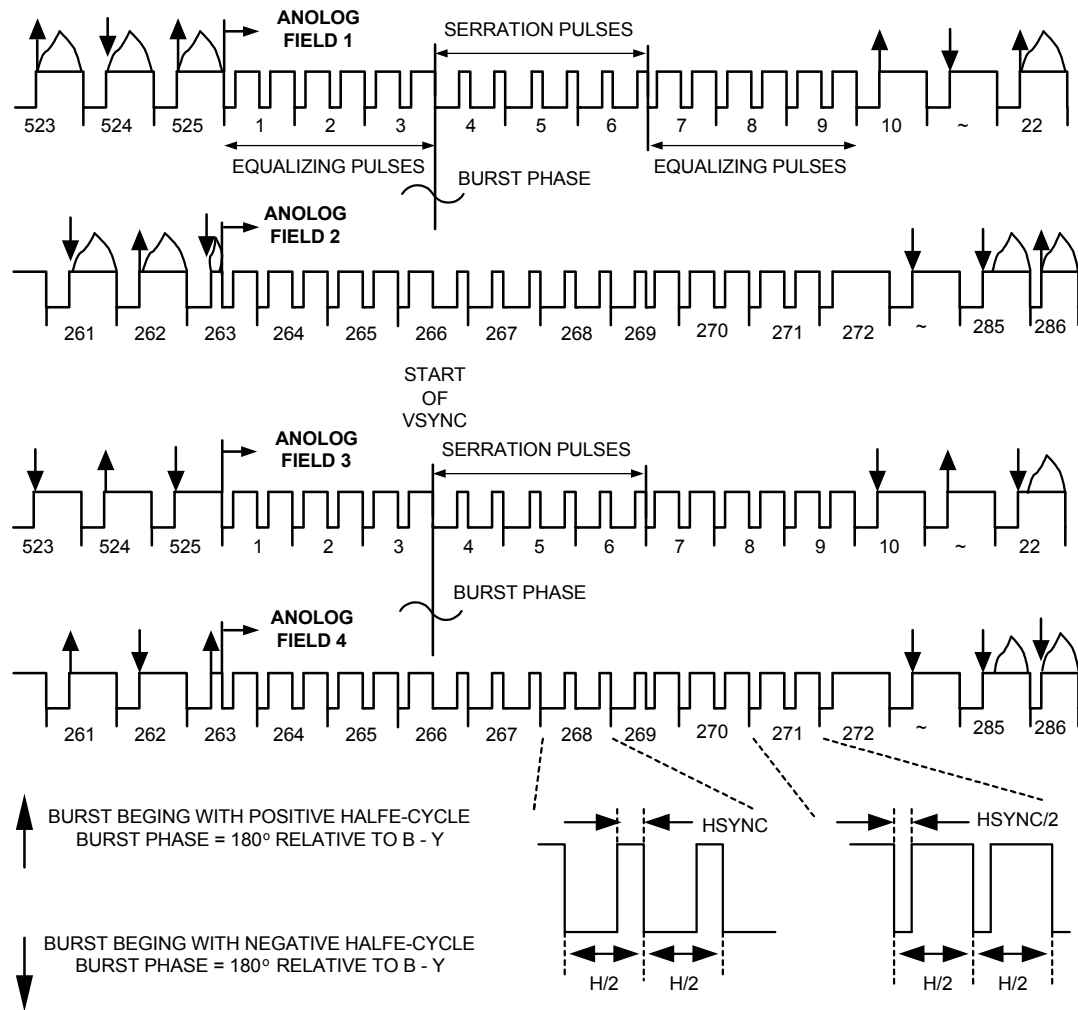


Figure 4.3-7 four-field (M) NTSC Format and Burst Blanking

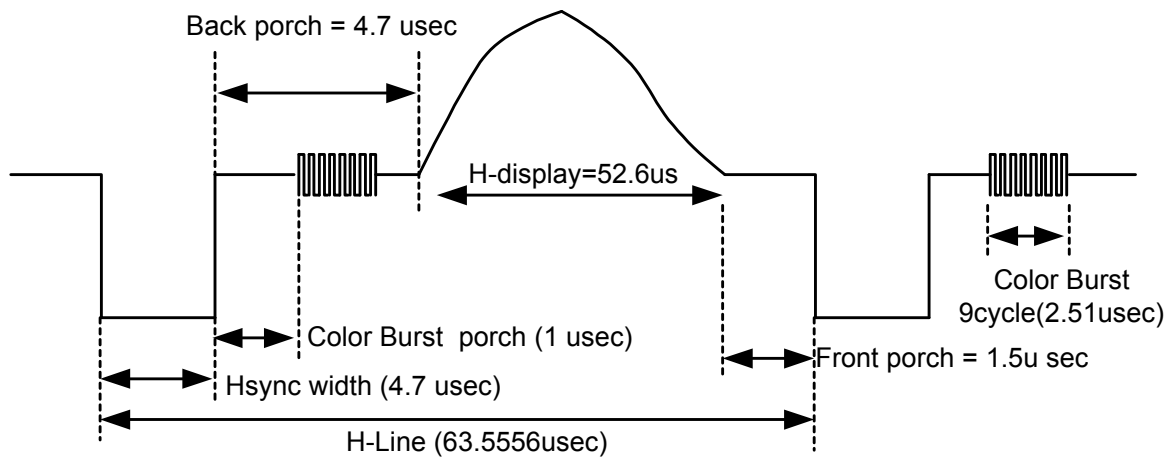


Figure 4.3-8 hsync line (NTSC)

사용자가 원하는 Csync를 발생시키기 위하여 아래와 같은 주파수 및 정보를 레지스터로 설정해야 한다.

cpclk = 14.3Mhz 또는 17.7Mhz, line 은 625/525의 frame H-line수를 결정한다.

Color burst frequency (CBF)는 3.58 또는 4.43방식을 결정 한다.

이렇게 하여 NTSC뿐만 아니라 PAL(B,D,G,H,I,N), PAL(M), PAL (Combi-N)방식도 지원한다.

다음 표는 이들 설정에 관한 표이다.

	Horizontal line수	CBF (Mhz)
NTSC	525	3.58
PAL(B,D,G,H,I,N)	625	4.43
PAL(M)	525	3.58
PAL(combi-N)	625	3.58

Table 4.3-5 NTSC & PAL 설정

4.3.4 Horizontal Sync.

Video 동기신호는 크게 vsync, hsync로 나눌 수 있다. Vsync는 수직 방향 display의 동기를 잡고 hsync는 수평방향 display 동기를 잡는다. H-Line은 다음 그림과 같은 요소를 가진다.

즉 hsync width, hsync back porch, hdisplay, hsync front porch로 되어 있다.

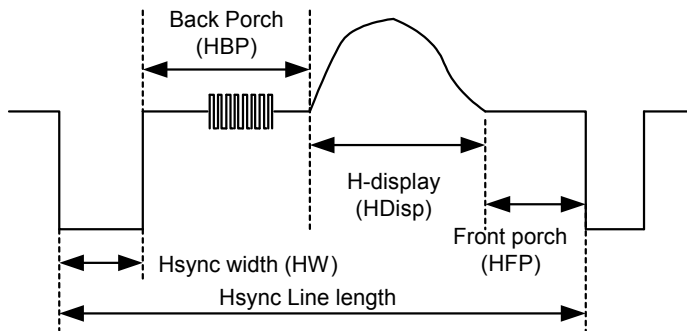


Figure 4.3-9 H-line

Hsync 발생 알고리즘:

Local mode에서는 CSync generator block에서 Hstrobe 받아 Hsync를 발생시키고,
 Remote No PLL mode에서는 CSyncToVH에서 Hstrobe 받아 Hsync를 발생시키고,
 그 외 Remote mode에서는 자체적으로 발생시킨다.

Visa3000에서는 이들의 길이를 Dot clock 단위로 설정하여 H-Line를 조작할 수 있다.

Local mode에서는 CSync generator block에서 공급된 Hstrobe를 시점으로 해서 Hsync Width, Hsync Back porch, H-Display까지 구성하고 다음 Hstrobe까지 Hsync Front porch로 처리한다. 그러나 Remote mode에서는 Hstrobe와 같은 동기신호가 없으므로 Register에 설정된 Hsync Width, Hsync Back porch, H-Display, Hsync Front porch 길이를 반복한다. 그러므로 Local Mode에서는 Hsync Front porch 설정은 필요 없지만 Remote Mode에서는 Hsync Front porch까지 정확히 설정해야 한다.

Q : Remote PLL mode에서 Visa3000에서 발생한 Hsync는 외부 비디오 신호의 Hsync와 똑 같아지는가?

A : rising edge 또는 falling edge 두개 중 하나는 반드시 같아지고 Hsync 주기도 같아진다.

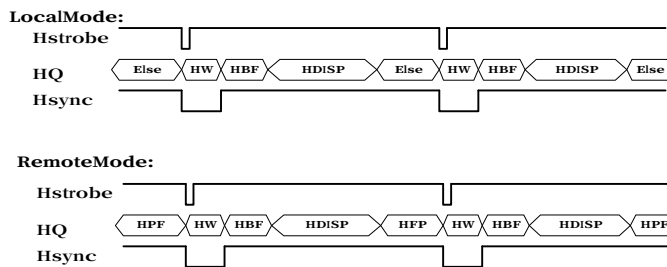


Figure 4.3-10 Hsync at Local & Remote mode

Field Detection 알고리즘:

Hsync generator block에서는 Hsync 신호 발생 외에도 field 판별회로가 내장되어 있다. field를 구분하기 위해서는 Vsync 신호가 필요하다.

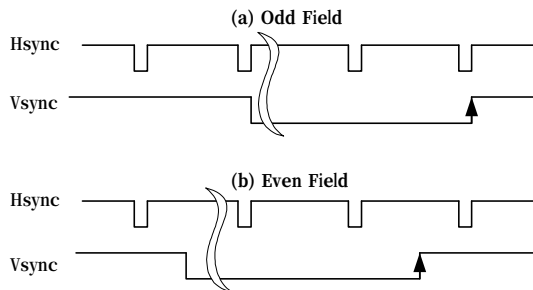


Figure 4.3-11 odd field vs. even Field

위 그림에서 알 수 있듯이 Vsync의 rising edge가 Hsync의 rising edge에 근접해 있으면 Odd field라 하고, Vsync의 rising edge가 H-line의 중앙 부분에 위치해 있으면 Even field라고 한다.

앞에서 서술했듯이 Local mode에서는 CSync generator block이 Vsync를 발생하고, Remote mode에서는 외부 영상신호에서 추출하여 Vsync를 만들어낸다. 그러나 실제 영상신호는 전송 도중에 잡음이나 전송선의 길이에 의한 지연때문에 Vsync의 rising edge의 위치가 정확하지 않을 수 있다. 그래서 Visa3000에서는 window개념을 적용하여 field를 탐지한다. 다음 그림을 살펴보면 이해할 수 있다.

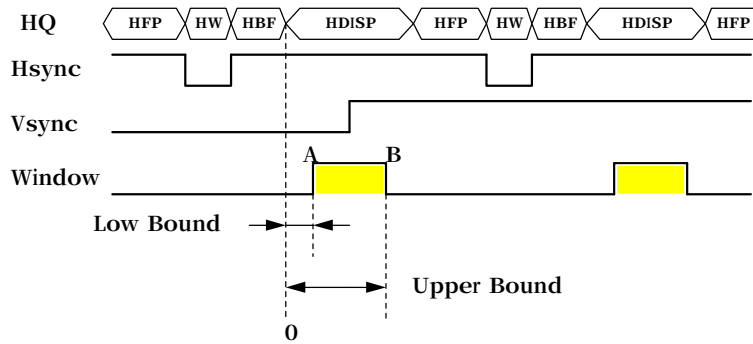


Figure 4.3-12 Field detection window

A점에서 B점 까지를 window내부라 한다. 내부에서 Vsync의 rising edge가 발생하면 even field이고 window 외부에서 발생하면 ,odd field라고 한다. window low bound, upper bound는 Hdisp부터 16pixel을 단위로 하여 설정한다.

예를 들면 A=2, B=8로 설정하면 Hdisp 시작부터 dot clock (Dclk)이 32 cycle되는 위치가 A점이 되고, 128(16x8) cycle이 되는 지점이 B점이 된다.

* Field Detection Window는 Hdisplay내에 존재한다.

4.3.5 Virtual Sync.

Vcnt generator block은 화면에서 세로축으로 영상신호의 보이는 시작점과 끝점을 정의하는 VDisp와 VDisp 시작점부터 Hsync신호가 입력될 때마다 1씩 증가하는 Vcnt[8:0]값을 발생한다. 그리고 Vdisp 끝나는 시점에서 interrupt가 발생한다.

Vsync =0 이면 vcnt=0이고 Vsync =1 이면 hsync rising edge 에서 vcnt는 1씩 증가 하는데 vbpr 값까지 증가하고

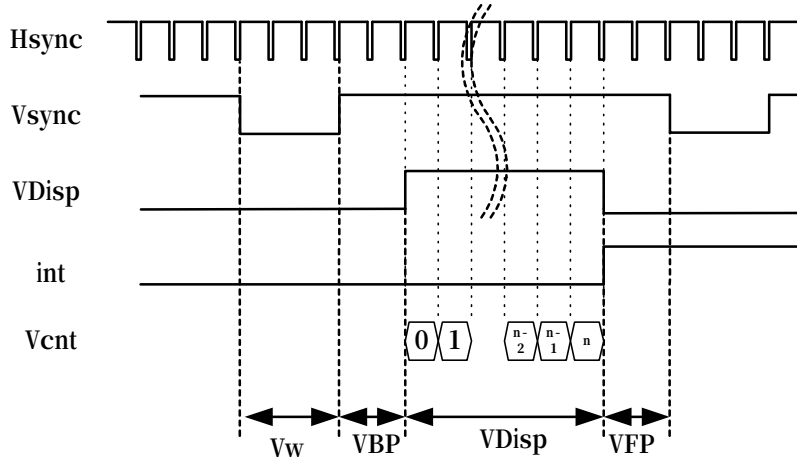


Figure 4.3-13 V-line structure

0으로 다시 reset 되고 vdispr 까지 증가하고 이기간에 vdisp=1, vbint=0을 출력한다.

4.3.6 CBF(Color Burst Frame) Generation

CBF 는 비디오 신호 중 color burst가 있는 구간에 발생하는 pulse 이다. NTSC 또는 PAL encoder중에는 cbf입력을 필요로 하는 경우가 있는데 color burst가 존재하는 위치에 pulse를 주어 외부 영상 신호와 color phase를 맞추기 위한 신호이다. 예를 들면 BA7230를 사용할 때 이 신호는 BA7230의 BFP단자에 입력된다.

다음은 CBF 위치에 대한 T_v 표준이다.

	M/NTSC	M/PAL	B, D, G, H, I, N/PAL
Start of CBF(us)	5.3 ± 0.07	5.8 ± 0.1	5.6 ± 0.1
Length of CBF(fsc cycle)	9 ± 1	9 ± 1	10 ± 1

Table 4.3-6 standard

예를 들면 NTSC신호는 hsync falling edge로부터 5.3 ± 0.07 us에서 CBF가 발생하고 fsc=3.58Mhz clock으로 5.8 ± 0.1 동안 유지된다. 다음은 fsc 의 4배 주파수인 fsc4로 이들을 계산한 값이다

	M/NTSC	M/PAL	B, D, G, H, I, N/PAL
Fsc4 (14.3/17.7) 개수	$75.8886 \pm ?$ $74.8841 \sim 76.8886$	$83.0455 \pm ?$ $81.6136 \sim 84.4773$	$99.3131 \pm ?$ $97.5396 \sim 101.0865$
Selected value	$76=0 \times 4c$	$83=0 \times 53$	$99=0 \times 63$

Table 4.3-7 start of CBF

*NTSC: fsc4=315/22=14.318Mhz *PAL: fsc4=17.734475Mhz

	M/NTSC	M/PAL	B, D, G, H, I, N/PAL
Fsc4 (14.3/17.7) 개수	36 ± 4	36 ± 4	40 ± 4
Selected value	$36=0 \times 24$	$36=0 \times 24$	$40=0 \times 28$

Table 4.3-8 Length of CBF

구현 원리는 Hsync falling edge부터 CBF Starting Point 까지 CBF를 1로 하고 CBF Length 동안 0으로 유지하고 다시 1로 한다. 여기서 사용되는 기본 clock은 fsc4이다.

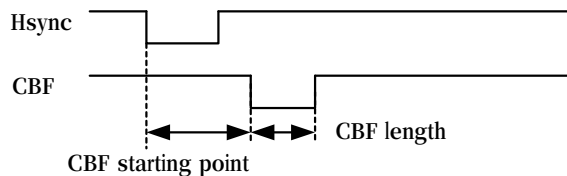


Figure 4.3-154 CBF (Color Burst Frame)

다음은 H-line에서 CBF위치는 Color Burst가 위치하는 곳이다.

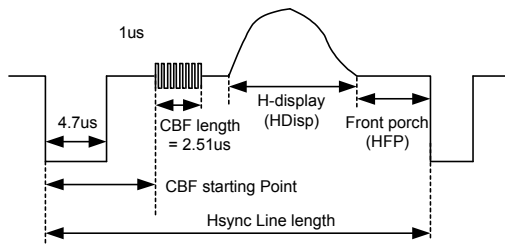


Figure 4.3-15 CBF at H-line

4.3.7 Csync로부터 Vsync/Hsync의 분리

이 block은 Csync를 Vsync, Hsync, even field로 분리하는 역할을 한다. 사용되는 CLK은 fsc4(14.3 또는 17.7 Mhz)여야 한다.

Csync는 Pre-EQ, Serration, Post-EQ, Hsync의 연속으로 이루어진다. 여기서는 Csync중 Hsync부분이 아닌 곳을 Vsync로 하고자 한다. 일반적으로 Hsync width는 4.7usec이다. 그래서 Csync의 low의 길이가 4.7usec인지 아닌지를 판단하여 Hsync, Vsync신호를 만들어낸다. Csync의 Falling Edge부터 low의 길이가 4.7usec이면 Vsync신호를 '0' (High)로 하고 4.7usec이 아니면 '1' (Low=active)로 한다. 그러나 실제 상황에서는 Csync에 포함된 Hsync의 폭이 정확히 4.7 usec 이지는 않다. 긴 경로를 거쳐오거나 잡음이 포함될 수 있기 때문이다. 그래서 window개념을 적용하여 Csync의 길이가 어느 정도 4.7usec에 근사하면 이 신호를 Hsync로 판단하여 Vsync를 '0' (Low)로 하고 4.7usec에서 크게 벗어나면 Vsync를 '1' (High)로 한다.

Csync신호에는 Hsync신호뿐만 아니라 pre/post EQ(EQualization) 신호와 serration 신호, 영상신호도 포함되어 있는 데 EQ신호의 Low 폭은 2.35usec이고 serration의 Low 폭은 27.08usec이다. 이 둘 Low폭의 길이를 가지고서 window 구간을 적절히 설정할 수 있다.

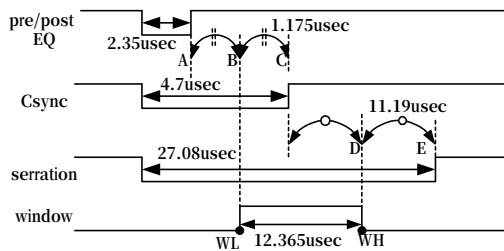


Figure 4.3-16 Checking Hsync from Csync

B는 C와 A를 이등분한 지점이고 D는 C와 E를 이등분한 지점이다. 이 B의 위치와 D의 위치를 각각 window low (WL), window high (WH)의 경계로 하여 window 구간을 결정한다.

따라서 Csync가 low가 되는 시점부터 WL, WH의 위치는 각각 다음과 같이 계산할 수 있다.

$$WL = 2.35 + \frac{4.7 - 2.35}{2} = 3.525 \mu\text{sec}$$

$$WH = 4.7 + \frac{27.08 - 4.7}{2} = 15.89 \mu\text{sec}$$

hstrobe 알고리즘:

hstrobe는 vsync가 아닌 부분의 csync falling detection을 내보내면 된다.

Even field 알고리즘:

ntsc에서 마지막 eq pulse의 high부분이 길면 even field 이고 짧으면 odd field이다 pal에서는 그 반대이다.

4.3.8 External Sync Detector

Sync detector은 외부에 영상신호가 있는지 없는지를 점검한다. 이 것은 노래 반주기에서는 반드시 필요한 기능이다. 보통 외부 영상 신호는 노래방의 분위기를 만드는데 사용하는 것으로 외부 영상의 수평 수직 동기신호에 맞추어 칩이 자막 data를 display한다. 그런데 갑자기 외부 영상 신호가 끊길 수도 있다. 이렇게 되면 동기 정보가 사라지므로 화면이 엉망이 된다. 이 때 Sync detector는 이를 탐지하여 system에 interrupt를 걸어 이 사실을 알려 주어야 한다. system은 Local mode로 전환하여 화면이 깨지지 않도록 한다. 다시 외부영상이 입력되면 Sync detector는 이를 탐지하여 system에 interrupt를 걸어 이 사실을 알려 주어야 한다. system은 다시 remote mode로 전환한다.

외부 영상 신호의 존재 유무를 탐지하기 위해서는 외부 영상 신호에서 분리한 Csync나 Hsync를 CsyncIn 단자에 입력한다. 약 10ms 동안 입력 받은 신호에 변화가 없으면 Ext_Sync detector block은 register EXTVIDEO에 '0'을 출력하고 신호에 변화가 있으면 바로 '1'을 설정한다.

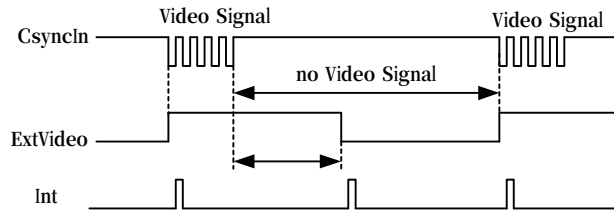


Figure 4.3-177 external Video Signal check

그림의 Interrupt를 보면 ext_sync신호의 edge를 탐지한다.

즉 Interrupt가 걸리는 조건은 영상신호가 존재하다가 없어질 때와 반대로 영상신호가 없다가 다시 생겨났을 때이다. Interrupt가 걸렸을 때 register EXTVIDEO를 읽어보면 영상 신호의 유무를 알 수 있다.

EXTVIDEO='1'이면 영상 신호가 존재하는 것이고, EXTVIDEO='0'이면 영상 신호가 없는 경우이다.

$10\text{msec} = T_{\text{clk}} \times 2^n$ (n은 counter 의 total bit -1)

ex) $\text{clk} = 5\text{Mhz}$, counter는 13bit counter 일 때 csync toggle에 의해 int 발생 후 ExtVideo는 얼마나 1값을 유지 할 수 있을까?

$T_{\text{delay}} = T_{5\text{mhz}} \times 2^{12}$

$T_{\text{delay}} = 200\text{ns} \times 4096 = 819.2\text{usec}$

4.4 Graphic Controller

이 장에서는 VISA3000의 그래픽 데이터의 처리 기능인 Bit Block Transfer, Raster Operation, Sprite 기능에 대해서 설명을 한다. VISA3000은 Bit block transfer (Bitblt) 기능을 위하여 Source 와 Destination Data를 Frame memory로부터 Read하여 저장하는 Buffer가 내장되어 있으며, Bitblt기능을 하지 않을 때는 Frame memory data read를 위한 read cache로 활용된다.

Graphic controller의 구성은 Bit block transfer 기능과 Raster Operation기능이 대표적이며, Bitblt를 활용한 Sprite 기능, Frame memory read 성능을 향상시키는 read cache가 부가적으로 구성되어 있다. Read Cache기능은 Block transfer기능을 사용하지 않을 경우에 자동적으로 Video Frame Data Read시에 동작되어 Frame memory data read의 performance 향상에 기여한다.

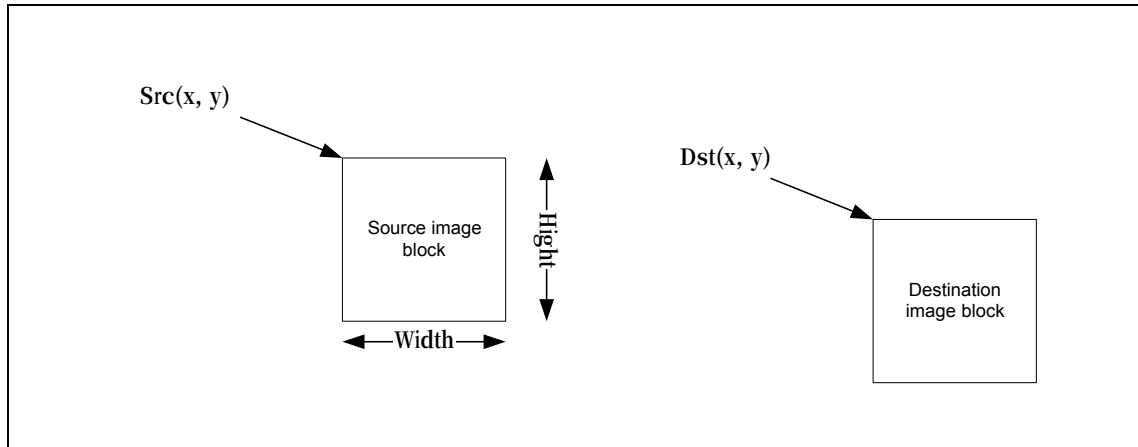


Figure 4.4-1 Source / Destination Image의 지정

위 그림처럼 Src block을 Dst(x,y) 위치로 전송할 경우, 해당 register에 parameter 값을 프로그램한 후 최종적으로 block transfer 명령을 Block Transfer Control Register를 통해 내릴 수 있다. 전송과정에서 Raster operation register 값에 의하여 Source, Destination Data와 Pattern Register값이 logical operation을 거쳐 새로운 data를 만들어 내어 destination 위치에 쓰게 된다.

4.4.1Block Overlapping

VISA3000의 Block Transfer는 Source와 Destination이 겹치는 경우가 Window처럼 많이 사용되지 않을 것이고 source와 destination block이 overlap 되는 경우는 제외시키는 것을 기본으로 함에 따라 모든 경우에 대해 고려 되지 않았다. 기본적인 Addressing은 Top-down, Left-Right 방향으로만 생성되므로 아래와 같은 제약이 따른다.

아래 그림의 ‘비정상적인 이미지 전송 수행’의 경우와 같은 위치의 overlap이 되는 상태의 bitblt를 수행하면 아직 전송되지 않은 source image가 destination data에 의하여 갱신됨에 따라 source image가 원래 선택한 source image와 다르게 되는 결과를 낳게 된다.

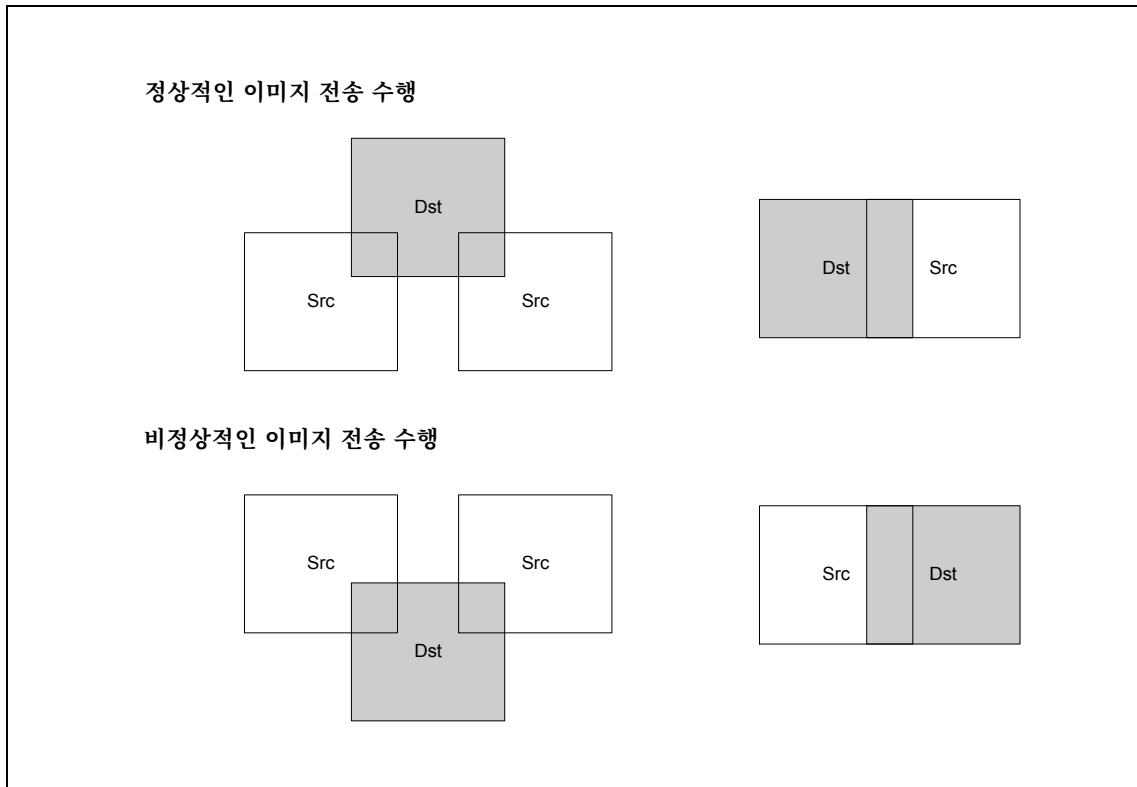


Figure 4.4-2 Bit Block Transfer에서의 Source와 Destination Image Overlap시 수행

4.4.2Sprite

Bitblt register들의 값들은 메모리 영역에 batch file처럼 저장해 놓음으로써, bitblt시 매번 발생하는 Interrupt를 batch로 모두 수행한 후 한번만 발생토록 한다. 이렇게 저장되어 있는 bitblt register table의 시작번지는 Sprite register에 program 되어야 하며, 필요에 따라 sprite register의 global ROP value를 setting함으로써 block의 개별적인 ROP 값을 무시하고 전체적으로 동일한 ROP를 수행할 수 있다. Sprite register의 bit 0를 setting함으로써 sprite 기능이 시작된다. (Sprite control register page 참조)

Sprite 기능 수행 단계

1. Sprite reg.에 지정된 base address로부터 8 Dword의 Data를 Src/Dst Buffer에 읽어온다.
2. 읽어온 Data를 모두 Register에 동시 Write. Check EOT.
3. Setting 된 값에 따른 BitBlit 수행.
4. EOT가 아니면 Sprite base address + 32 후, Step 1부터 수행.
EOT면 Interrupt 발생 후 종료.

만일 Sprite 수행 중에 Sprite active bit이 Disable 된다면, 현재 수행중인 Block의 전송을 마치고 EOT와 상관없이 Interrupt 발생하고 종료한다.

4.4.3Frame Memory Read Data Cache

VISA3000은 Frame memory data read의 성능을 향상시키기 위하여 Bitblt 기능을 수행하지 않을 때 사용하지 않는 Destination buffer를 활용하여 32bit x 4 size를 갖는 simple read cache controller가 내장되어있다.

처음 메모리 read시에는 address[23:4]이 tag로써 저장되며 32bit x 4 개의 data가 destination buffer에 읽혀오며 address[3:2]에 의하여 선택된 32bit data가 CPU로 보내진다. 다음에 read command가 들어올 경우는 현재의 address와 tag를 비교하여 hit가 될 경우 외부로부터 data를 읽지 않고 이미 읽어온 data를 선택해 보내게 된다. 만일 bitblt 명령이 수행되거나 tag address와 같은 address로 memory write가 일어난다면 dirty bit이 setting 되어 다음 번에 수행되는 memory read시에는 무조건 frame memory로부터 32bit x 4의 data를 읽어오는 cycle이 발생된다.

* Page 0, Page 1 Overlay & Bitblt Simulation Example

Page 0 영역 : 64K Color Mode



Page 1 영역 : 256 Color Mode



최종 출력 영상



Page 0 내에서 하나의 Source Block Image가 두군데로 Transfer (Move) 하는 명령을 거치고, 두 Page가 Overlay되는 결과임.

Figure 4.4-3 Overlay & Bitblt Simulation Example

4.5 Video & Palette

4.5.1 Pixel Data Path

pixel data path 는 다음 그림처럼 4부분으로 분리 될 수 있다. 이는 VFMC, pre-pixel-data-path, page-block, post-pixel-data-path이다.

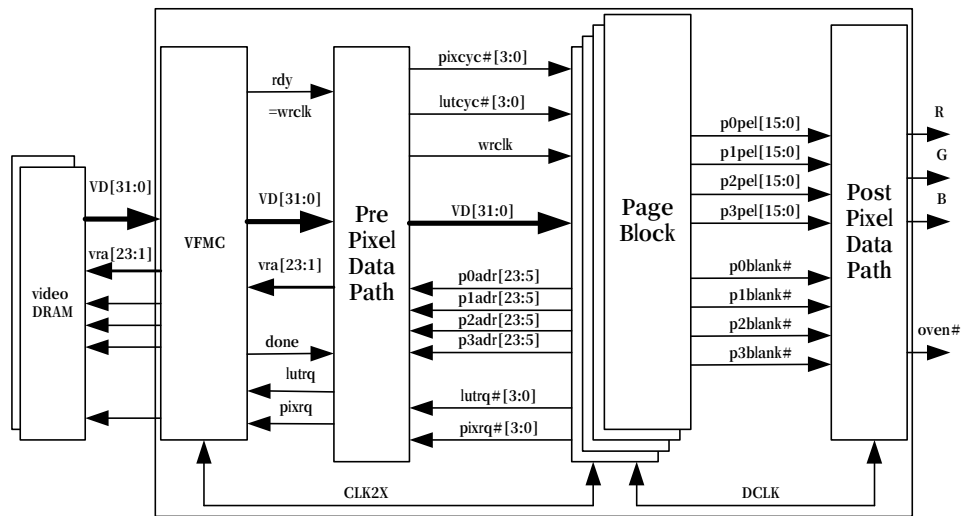


Figure 4.5-1 pixel data path

pixel 은 Video RAM에서 시작하여 VFMC, pre-pixel-data-path, page-block, post-pixel-data-path 경유하여 최종 analog 신호 R,G,B로 출력된다.

VGA에서 video RAM을 monitor로 일대일 대응시키는 구조에 비해 visa3000에서의 display단위는 line이다. 이것은NTSC 신호에서 주사선이 왼쪽에서 오른쪽으로 주사해 가면서 한 line을 그리는 길이와 같다. 그러므로 Hsync가 발생할 때마다. 새로운 line이 시작된다.

Visa3000은 graphic data를 처리하기 위해 line 단위로 화면을 제어한다. 그래서 매 line 마다 LUT data 32bit header를 VFDRAM에서 읽어온다. LUT header는 현재 line의 LBP(Line Back Porch), PIXA(line pixel start address), En(line display enable), Xn(display expand)등의 정보를 포함하고 있다. LUT cycle은 Hsync falling edge에서 발생하고 그 후로 pixel read cycle이 시작되는데 위치는 LUT header에서 지정한 pixel start address에서부터 이다.

VFDRAM에서 읽어 들이는 32bit LUT data 형식은 다음과 같다.

B[31:22]	b[21:3]	B2	b[1:0]
≡ LBP[9:0] LineBack Porch	≡ PIXSA[23:5] line pixel start address	En	Xn[1:0]

Table4.5-1 LUT Data Format

Line Back Porch (LUTB[31:22]) : Hsync rising Edge부터 Pixel data가 display에 나타나기까지의 간격 단위는 dclk이다.

PIXSA (LUTB[21:3]) : Pixel data의 시작 address[23:0] 중 address[23:5]를 나타낸다

LUTB[2] - 현 line의 Display Enable / Disable를 나타낸다.

When “1”, Display Enable

When “0”, Display Disable

LUTB[1:0] Line 수평확장(EXpand)

when “00”, x1수평확장 없음

When “01”, x2 (2개의 DCLK에 1개의 Pixel data)

When “10”, x3 (3개의 DCLK에 1개의 Pixel data)

When “11”, x4 (4개의 DCLK에 1개의 Pixel data)

Hsync가 발생할 때 page block은 LUT Data를 VFDC에 요구하게 된다. VFDC는 이중 우선 순위가 높은 것부터 처리한다. page 3가 제일 높고 page 0가 제일 낮다. 우선 순위가 결정되면 LUT_CYCLE이 시작되는 데 이것은 LUT address를 발생하고, VFDRAM에서 LUT data 32bit를 읽어와서 page block내의 관련 flag에 latch하는 것이다.

	Page 0	page 1	page 2	Page 3
LUT ADDR [23:12]	R3c04B[11:0]	R3c14B[11:0]	R3c24B[11:0]	R3c34B[11:0]
LUT ADDR [11:3]	VCNT[8:0]	VCNT[8:0]	VCNT[8:0]	VCNT[8:0]
LUT ADDR2	R3400B3 ⊕ even field	R3400B3 ⊕ even field	R3400B3 ⊕ even field	R3400B3⊕even field
LUT ADDR1	From VFDC Core	From VFDC Core	From VFDC Core	From VFDC Core

*LUT ADDR1은 16BIT BUS WIDTH에서만 사용됨

Table 4.5-2 LUT address generation

VFDRAM의 bus width가 16bit이면 2번의 VFDRAM access가 필요하고 32bit이면 한 번으로 마무리된다.

LUT 주소는 위 표와 같이 발생된다. page에 따라 다른 base address만 가지며 나머지 주소는 같다. base address는 register를 설정하여 지정할 수 있다.

LUT data는 32bit이므로 double word address로 처리하면 간편하다. 이 때 address bit로는 LUT ADDR[23:2]만 관계된다. 그래서 LUT ADDR 2가 '1'이면 홀수 double word Address이고, '0'이면 짝수 double word Address이다.

NTSC mode인 경우에, even field에서 LUT의 짝수 address를 사용하고자 하면 R3400B3='1'로 설정한다. even field가 홀수 address를 원하면 R3400B3='0'으로 설정한다. 화면에서 두 개의 field가 서로 interlace되어 있듯이 각 field에 사용될 LUT data도 Address처리 관점에서 보면 interlace되어 있다. 이는 시간적으로 짝수 double word Address의 LUT data가 모두 처리되고 홀수 double word Address의 LUT data가 처리되기 때문이다. R3400b3설정에 따라 그 반대의 순서로 처리될 수 있다.

다음 그림은 LUT cycle과 pixel cycle, display timing과의 관계 및 VFDRAM과의 대응관계를 나타내었다.

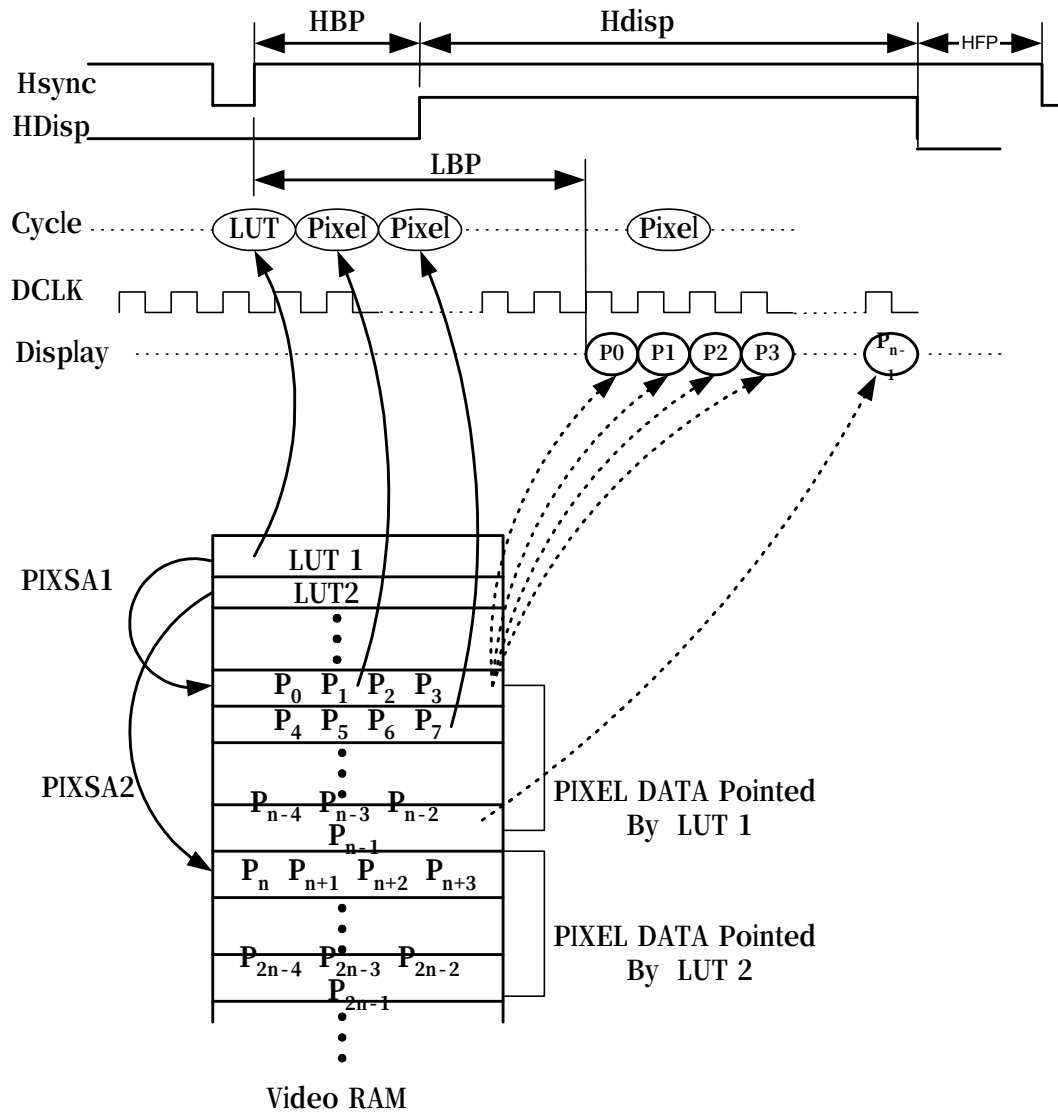


Figure 4.5-2 LUT at H-line

Hsync의 falling edge에서부터 LUT cycle이 시작된다.

LUT cycle이 시작되면 LUT data를 읽어 들어서 한 line에 대해서 분석을 한다. 여기에는 line back porch, pixel data가 저장되어 있는 시작 주소, 그리고 현 line의 display 가능 여부 및 수평 확장에 대한 정보를 읽어서 처리를 한다. pixel data가 저장되어 있는 시작 주소부터 pixel data를 읽어서 지정된 LBP만큼 delay된 후에 순서적으로 화면에 display한다.

다음은 일반적인 Video memory map을 보인 것이다.

크게 LUT table과 pixel data로 나뉜다. LUT table은 page마다 하나씩 필요하므로 4개의 page LUT로 나뉜다. color mode와 해상도에 상관없이 하나의 page LUT는 32bit x 480 만큼의 memory 용량을 차지한다. pixel data도 4부분으로 나뉜다. 한 page pixel data는 LUT pointer 지정에 의해 480개의 H-line pixel data를 가진다. 그림 예에서는 256 color mode 이므로 하나의 H-line pixel data는 640x8bit의 용량이다.

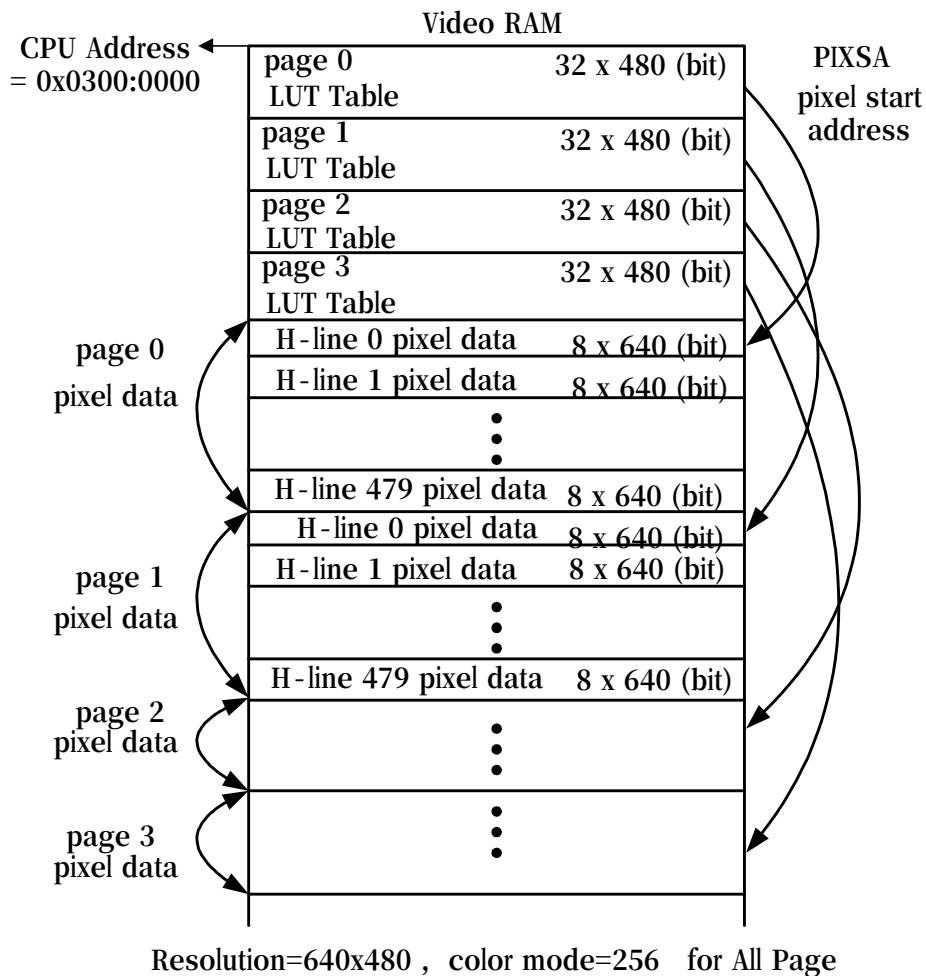


Figure 4.5-3 Typical Video Memory Map

LUT cycle이 종료되면 이어서 pixel cycle이 시작된다.

pixel cycle은 DRAM access 방식 중에서 page mode로 동작하는 한 pixel cycle에서 32 x 8 bit를 읽어 들인다.

4.5.2 VFMC

VFMC (Video Frame Memory Controller)는 VFDRAM과의 interface를 총괄하고 있다.

interface에는 *Video_DRAM_refresh*, *CPU_interface*, *LUT_cycle*, *Pixel_cycle*, *Blit_cycle*이 있다.

여기서는 관련된 *LUT_cycle*, *Pixel_cycle*을 처리한다.

4.5.3 Pre Pixel Data Path

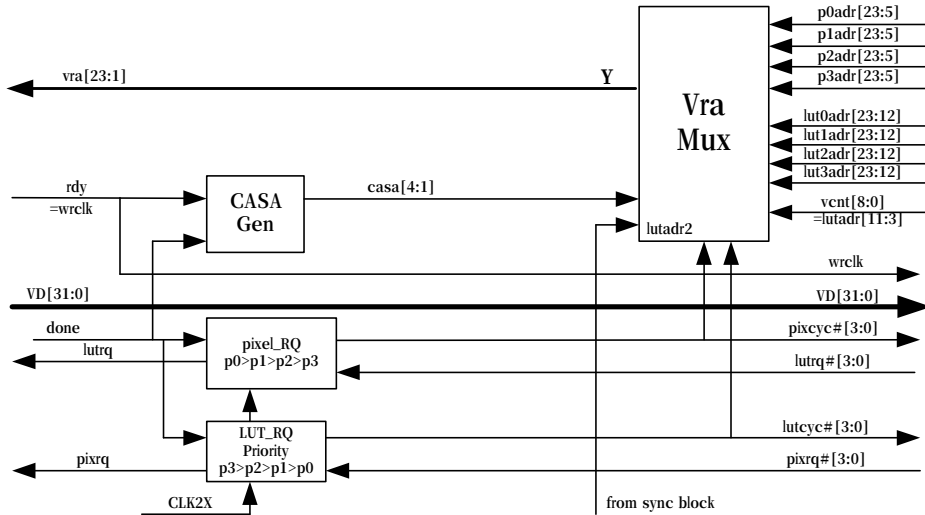


Figure 4.5-4 Pre-pixel data path

prepixel data path 는 lut 나 pixel request 가 발생하면 우선순위에 따라 하나의 page를 선택하여 준다. 그리고 현 cycle 에서 lut data 나 pixel data를 읽을 video dram address를 발생한다. 16bit video dram 을 사용한 경우 casa gen block에서 casa1 을 사용하고 32bit video dram 을 사용한 경우는 이를 사용하지 않는다.

LUT data를 읽기 위해 사용되는 address구조는 다음 그림과 같다.

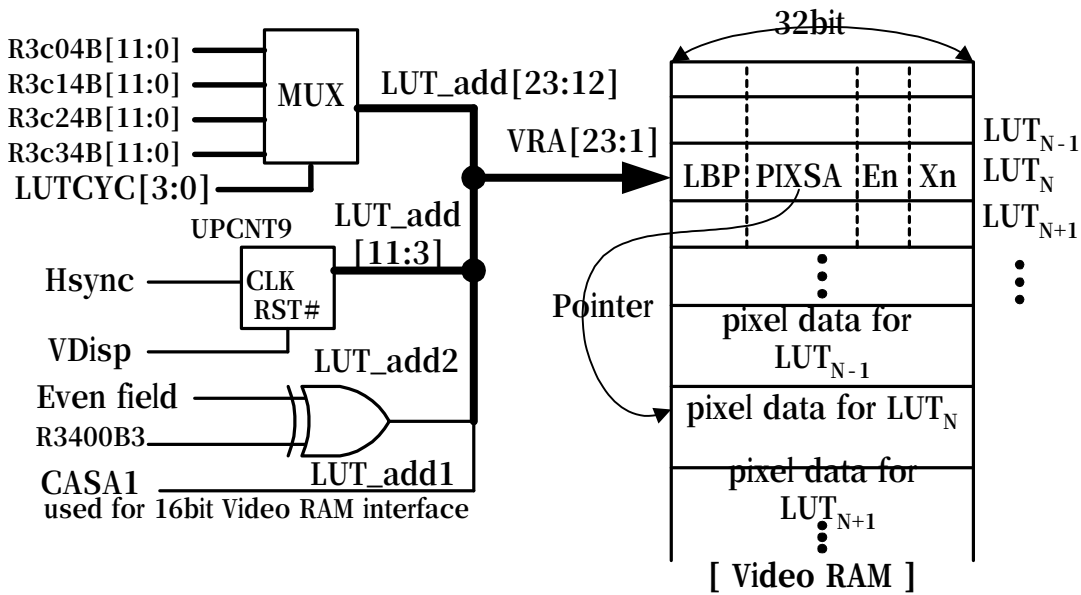


Figure 4.5-5 LUT Address

pixel cycle에서는 page block에서 공급되는 pixa[23:5]를 casa gen block에서 발생되는 casa[4:1]을 사용한다.

4.5.4 Page Block

하나의 Page block 은 다음과 같이 이루어진다.

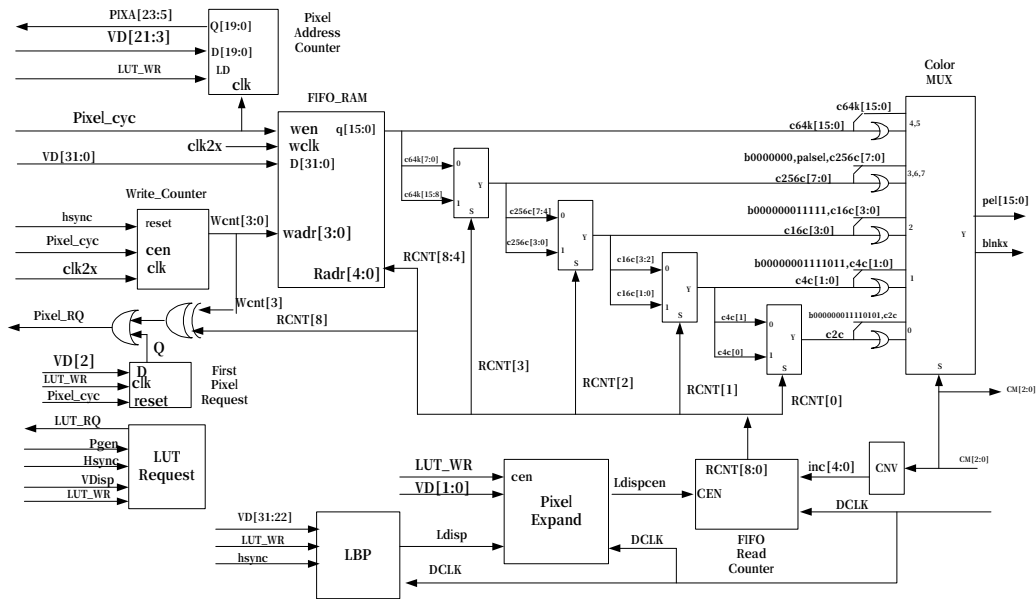


Figure 4.5-6 Page block Diagram

page block에 data 가 입력되는 것은 lut cycle과 pixel cycle인 경우 이다. 이는 궁극적으로 clk2x에 동기 되는 신호이다. 출력은 dclk에 동기 되어 출력된다. 다음 그림은 이들의 timing 도 이다.

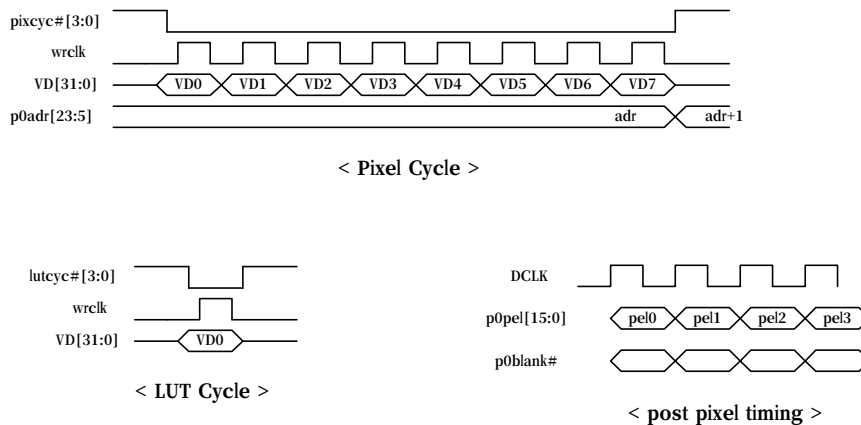


Figure 4.5-7 Page Block Timing Diagram

VISA3000에서는 4 개의 page를 가진다. 이들은 독립적으로 하나의 화면을 발생할 수 있는 processor로서 color mode도 독립적으로 정할 수 있다. 그러나 해상도는 sync block의 dclk에 의해 결정되므로 모두 공통적으로 적용되어 같은 해상도를 가진다.

page block은 하나의 pixel address counter, fifo write counter, fifo ram, fifo read counter, lut request, first fixel request, lbp controller, pixel expansion, color mux로 이루어진다.

lut request 는 현 page 가 enable 이고 vdisp가 active일 때 hsync falling edge가 발생하면 요구하고 lut-wr신호가 발생하면 취소된다.

first fixel request는 lut cycle에서 vd[2] 가 1이면 발생하고 pixel cycle가 발생하면 취소된다. 이는 한 라인에서 처음에 한 번 요구하는 역할을 하는데 다음에 연속적으로 발생될 pixel cycle의 seed역할을 하는 것이다. 다음에 발생하는 pixel cycle request 는 fifo write counter 와 fifo read counter의 최상위 bit 값이 다를 때 요구되는데 first fixel cycle에서 fifo의 절반을 채우므로 자동 요구된다. 그래서 2번째 pixel cycle가 발생하면 fifo ram 은 32bit x 16 = 512 bit로 가득 차 있게 된다. 이때는 최상위 bit 가 0으로 같기 때문에 pixel request 는 발생하지 않고 fifo read counter가 절반을 읽었을 때 발생한다 그러면 또 pixel cycle 이 발생하고 가득 찬 상태가 된다. 이런 식으로 pixel cycle는 계속된다.

pixel address counter 는 lut cycle에서 vd[21:3]을 래치한다 이는 현재 라인의 pixel start address 이다 그리고 pixel cycle에서 video dram address[23:5] 를 공급하고 현 pixel cycle이 끝나면 1 증가 한다. video dram address[4:0]은 공급할 필요가 없다 왜냐하면 pixel cycle 는 무조건 32 byte를 읽어오기 때문이다.

fifo write counter 는 fifo write address[3:0]를 공급한다.

fifo ram은 dual port ram으로 입력 address[3:0] 입력 data[31:0] 출력 address[4:0] 출력 data[15:0] 로 이루어 진다. 용량은 64 byte = 512 bit 이다.

fifo read counter 는 상위 bit[8:4]에서 fifo read address[4:0]를 공급하고 하위 bit 는 fifo ram에서 출력된 data 를 mux 하는데 사용된다. 칼라 mode 에 따라 count 값이 다르다. 1bit color mode에서는 dclk 당 1씩 증가하고 2bit color mode에서는 dclk 당 2씩 증가하고 4bit color mode에서는 dclk 당 4씩 증가하고 8bit color mode에서는 dclk 당 8씩 증가하고 16bit color mode에서는 dclk 당 16씩 증가한다.

lbp controller는lut cycle에서 vd[31:22] 10bit를 래치하고 hsync rise edge 이후 lbp 값만큼 pixel display를 delay 시킨다. 단위는 dclk 이다.

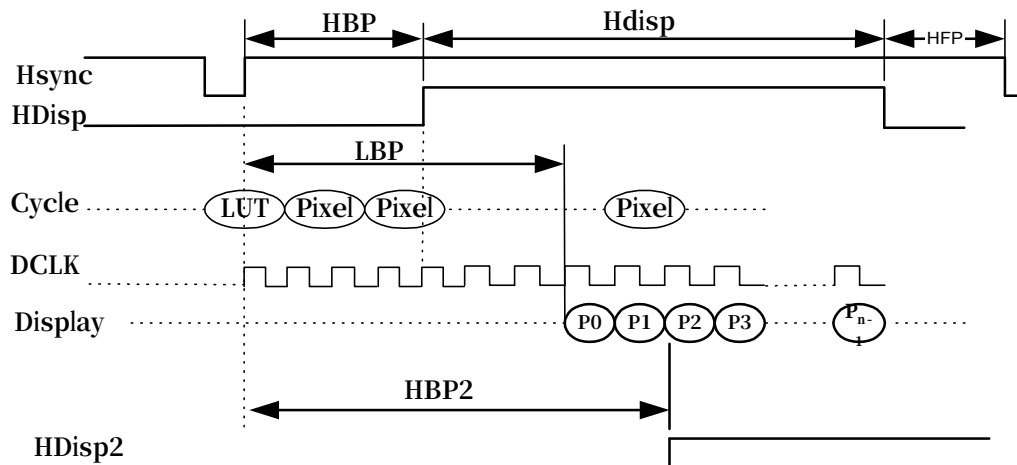


Figure 4.5-8 Line Back Porch

위의 그림에서 LBP(Line Back Porch) = 7DCLK로 HBP=4DCLK로 설정된 경우이다. 만약 HBP(Hsync Back Porch)가 HBP2=9DCLK처럼 설정되면 P0, P1 pixel은 화면에 나타나지 않는다. 그러므로 현 Hline의 첫 pixel이 Display되기 위해서 LBP는 HBP보다 커야 한다. LBP값 조정은 화면의 좌우 스크롤에 응용될 수 있다.

pixel expansion 은 lut cycle에서 vd[1:0]를 래치하고 lbp 이후 pixel display enable이 될 때 fifo read counter의 counter enable 신호를 발생함으로써 수평확장 기능을 구현한다. 0인 경우 항상 count enable이므로 pixel 이 확대 되지 않는다. 1이면 한 dclk 동안 count enable 이 disable 되어 2배 확대 되고 2이면 두 dclk 동안 count enable 이 disable 되어 3배 확대 되고 3이면 세 dclk 동안 count enable 이 disable 되어 4배 확대 된다.

color mux 는 현재의 color mode 에 따라 5 입력 중 하나를 선택하여 최종 출력을 만든다. 64k 즉 16 bit color 에서는 추가 되는 bit 없이 그대로 출력되고 256 즉 8 bit color 에서는 0000 000,palsel 이 추가되는데 palsel 은 palette table 0,1 중 하나를 선택하게 한다. 16 즉 4 bit color 에서는 0000 0001 1111 이 추가 되는데 이는 palette table 0,1 중 palette table 1 을 사용하게 하고 palette address 0x1f0부터 시작하게 한다. 4 즉 2 bit color에서는 0000 0001 1110 11 이 추가 되는데 이는 palette table 0,1 중 palette table 1 을 사용케하고 palette address 0x1d8부터 시작하게 한다. 2 즉 1 bit color에서는 0000 0001 1110 101 이 추가 되는데 이는 palette table 0,1 중 palette table 1 을 사용하게 하고 palette address 0x1ea부터 시작하게 한다.

Color mode에 따라서 출력하는 pixel data 형식은 다음과 같다.

Color mode	Size (Bit/Dclk)	Output Pixel data bit pattern
16Mc	8	0000 0000 R ₇ R ₆ R ₅ R ₄ R ₃ R ₂ R ₁ R ₀
64Kc	16	R ₇ R ₆ R ₅ R ₄ R ₃ G ₇ G ₆ G ₅ G ₄ G ₃ G ₂ B ₇ B ₆ B ₅ B ₄ B ₃
256c	8	0000 000,palsel P ₇ P ₆ P ₅ P ₄ P ₃ P ₂ P ₁ P ₀
16c	4	0000 0001 1111 P ₃ P ₂ P ₁ P ₀
4c	2	0000 0001 1110 11 P ₁ P ₀
2c	1	0000 0001 1110 101P ₀

Table 4.5-3 page0 Color mode & Pixel data Structure

Color mode	Size (Bit/Dclk)	Output Pixel data bit pattern
16Mc	8	0000 0000 G ₇ G ₆ G ₅ G ₄ G ₃ G ₂ G ₁ G ₀
64Kc	16	R ₇ R ₆ R ₅ R ₄ R ₃ G ₇ G ₆ G ₅ G ₄ G ₃ G ₂ B ₇ B ₆ B ₅ B ₄ B ₃
256c	8	0000 000,palsel P ₇ P ₆ P ₅ P ₄ P ₃ P ₂ P ₁ P ₀
16c	4	0000 0001 1111 P ₃ P ₂ P ₁ P ₀
4c	2	0000 0001 1110 11 P ₁ P ₀
2c	1	0000 0001 1110 101P ₀

Table 4.5-4 page1 Color mode & Pixel data Structure

Color mode	Size (Bit/Dclk)	Output Pixel data bit pattern
16Mc	8	0000 0000 B ₇ B ₆ B ₅ B ₄ B ₃ B ₂ B ₁ B ₀
64Kc	16	R ₇ R ₆ R ₅ R ₄ R ₃ G ₇ G ₆ G ₅ G ₄ G ₃ G ₂ B ₇ B ₆ B ₅ B ₄ B ₃
256c	8	0000 000,palsel P ₇ P ₆ P ₅ P ₄ P ₃ P ₂ P ₁ P ₀
16c	4	0000 0001 1111 P ₃ P ₂ P ₁ P ₀
4c	2	0000 0001 1110 11 P ₁ P ₀
2c	1	0000 0001 1110 101P ₀

Table 4.5-5 page2 Color mode & Pixel data Structure

Color mode	Size (Bit/Dclk)	Output Pixel data bit pattern
64Kc	16	R ₇ R ₆ R ₅ R ₄ R ₃ G ₇ G ₆ G ₅ G ₄ G ₃ G ₂ B ₇ B ₆ B ₅ B ₄ B ₃
256c	8	0000 000,palsel P ₇ P ₆ P ₅ P ₄ P ₃ P ₂ P ₁ P ₀
16c	4	0000 0001 1111 P ₃ P ₂ P ₁ P ₀
4c	2	0000 0001 1110 11 P ₁ P ₀
2c	1	0000 0001 1110 101P ₀

Table 4.5-6 page3 Color mode & Pixel data Structure

위의 표에서 16M color mode는 page 0, 1, 2에서 각각 RED data 8bit, GREEN data 8bit, BLUE data 8bit을 갖는다. 그러나 page 3에서는 16M color mode는 지원하지 않는다.

4.5.5 Post Pixel Data Path

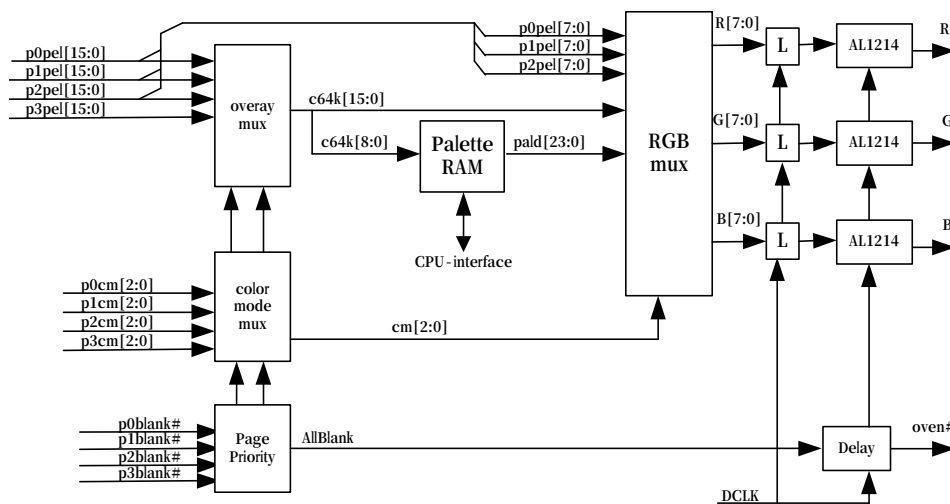


Figure 4.5-9 Post Pixel Data Path

post pixel data path에서 이전의 page block에서 출력되는 blank 신호에 의하여 pixel data와 color mode가 래치된다.

Overlay MUX는 0이 아닌 pixel data를 출력하는 page 0,1,2,3중에서 가장 우선권이 있는 page의 data만 선택하는 부분이다. 우선권은 page3이 가장 순위가 높으며 그 다음 page2, page1, page0이다. 그러므로 화면에 나타나는 영상도 Page3이 가장 위에 Overlay되며 Page0는 가장 아래에 놓인다. 각 page block에서는 pblank#신호를 출력한다. 이 신호들의 의해서 우선권을 갖는 page를 선택한다.

P3 Blank#	P2 Blank#	p1 blank#	p0 blank#	selected page
1	X	x	x	page 3
0	1	x	x	page 2
0	0	1	x	page 1
0	0	0	x	page 0

Table 4.5-7 Priority Page

각 page에서 출력되는 pblank#신호는 '0'이 아닌 pixel 값이 존재하면 '1'이되고, 반대로 pixel data가 모두 '0'이면 pblank#신호도 '0'이 된다. 각 page에 대한 pblank#신호는 dot clock의 한 주기마다 새롭게 발생한다.

Palette RAM은 256c, 16c, 4c, 2c mode에서만 사용되므로 Overlay MUX에서 출력된 data는 palette ram의 address로 사용되어 palette RAM 을 경우하지만 64K와 16M color data는 palette RAM을 거치지 않고 바로 RGB MUX로 입력된다. palette RAM은 9bit의 Address[8:0] 입력을 가지고 24bit의 Data출력을 가진다.

palette RAM Address[8] 이 0이면 palette RAM table 0를 가리키고

palette RAM Address[8] 이 1이면 palette RAM table 1를 가리킨다.

cpu 에서 access 할 때 R4000B8을 1로 설정하고 access 해야 한다. 그리고 access 후에는

R4000B8을 0으로 해야 정상적인 color가 나온다. 또 cpu 가 access 할 때 32bit 단위로 한번에 access 해야 한다.

최하위 byte부터 B[7:0], G[7:0], R[7:0] 이고 최상위 byte 는 사용되지 않는다.

cpu address 와 palette ram data 연결 관계는 다음과 같다.

Cpu address	Palette address	Byte3	Byte2	Byte1	Byte0	Used color mode	table			
0x0180 4400 0x0180 4404 0x0180 4408 0x0180 440C 0x0180 47FC	0x000 0x001 0x002 0x003 0x0ff	Reserved Reserved Reserved Reserved Reserved	R0 R1 R2 R3 R255	G0 G1 G2 G3 G255	B0 B1 B2 B3 B255	256c	Table 0			
0x0180 4800 0x0180 4804 0x0180 4808 0x0180 4ba4	0x100 0x101 0x102 0x1e9	Reserved Reserved Reserved Reserved	r0 r0 r0 r243	g0 g0 g0 g243	b0 b0 b0 b243			256c	Table 1	
0x0180 4ba8 0x0180 4bac	0x1ea 0x1eb	Reserved Reserved	r244 r245	g244 g245	b244 b245					2c , 256c
0x0180 4bb0 0x0180 4bb4 0x0180 4bb8 0x0180 4bbc	0x1ec 0x1ed 0x1ee 0x1ef	Reserved Reserved Reserved Reserved	r246 r247 r248 r249	g246 g247 g248 g249	b246 b247 b248 b249					4c , 256c
0x0180 4bc0 0x0180 4bc4 0x0180 4bc8 0x0180 4bfc	0x1f0 0x1f1 0x1f2 0x1ff	Reserved Reserved Reserved Reserved	r250 r251 r252 r255	g250 g251 g252 g255	b250 b251 b252 b255					16c , 256c

Table 4.5-8 cpu address 와 palette ram data 연결 관계

RGB MUX에 입력되는 data width는 24bit (16Mc, 256c, 16c, 4c, 2c) 과 16bit(64kc)이다. 24bit color mode에서는 R,G,B 각각 8bit이므로 그대로 DAC로 출력하지만, 64K color mode에서는 16bit pixel data가 R:G:B=5:6:5비율로 분리되어 상위 color bit로 사용되고 하위 bit는 0으로 채워진다.

Color mode	RGB data pattern
16M	page 0 (D[7:0]) ----> R[7:0] page 1 (D[7:0]) ----> G[7:0] page 2 (D[7:0]) ----> B[7:0]
64K	P[15:11] ----> R[7:3]&"000" P[10:5] ----> G[7:2]&"00" P[4:0] ----> B[7:3]&"000"
256c 16c 4c 2c	Palette D[23:16]----> R[7:0] Palette D[15:8] ----> G[7:0] Palette D[7:0] ----> B[7:0]

Table 4.5-9 COLOR mode & R,G,B data

Oven#신호는 MC1378과 같은 video encoder칩에 연결되며 현재 출력되는 analog RGB pixel data를 외부 영상에 Overlay하게 하는 신호이다. Oven#신호는 출력되는 Pixel data가 0이 아니면 Active되며 MC1378에서는 Active Low신호로 BA7230에서는 Active High신호로 출력된다. 예를 들어서 Remote mode인 경우 외부에서 영상 신호가 발생하고 VISA3000에서 화면에 display할 자막 문자를 발생한 경우에 MC1378이나 BA7230에서 외부에서 입력되는 영상 신호 위에 문자가 나타나도록 자막 data를 overlay시킨다.

VISA3000에서 출력된 R,G,B신호는 Overlay칩에 입력되지만 Overlay칩 내에서 oven#신호보다 Delay가 심하다. 그래서 Overlay timing을 맞추기 위해서 oven#신호를 400nsec 정도 지연시킬 필요가 있다. VISA3000 은 oven#신호를

최대 7DCLK+3VCLK 까지 delay시킬 수 있다. DCLK 단위로 delay를 setting하기 위해 R3404B[10:8]가 사용되고 VCLK 단위로 delay를 setting하기 위해 R3404B[15:14]가 사용된다

R3404B[10:8]	OVEN# delay value (단위:DCLK)
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Table 4.5-10 Oven# delay setting by DCLK

R3404B[15:14]	OVEN# delay value (단위:VCLK)
000	0
001	1
010	2
011	3

Table 4.5-11 Oven# delay setting by VCLK

RGB MUX에서 출력되는 디지털 data는 D/A Converter AL1214H를 거쳐 Analog신호로 최종 출력된다. D/A Converter는 전류 출력형이므로 출력 핀에 적당한 값의 저항을 달아 사용하는 전압을 맞추어야 한다. D/A의 CLK에 DCLK이 연결되고 20MHz 이하의 주파수를 사용하고 있다. (주의:clk이 두 번 rising edge 가 발생하면 입력된 digital data가 analog로 출력된다.

D/A output Voltage :

In order to change the output swing, use following equation.

$$V_{out} = \{ V_{ref}/(RSET \times 116) \} \times 1023 \times R_{io}$$

Output swing width is a function of V_i ($V_{BIAS}=3.2V$, $V_{ref}=1.4V$), $RSET$ ($1k\Omega$ or 4630Ω), and R_{io} (1600Ω or 37.5Ω). The maximum output swing width is ($2V_{pp}$ or $1V_{pp}$)

5 ELECTRICAL DATA

5.1 Absolute Maximum Rating

Parameter	Symbol	Rating		Units
Supply Voltage	V_{DD}/V_{DDA}	– 0.3 to 3.8		V
DC input Voltage	V_{IN}	3.3 V I/O	– 0.3 to $V_{DD} + 0.3$	V
		5 V-tolerant	– 0.3 to 5.5	
DC input current	I_{IN}	± 10		mA
Operating temperature	T_{OPR}	0 to 70		°C
Storage temperature	T_{STG}	– 40 to 125		°C

5.2 DC Characteristics

Parameter	Symbol	Rating	Units
Supply Voltage	V_{DD}/V_{DDA}	3.0 to 3.6	V
Oscillator frequency	f_{OSC}	10 to 40	MHz
External Loop Filter Capacitance	L_F	820	pF
Commercial temperature	T_A	0 to 70	°C

NOTES

It is strongly recommended that all the supply pins (V_{DD}/V_{DDA}) be powered from the same source to avoid power latch-up.

DC ELECTRICAL CHARACTERISTICS

VDD=3.3V+/-0.3V, VEXT = 5+/-0.25V, TA=0 to 70 Centigrade (In case of 5V-tolerant I/O)

Parameter		Symbol	Conditions	Min	Typ	Max	Unit
High level input voltage	LVC MOS interface	$V_{IH}^{(1)}$	–	2.0	–	–	V
Low level input voltage	LVC MOS interface	$V_{IL}^{(1)}$	–	–	–	0.8	V
Switching threshold		V_T	LVC MOS	–	1.4	–	V
Schmitt trigger positive-going threshold		V_{T+}	LVC MOS	–	–	2.0	–
Schmitt trigger negative-going threshold		V_{T-}	LVC MOS	0.8	–	–	–
High level input current	Input buffer	I_{IH}	$V_{IN} = V_{DD}$	– 10	–	10	μA
	Input buffer with pull-up			10	30	60	
Low level input current	Input buffer	I_{LH}	$V_{IN} = V_{SS}$	– 10	–	10	μA
	Input buffer with pull-up			– 60	– 30	– 10	
High level output voltage	Type B1 to B16(2)	V_{OH}	$I_{OH} = -1 \mu A$	$V_{DD} - 0.05$	–	–	V
	Type B1		$I_{OH} = -1 \text{ mA}$	2.4			
	Type B2		$I_{OH} = -2 \text{ mA}$				
	Type B4		$I_{OH} = -4 \text{ mA}$				
	Type B6		$I_{OH} = -6 \text{ mA}$				
Low level output voltage	Type B1 to B16(2)	V_{OL}	$I_{OL} = -1 \mu A$			0.05	V
	Type B1		$I_{OL} = -1 \text{ mA}$			0.4	
	Type B2		$I_{OL} = -2 \text{ mA}$				
	Type B4		$I_{OL} = -4 \text{ mA}$				
	Type B6		$I_{OL} = -6 \text{ mA}$				
Tri-state output leakage current		I_{OZ}	$V_{OUT} = V_{SS}$ or V_{DD}	– 10		10	μA
Maximum operating current		I_{DD}	$V_{DD} = 3.3 \text{ V}$, $f_{MCLK} = 40 \text{ MHz}$				mA

Table 5-1. DC Electrical Characteristics

NOTES:

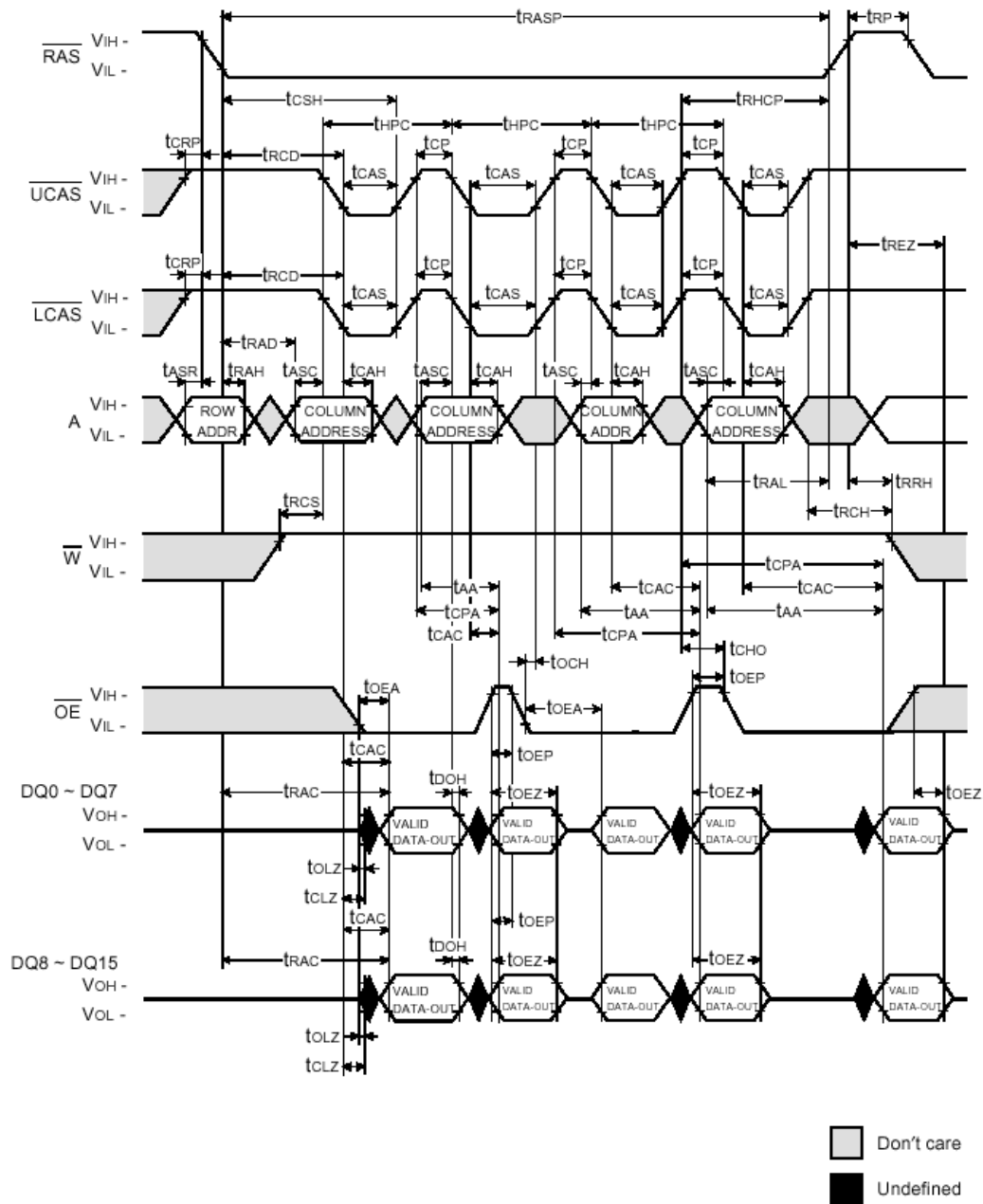
1. All 5V-tolerant input have less than 0.2V hysteresis.
2. Type B1 means 1mA output driver cells, and Type B6/B24 means 6mA/24mA output driver cells.

5.3 AC Characteristics

5.3.1 System Clock Input Waveform

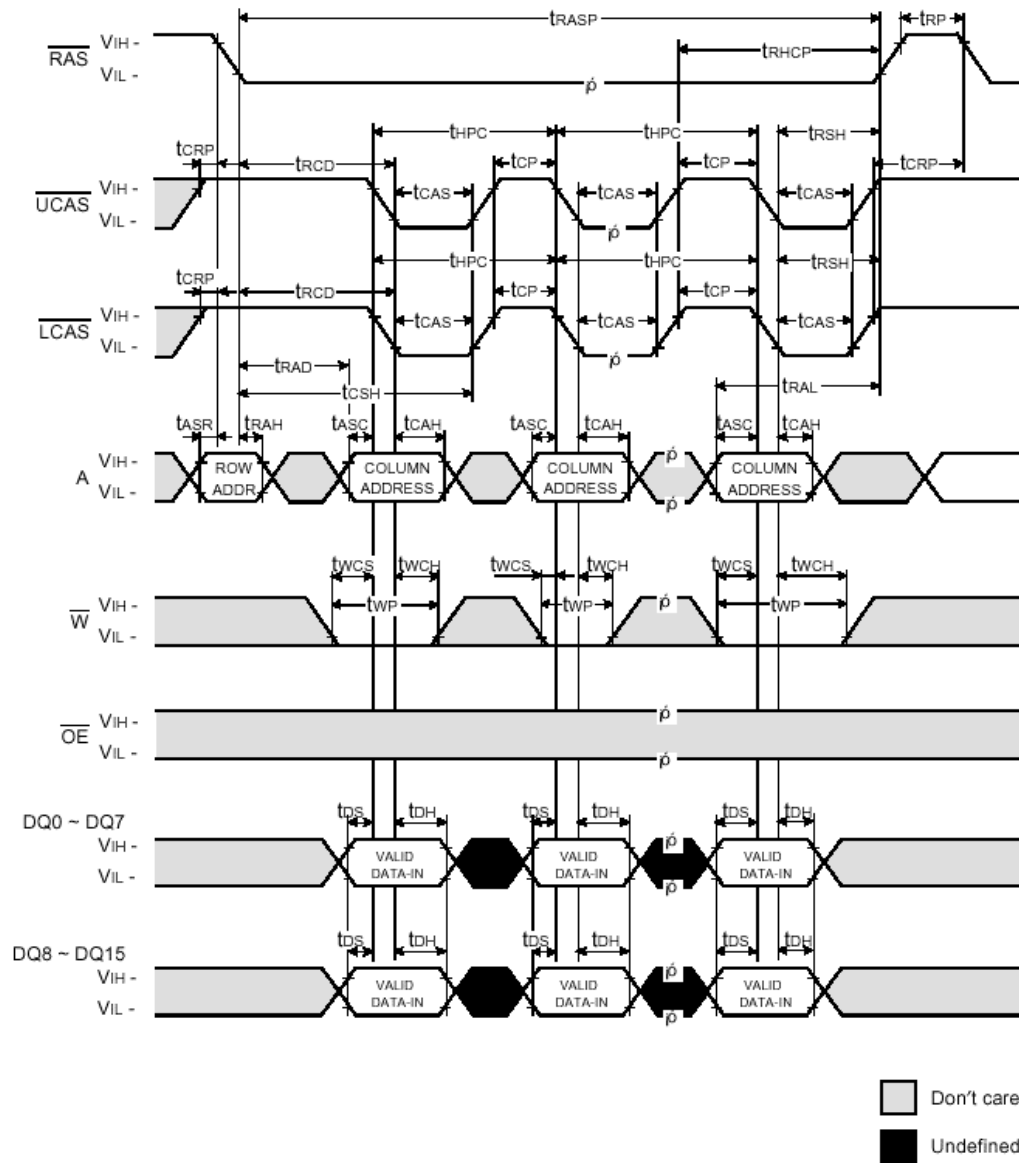
5.3.2 Local Memory(EDO DRAM) Interface

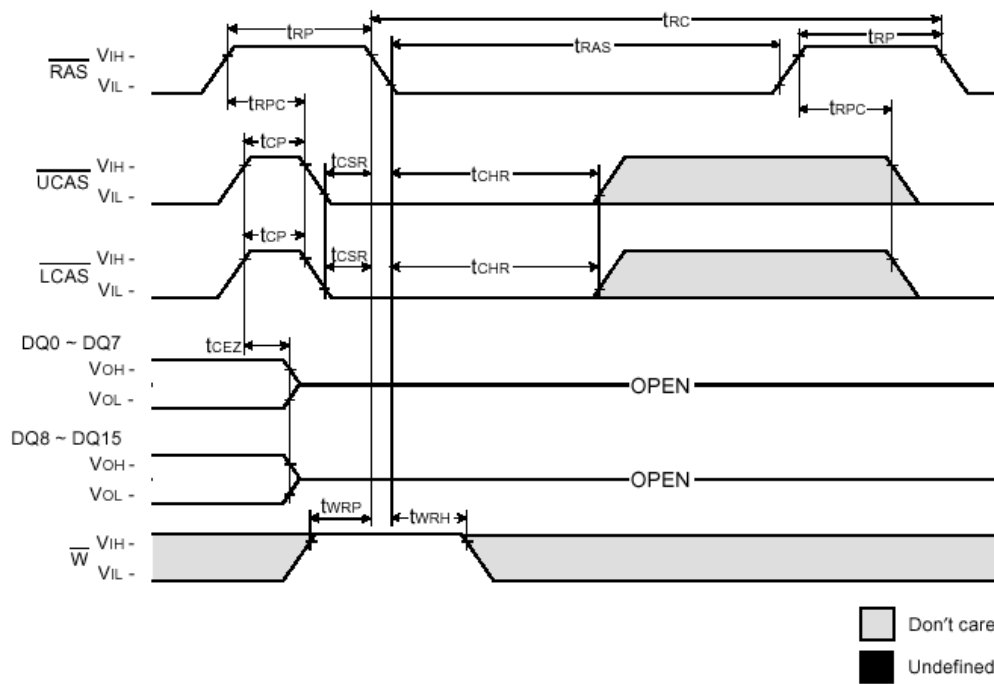
HYPER PAGE MODE WORD READ CYCLE



HYPER PAGE MODE WORD WRITE CYCLE (EARLY WRITE)

NOTE : DOUT = OPEN



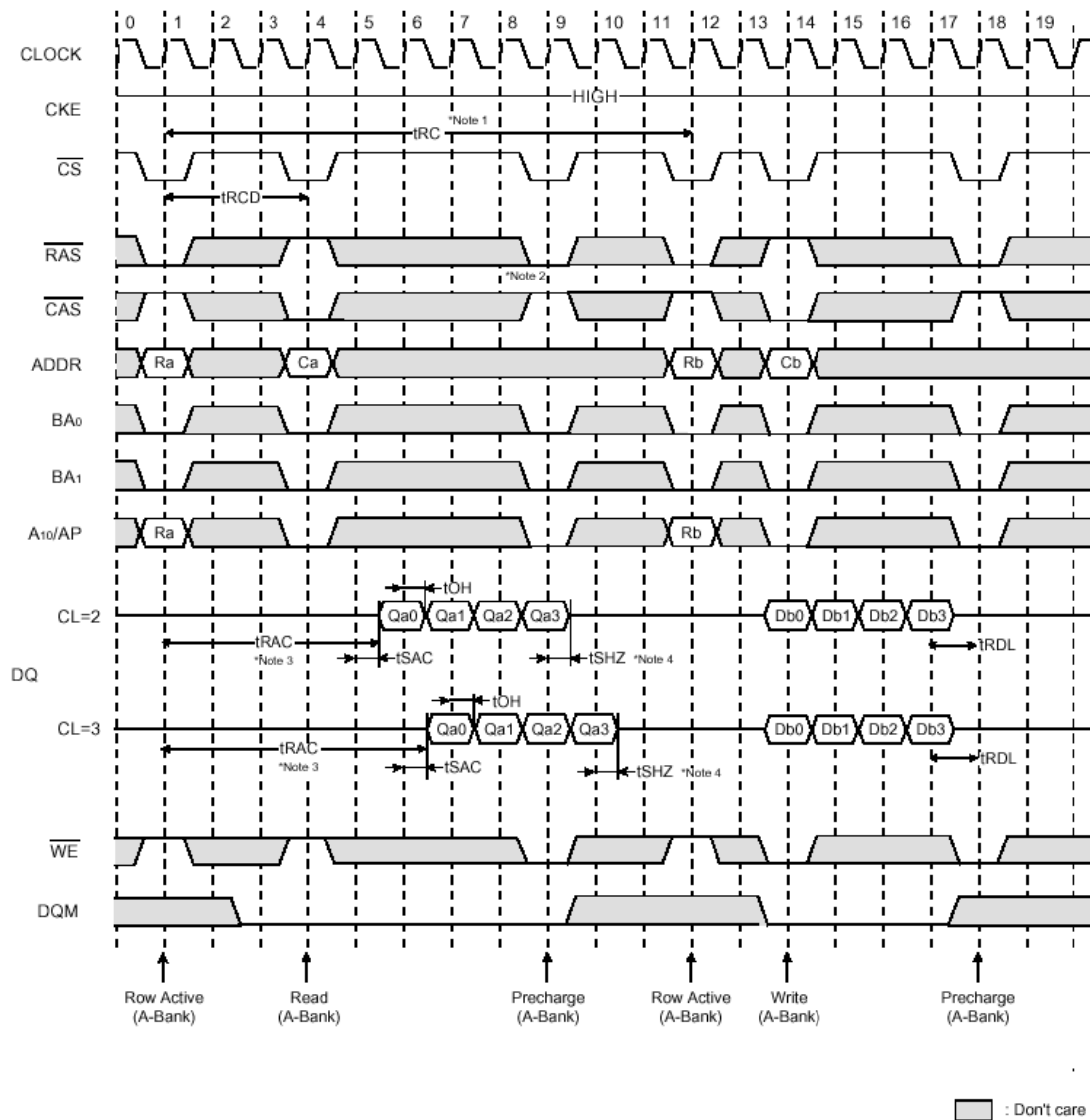
CAS - BEFORE - RAS REFRESH CYCLENOTE : $\overline{OE}$, A = Don't care

AC CHARACTERISTICS (Continued)

Parameter	Symbol	-50		-60		Units	Note
		Min	Max	Min	Max		
Data hold time	t _{DH}	8		10		ns	9,19
Refresh period (4K, Normal)	t _{REF}		64		64	ms	
Refresh period (8K, Normal)	t _{REF}		64		64	ms	
Write command set-up time	t _{WCS}	0		0		ns	7
CAS to $\overline{W}$ delay time	t _{CWD}	30		32		ns	7,15
RAS to $\overline{W}$ delay time	t _{RWD}	67		77		ns	7
Column address $\overline{W}$ delay time	t _{AWD}	42		47		ns	7
CAS set-up time ($\overline{CAS}$ -before- $\overline{RAS}$ refresh)	t _{CSR}	5		5		ns	17
CAS hold time ($\overline{CAS}$ -before- $\overline{RAS}$ refresh)	t _{CHR}	10		10		ns	18
RAS to $\overline{CAS}$ precharge time	t _{RPC}	5		5		ns	
Access time from $\overline{CAS}$ precharge	t _{CPA}		28		35	ns	3
Hyper Page cycle time	t _{HPC}	20		25		ns	20
Hyper Page read-modify-write cycle time	t _{HPRWC}	47		56		ns	20
$\overline{CAS}$ precharge time (Hyper page cycle)	t _{CP}	8		10		ns	14
RAS pulse width (Hyper page cycle)	t _{RASP}	50	200K	60	200K	ns	
RAS hold time from $\overline{CAS}$ precharge	t _{RHCP}	30		35		ns	
$\overline{OE}$ access time	t _{OEa}		13		15	ns	
$\overline{OE}$ to data delay	t _{OEaD}	13		13		ns	
$\overline{CAS}$ precharge to $\overline{W}$ delay time	t _{CPWD}	45		54		ns	
Output buffer turn off delay time from $\overline{OE}$	t _{OEZ}	3	13	3	13	ns	6
$\overline{OE}$ command hold time	t _{OEh}	13		15		ns	
Write command set-up time (Test mode in)	t _{WTS}	10		10		ns	11
Write command hold time (Test mode in)	t _{WTH}	10		10		ns	11
$\overline{W}$ to RAS precharge time ($\overline{C}$ - $\overline{B}$ - $\overline{R}$ refresh)	t _{WRP}	10		10		ns	
$\overline{W}$ to RAS hold time ($\overline{C}$ - $\overline{B}$ - $\overline{R}$ refresh)	t _{WRH}	10		10		ns	
Output data hold time	t _{DOH}	5		5		ns	
Output buffer turn off delay from $\overline{RAS}$	t _{REZ}	3	13	3	13	ns	6,21
Output buffer turn off delay from $\overline{W}$	t _{WEZ}	3	13	3	13	ns	6
$\overline{W}$ to data delay	t _{WED}	15		15		ns	
$\overline{OE}$ to $\overline{CAS}$ hold time	t _{OCH}	5		5		ns	
CAS hold time to $\overline{OE}$	t _{CHO}	5		5		ns	
$\overline{OE}$ precharge time	t _{OEP}	5		5		ns	
$\overline{W}$ pulse width (Hyper page cycle)	t _{WPE}	5		5		ns	
RAS pulse width ($\overline{C}$ - $\overline{B}$ - $\overline{R}$ self refresh)	t _{RASS}	100		100		us	22,23,24
RAS precharge time ($\overline{C}$ - $\overline{B}$ - $\overline{R}$ self refresh)	t _{RPS}	90		110		ns	22,23,24
CAS hold time ($\overline{C}$ - $\overline{B}$ - $\overline{R}$ self refresh)	t _{CHS}	-50		-50		ns	22,23,24

5.3.3 Local Memory(SDRAM) Interface

Read & Write Cycle at Same Bank @Burst Length=4, tRDL=1CLK



OPERATING AC PARAMETER

(AC operating conditions unless otherwise noted)

Parameter		Symbol	Version						Unit	Note	
			-60	-70	-75	-80	-1H	-1L			-10
Row active to row active delay		tRRD(min)	12	14	15	16	20	20	20	ns	1
RAS to CAS delay		tRCD(min)	18	20	20	20	20	20	24	ns	1
Row precharge time		tRP(min)	18	20	20	20	20	20	24	ns	1
Row active time		tRAS(min)	42	48	45	48	50	50	50	ns	1
		tRAS(max)	100						us		
Row cycle time		tRC(min)	60	68	65	68	70	70	80	ns	1
Last data in to row precharge		tRDL(min)	6	7	15	8	10	10	12	ns	2
Last data in to new col. address Delay		tCDL(min)	1						CLK	2	
Last data in to burst stop		tBDL(min)	1						CLK	2	
Col. address to col. address delay		tCCD(min)	1						CLK	3	
Number of valid output data	CAS latency=3	2						ea	4		
	CAS latency=2	-				1					

Notes : 1. The minimum number of clock cycles is determined by dividing the minimum time required with clock cycle time and then rounding off to the next higher integer.

2. Minimum delay is required to complete write.

3. All parts allow every cycle column address change.

4. In case of row precharge interrupt, auto precharge and read burst stop.

AC CHARACTERISTICS (AC operating conditions unless otherwise noted)

Parameter		Symbol	-60		-70		-75		-80		-1H		-1L		-10		Unit	Note
			Min	Max	Min	Max	Min	Max	Min	Max	Min	Max	Min	Max	Min	Max		
CLK cycle time	CAS latency=3	tCC	6		7	1000	7.5	1000	8	1000	10	1000	10	1000	10	1000	ns	1
	CAS latency=2		-		-		-		-		10		12		13		ns	
CLK to valid output delay	CAS latency=3	tSAC		5.5		6		5.4		6		6		6		7	ns	1,2
	CAS latency=2			-		-		-		-		6		7		7	ns	
Output data hold time	CAS latency=3	tOH	2.5		3		2.7		3		3		3		3		ns	2
	CAS latency=2		-		-		-		-		3		3		3		ns	
CLK high pulse width		tCH	2.5		3		2.5		3		3		3		3.5		ns	3
CLK low pulse width		tCL	2.5		3		2.5		3		3		3		3.5		ns	3
Input setup time		tSS	1.5		2		1.5		2		2		2		2.5		ns	3
Input hold time		tSH	1		1		0.8		1		1		1		1.5		ns	3
CLK to output in Low-Z		tSLZ	1		1		1		1		1		1		1		ns	2
CLK to output in Hi-Z	CAS latency=3	tSHZ		5.5		6		5.4		6		6		6		7	ns	
	CAS latency=2			-		-		-		-		6		7		7	ns	

Notes : 1. Parameters depend on programmed CAS latency.

2. If clock rising time is longer than 1ns, (tr/2-0.5)ns should be added to the parameter.

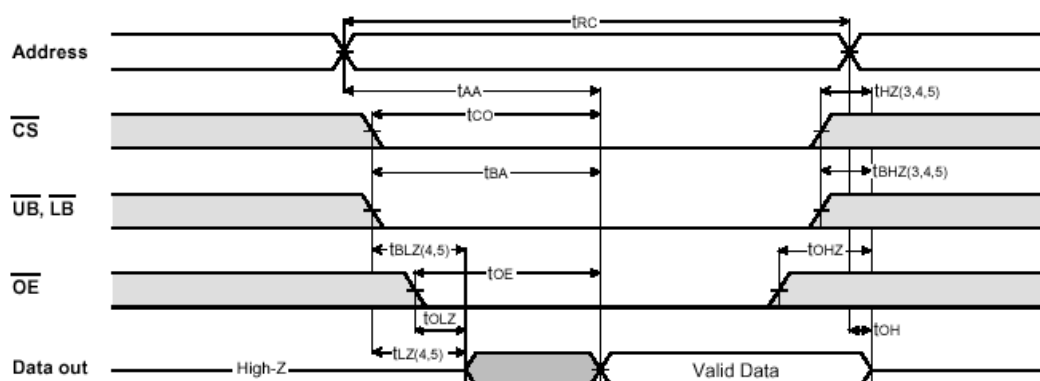
3. Assumed input rise and fall time (tr & tf) = 1ns.

If tr & tf is longer than 1ns, transient time compensation should be considered,

i.e., [(tr + tf)/2-1]ns should be added to the parameter.

5.3.4 Local Memory(SRAM) Interface

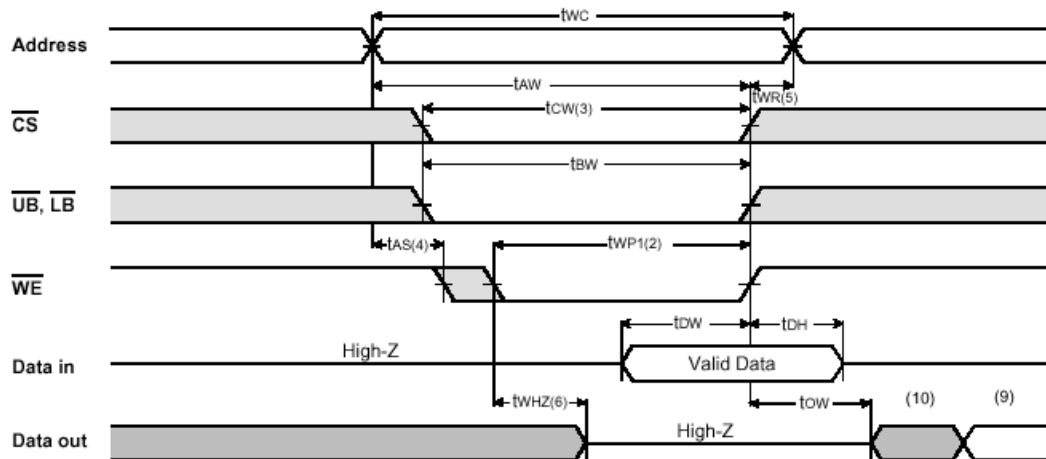
TIMING WAVEFORM OF READ CYCLE(2) ($\overline{WE}=V_{IH}$)



READ CYCLE*

Parameter	Symbol	K6R4016C1B-10		K6R4016C1B-12		K6R4016C1B-15		Unit
		Min	Max	Min	Max	Min	Max	
Read Cycle Time	tRC	10	-	12	-	15	-	ns
Address Access Time	tAA	-	10	-	12	-	15	ns
Chip Select to Output	tCO	-	10	-	12	-	15	ns
Output Enable to Valid Output	tOE	-	5	-	6	-	7	ns
UB, LB Access Time	tBA	-	5	-	6	-	7	ns
Chip Enable to Low-Z Output	tLZ	3	-	3	-	3	-	ns
Output Enable to Low-Z Output	tOLZ	0	-	0	-	0	-	ns
UB, LB Enable to Low-Z Output	tBLZ	0	-	0	-	0	-	ns
Chip Disable to High-Z Output	tHZ	0	5	0	6	0	7	ns
Output Disable to High-Z Output	tOHZ	0	5	0	6	0	7	ns
UB, LB Disable to High-Z Output	tBHZ	0	5	0	6	0	7	ns
Output Hold from Address Change	tOH	3	-	3	-	3	-	ns

* The above parameters are also guaranteed at industrial temperature range.

TIMING WAVEFORM OF WRITE CYCLE(2) ($\overline{OE}$ =Low fixed)**WRITE CYCLE***

Parameter	Symbol	K6R4016C1B-10		K6R4016C1B-12		K6R4016C1B-15		Unit
		Min	Max	Min	Max	Min	Max	
Write Cycle Time	tWC	10	-	12	-	15	-	ns
Chip Select to End of Write	tCW	7	-	8	-	10	-	ns
Address Set-up Time	tAS	0	-	0	-	0	-	ns
Address Valid to End of Write	tAW	7	-	8	-	10	-	ns
Write Pulse Width($\overline{OE}$ High)	tWP	7	-	8	-	10	-	ns
Write Pulse Width($\overline{OE}$ Low)	tWP1	10	-	12	-	15	-	ns
UB, LB Valid to End of Write	tBW	7	-	8	-	10	-	ns
Write Recovery Time	tWR	0	-	0	-	0	-	ns
Write to Output High-Z	tWHZ	0	5	0	6	0	7	ns
Data to Write Time Overlap	tDW	5	-	6	-	7	-	ns
Data Hold from Write Time	tDH	0	-	0	-	0	-	ns
End Write to Output Low-Z	tOW	3	-	3	-	3	-	ns

* The above parameters are also guaranteed at industrial temperature range.

Video Interface
 Frame Memory(EDO DRAM) Interface
 Frame Memory(SDRAM) Interface

6 PROGRAM GUIDE

6.1 System Initialize & Software Flow

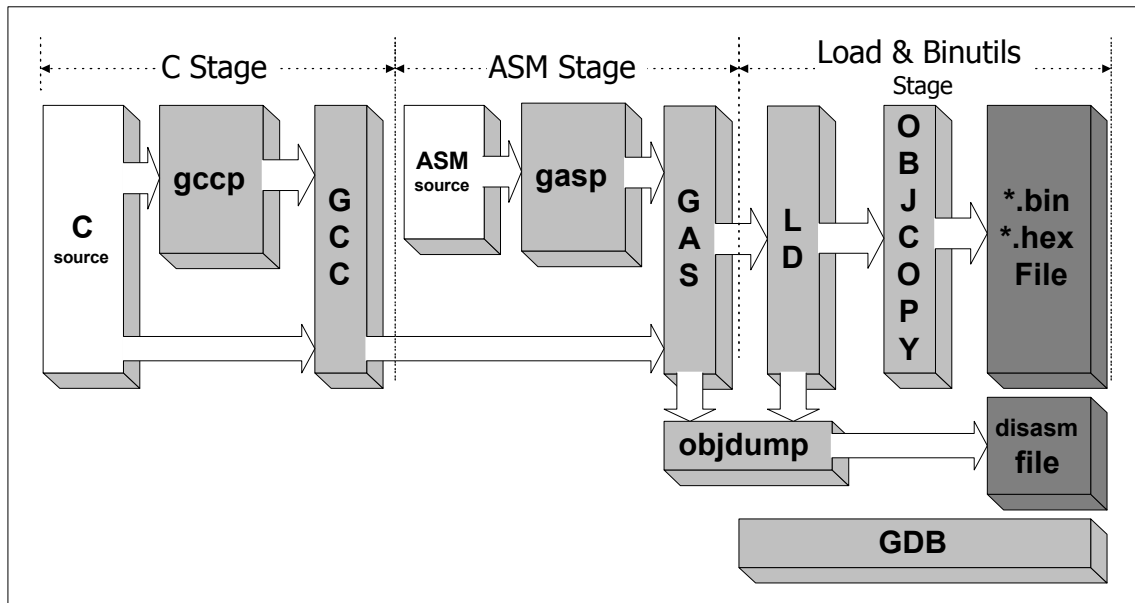


Figure 6.1-1 Software Flow

GNU Software를 사용할 때의 프로그래밍 flow는 위와 같다. 여러 프로그램을 구동 시켜야 하지만 실제로는 gcc 프로그램을 사용 함으로서 나머지 프로그램은 gcc에서 구동 시킬 수 있다. Objdump 프로그램은 disassembler 프로그램이고 gdb는 debugger 이며 objcopy는 파일 포맷을 다른 포맷으로 변경시키는 유틸리티이다.

6.1.1 System Initialize Flow

- 관련 프로그램 구성 및 특징

1. Cygwin : 참조 (www.cygwin.com)
 - Cygwin은 cygnus사에서 제작한 windows 환경하에서 유닉스 프로그램을 사용할 수 있도록 만들어 놓은 툴이다. 간단하게 말한다면 유닉스 에뮬레이터라고도 볼 수 있다.
2. allgcc.sh
 - Cygwin 환경에서 SE3208 프로그램을 Compile하기 위한 Batch File
3. se3208.vct
 - ROM, RAM 의 Size Set
 - Stack Pointer Set
4. crt0.s (System Initialize 과정)
 - PLL Register Set
 - PIN_MUX Register Set
 - PIO Register Set
 - Local Memory Configuration
 - 변수 값을 RAM에 Copy
 - Main Routine Call
5. se3208.c
 - Hardware Vector Table 정의
 - Peek / Poke함수 정의 : Memory 또는 Register Read / Write
6. se3208.h
 - 함수선언
 - Macro 정의 (Define 문)

7. test.c (Main 함수 routine : Test 하기위한 프로그램)

- VISA3000 Program Flow
 - Stack Pointer Set (se3208.vct)
 - Crt0.s 작성 : PLL Register Set, PIN_MUX Set, PIO Set, RAM 관련 Register Set ...
 - Cache Configuration
 - 각종 Register Set

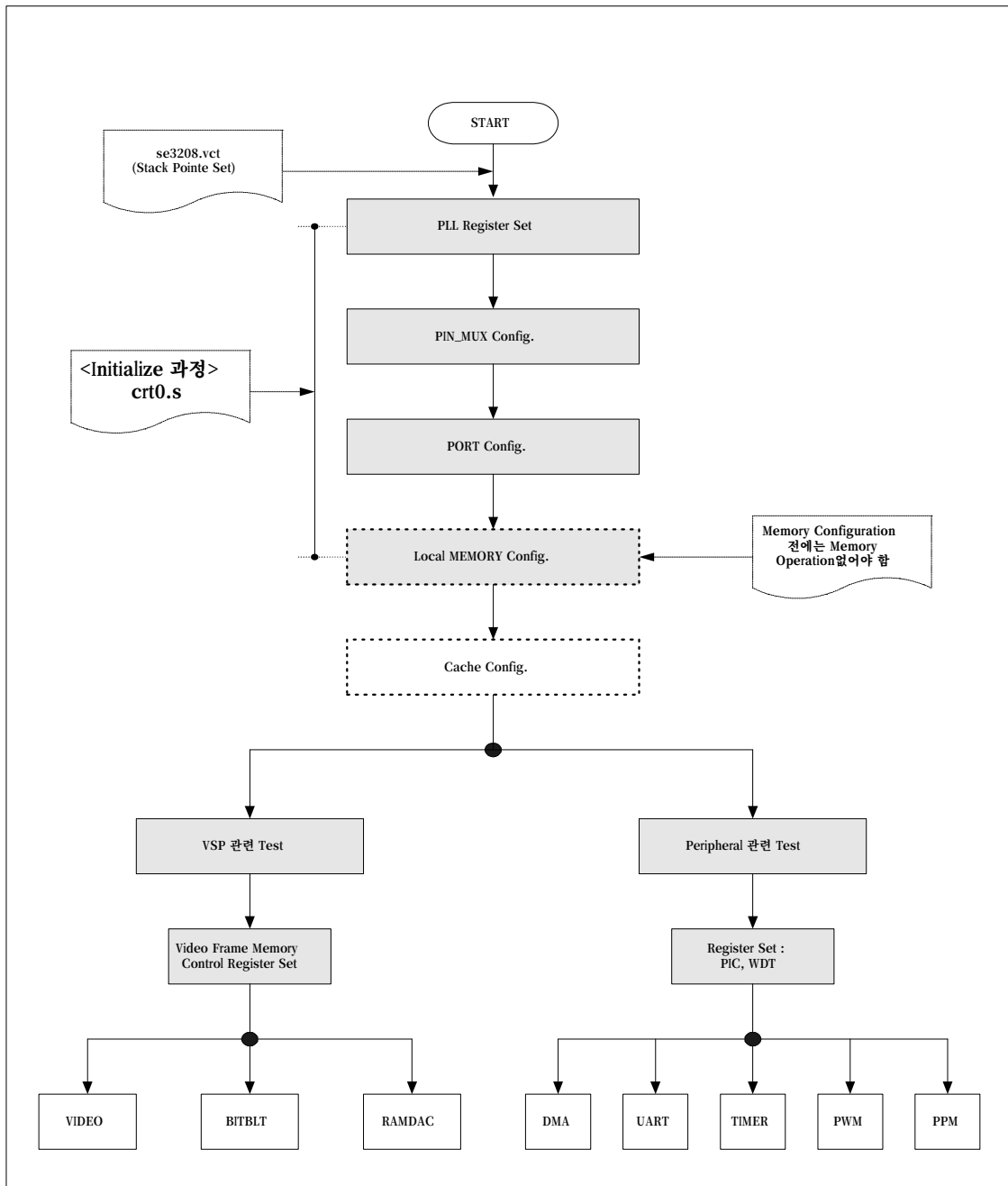


Figure 6.1-2 VISA3000 Test Program Flow

6.1.1.1 Cygnus Environment

• Cygwin 이란?

- Cygwin은 cygnus사에서 제작한 windows 환경하에서 유닉스 프로그램을 사용할 수 있도록 만들어 놓은 툴이다. 간단하게 말한다면 유닉스 에뮬레이터라고도 볼 수 있다.
- Cygwin is a UNIX/Linux shell environment and portability layer enabling delivery of open source projects to Windows.
- Cygwin provides corporate IT and software developers a solution for integrating a heterogeneous environment of Windows and UNIX-based systems.
- In addition, developers can use Cygwin to quickly migrate applications from UNIX to Windows.

• Cygwin 실행

- C 드라이브에 tmp 디렉토리 만든다.
- Cygnus.bat 파일을 실행하면 아래와 같은 화면이 나온다.
- Cygnus.bat 파일은 다음과 같다.

```
@ECHO OFF
SET MAKE_MODE=UNIX
SET PATH=c:\CYGNUS\CYGWIN~1\H-I586~1\BIN;c:\usr\local\bin;
Bash
```

- Windows 환경에서 Compile 하기 위해서는 Cygwin을 활성화 시킨 후 프로그램이 있는 디렉토리에서 Compile 한다.

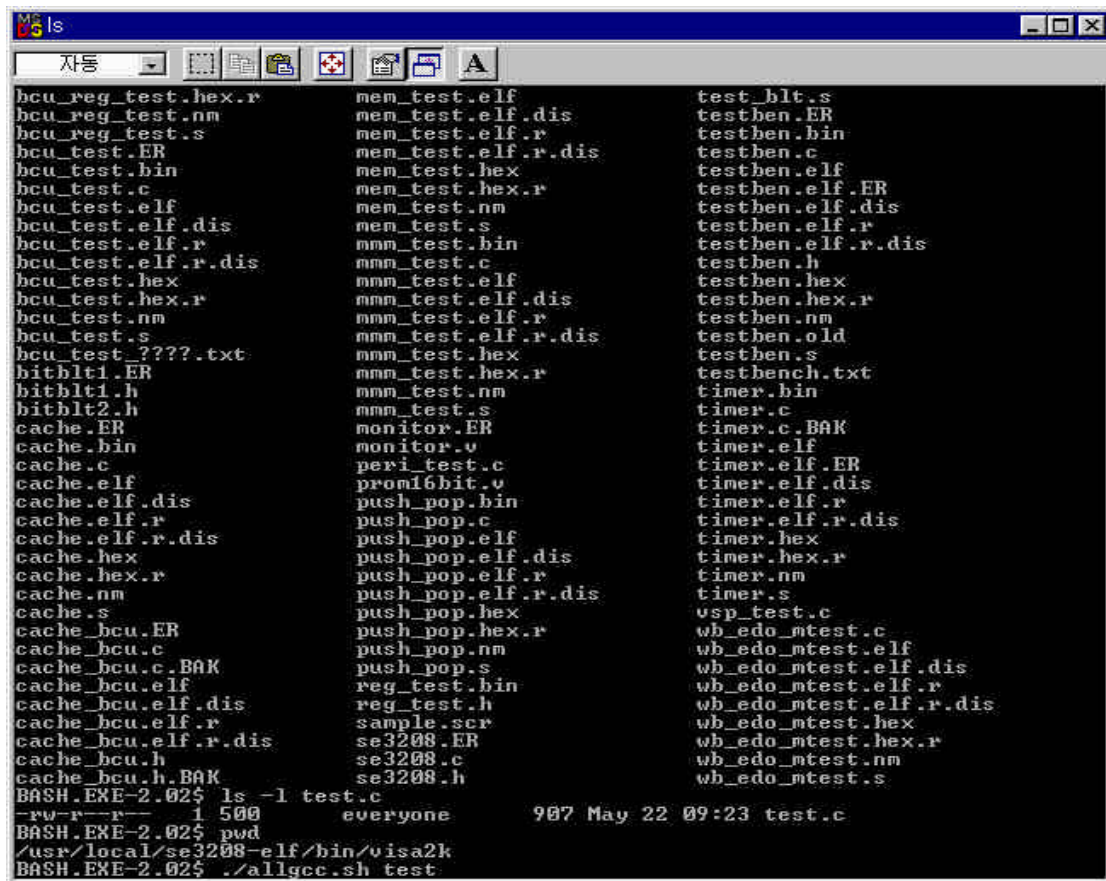


Figure 6.1-3 Cygwin Environment

6.1.1.2 Allgcc.sh

- 실제로 C 파일에서 최종 롬 파일인 HEX 파일이나 BIN 파일을 얻기 위해서는 몇 개의 단계를 거쳐야 하는데 이를 다음과 같은 배치 파일로 만들어 일괄 처리하도록 해 두었다.
- Allgcc.sh 파일은 일괄 처리 파일로 C 코드에서부터 최종 코드까지 만들어 낸다.

```

echo "#####"
echo "not relaxation batch file"
echo "#####"
echo

echo "#####"
echo "Object File Generation : crt0.S ==> crt0.o"
echo "#####"
echo

/usr/Local/Bin/se3208-elf-as -o crt0.o crt0.S

echo "#####"
echo "          gcc compile"
echo "#####"
echo

./gcc -O2 se3208.c -o se3208.o -c
./gcc -O2 se3208.o $1.c -Xlinker -Tse3208.vct -lm -o $1.elf
./gcc -O2 se3208.o $1.c -Xlinker -Tse3208.vct -Xlinker -relax -o $1.elf.r -lm
./gcc -O2 $1.c -S -I/usr/local/se3208-elf/include

ls -l $1.elf
ls -l $1.elf.r

echo "#####"
echo "          Disassemble all "
echo "#####"
echo

/usr/local/bin/se3208-elf-objdump --disassemble-all $1.elf.r > $1.elf.r.dis
/usr/local/bin/se3208-elf-objdump --disassemble-all $1.elf > $1.elf.dis
ls -l $1.elf.r.dis
ls -l $1.elf.dis

echo "#####"
echo "          generate intel hex file"
echo "#####"
echo

se3208-elf-objcopy -O ihex $1.elf -I elf32-se3208 $1.hex
se3208-elf-objcopy -O ihex $1.elf.r -I elf32-se3208 $1.hex.r
ls -l $1.hex
ls -l $1.hex.r

echo "#####"
echo "Difference between relax elf file and elf file"
echo "#####"
echo

#diff $1.elf.r.dis $1.elf.dis | more

echo "#####"
echo "          Generate nm file"
echo "#####"
echo

./nm $1.elf > $1.nm

echo "#####"
echo "          generate binary file"
echo "#####"
echo

se3208-elf-objcopy -O binary $1.elf -I elf32-se3208 $1.bin
ls -l $1.bin

```

Figure 6.1-4 Allgcc.sh

- 사용법은 테스트 프로그램이 <TEST.C> 이면
allgcc.sh test <엔터> 이다.
- 위와 같이 하면 elf 파일 및 롬화 할 수 있는 hex , bin 파일 그리고 disassembler 된 파일들과 컴파일 된 어셈블러 소스 파일이 생성된다.

Program 구조 및 Compile 과정

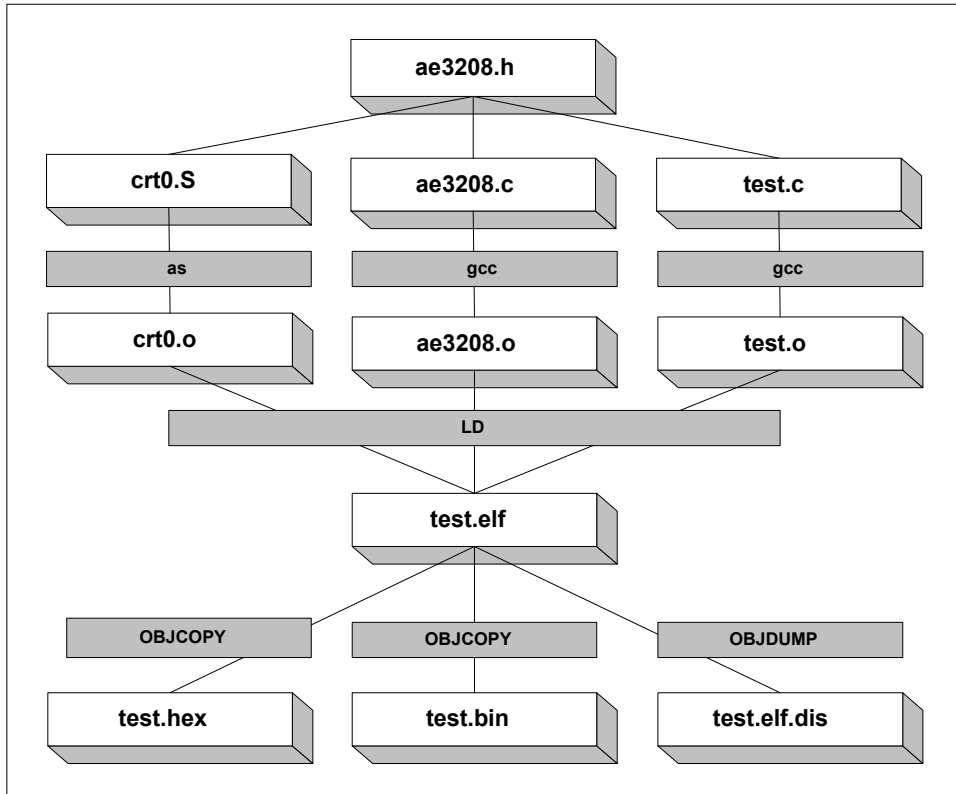


Figure 6.1-5 Program 구조 및 Compile 과정

- Allgcc.sh Batch File 실행 과정

- 검은 상자 안의 명칭은 실행하는 파일 이름이다.
- 실제로 위에서는 gcc와 ld를 거쳐야 한다고 되어 있지만 실제로는 gcc 프로그램에서 loader를 자동적으로 호출하여 사용하므로 gcc만 사용하면 된다.
- 결과 file은 elf파일로 생성되며 이것은 ELF 포맷에 맞게 생성된다.
- Elf 파일을 이용해서 롬화 시킬 수 있는 hex , bin 파일 및 disassemble 한 elf.dis 파일을 생성 시킨다. 파일들의 설명은 아래 표를 참조하기 바란다.

생성된 파일 명	설명
Test.elf	Elf format으로 생성된 코드 파일
Test.elf.r	Elf format으로 생성된 코드 파일 linker 단계에서 relax 처리 되어 생성된 코드이다.
test.elf.dis	Elf format을 disassemble 한 파일
test.elf.r.dis	Relax 시켜 생성된 elf 파일을 disassemble 한 파일이다.
test.hex	롬화 할 수 있는 Intel Hex Format의 파일
test.hex.r	relax시켜서 생성된 elf파일을 hex포맷으로 변환시킨 파일
Test.nm	Test.elf내에 있는 심볼 의 정보를 한데 모아둔 파일
Test.s	Test.c파일을 컴파일 한 어셈블러 파일
Test.c	C 컴파일러가 동작하는 정보 파일

6.1.1.3 se3208.vct

- Loader Script
- ROM, RAM 의 Size Set
- Stack Pointer Set

```

OUTPUT_FORMAT("elf32-se3208", "elf32-se3208",
              "elf32-se3208")
OUTPUT_ARCH(se3208)

ENTRY(_start)

GROUP(-lc -leval -lgcc)

STARTUP(crt0.o)

SEARCH_DIR(/usr/local/se3208-elf/lib);
/* Do we need any of these for elf?
   __DYNAMIC = 0; */
MEMORY
{
    rom : ORIGIN = 0x0      , LENGTH = 1M
    ram : ORIGIN = 0x02000000 , LENGTH = 1M
}

SECTIONS
{
    /* Read-only sections, merged into text segment: */
    . = 0x0;
    .text :
    {
        *(.vectors)
        *(.text)
        *(.stub)
        *(.rodata)
        *(.rodata1)
        __ctors = . ;
        *(.ctors)
        __ctors_end = . ;
        __dtors = . ;
        *(.dtors)
        __dtors_end = . ;
        _etext = . ;
        __shadow_data = . ;
    } > rom
    PROVIDE (etext = .);
    PROVIDE (__shadow_data = .);
    _erodata = .;
    _data = .;
    .data :
        AT ( ADDR (.text) + SIZEOF(.text) )
    {
        _data_reload = . ;
        PROVIDE(__data_reload = . );
        *(.data)
        *(.sdata)
    }
    CONSTRUCTORS
    } > ram
    _edata = .;
    PROVIDE (edata = .);
    .bss (NOLOAD) :
    {
        __bss_start = .;
        *(.dynbss)

```

```

*(.bss)
*(COMMON)
} > ram
_end = . ;
PROVIDE (end = .);
/* Stabs debugging sections. */
.stab 0 : { *(.stab) }
.stabstr 0 : { *(.stabstr) }
.stab.excl 0 : { *(.stab.excl) }
.stab.exclstr 0 : { *(.stab.exclstr) }
.stab.index 0 : { *(.stab.index) }
.stab.indexstr 0 : { *(.stab.indexstr) }
.comment 0 : { *(.comment) }
/* DWARF debug sections.
   Symbols in the DWARF debugging sections are relative to the beginning
   of the section so we begin them at 0. */
/* DWARF 1 */
.debug      0 : { *(.debug) }
.line       0 : { *(.line) }
/* GNU DWARF 1 extensions */
.debug_srcinfo 0 : { *(.debug_srcinfo) }
.debug_sfnames 0 : { *(.debug_sfnames) }
/* DWARF 1.1 and DWARF 2 */
.debug_aranges 0 : { *(.debug_aranges) }
.debug_pubnames 0 : { *(.debug_pubnames) }
/* DWARF 2 */
.debug_info  0 : { *(.debug_info) }
.debug_abbrev 0 : { *(.debug_abbrev) }
.debug_line  0 : { *(.debug_line) }
.debug_frame 0 : { *(.debug_frame) }
.debug_str    0 : { *(.debug_str) }
.debug_loc    0 : { *(.debug_loc) }
.debug_macinfo 0 : { *(.debug_macinfo) }
/* SGI/MIPS DWARF 2 extensions */
.debug_weaknames 0 : { *(.debug_weaknames) }
.debug_funcnames 0 : { *(.debug_funcnames) }
.debug_tynames 0 : { *(.debug_tynames) }
.debug_varnames 0 : { *(.debug_varnames) }
/* These must appear regardless of . */
.stack 0x020ffff : { _stack = .; *(.stack) }
}
PROVIDE(__data_size = SIZEOF(.data));

```

Figure 6.1-6 Se3208.vct

6.1.1.4 crt0.s

- Program Initialize 과정
 - PLL Register Set
 - PIN_MUX Register Set
 - PIO Register Set
 - Local Memory Configuration
 - 변수값을 RAM에 Copy
 - Main Routine Call

```

#####
##
##      crt0.S
##
##      SE3208 startup code
##
#####
##
## Copyright (c) 1995, 1996, 1997, 1998 Cygnus Solutions
##
## The authors hereby grant permission to use, copy, modify, distribute,
## and license this software and its documentation for any purpose, provided
## that existing copyright notices are retained in all copies and that this
## notice is included verbatim in any distributions. No written agreement,
## license, or royalty fee is required for any of the authorized uses.
## Modifications to this software may be copyrighted by their authors
## and need not follow the licensing terms described here, provided that
## the new terms are clearly indicated on the first page of each file where
## they apply.
##
##-----
##
##      .file      "crt0.S"
##-----
## Startup code
##      .section .text
##      .global    _start
_start:
    ldi    _stack-8, %r3      # Load up the stack pointer and allocate
    lea    (%r3), %esp # our current frame.

#####
# PLL Register Set
#####
    ldi    0x01804000,      %R6      # RAMDAC & PLL Control Register
    ldi    0x00000004,      %R7
    st     %R7,    (%R6,0)

    ldi    0x01804004,      %R6      # PLL Program Register
    ldi    0x00003E21,      %R7      # M = 62, P = 8, S = 1
    st     %R7,    (%R6,0)

#####
# PIN MUX SET
#####
    ldi    0x01800018,      %R6      # Pin Mux Control Register 1
    ldi    0x00000000,      %R7
    st     %R7,    (%R6,0)

    ldi    0x0180001C,      %R6      # Pin Mux Control Register 2
    ldi    0x00020000,      %R7
    st     %R7,    (%R6,0)

#####
# PIO SET
#####
    ldi    0x01802000,      %R6      # PIO Mode Register (PIOMOD)
    ldi    0x00000000,      %R7
    st     %R7,    (%R6,0)

#      ldi    0x01802004,      %R6      # PIO Latched Output Data Register (PIOLDAT)

```

```

# ldi 0x00000000, %R7
# st %R7, (%R6,0)
#####
# LOCAL RAM CFG
#####
# SRAM TYPE
# ldi 0x01800408, %R6 # Local DRAM Control Register
# ldi 0x00000000, %R7
# st %R7, (%R6,0)
#####
# EDO DRAM TYPE
# ldi 0x01800408, %R6 # Local DRAM Control Register
# ldi 0x00120011, %R7
# st %R7, (%R6,0)
#####
# Fast Page DRAM TYPE
# ldi 0x01800408, %R6 # Local DRAM Control Register
# ldi 0x00020012, %R7
# st %R7, (%R6,0)
#####
# SDRAM TYPE
# ldi 0x01800408, %R6 # Local DRAM Control Register
# ldi 0x00000013, %R7
# st %R7, (%R6,0)
#####
ldi _etext, %r3 # ROM has data at end of text; copy it. */
ldi _data_reload, %r1

cmp %r3, %r1
jz .L2 # If no data, then do nothing

ldi _edata, %r4

.L3:
ldb (%r3), %r2
stb %r2, (%r1)
addi %r3, 1, %r3
addi %r1, 1, %r1
cmp %r4, %r1
jnz .L3

.L2:
call _main # Call main program
.section .stack
stack: .long 1

```

Figure 6.1-7 Crt0.s

- 우선 RESET 상태를 벗어나면 프로그램은 Reset Vector가 가리키는 CRT0.S로 들어간다.

```

/* initialize stack pointer */
_start:
ldi _stack-8, %r3 # Load up the stack pointer and allocate
lea (%r3), %esp # our current frame.

```

- Stack pointer를 초기화 시킨다. 이때 Stack pointer에 들어가는 값은 Loader Script에서 정의된 값이다. GCC에서는 Stack도 하나의 Section 형태로 취급할 수 있으므로 Stack section을 만들기 위해서 crt0.S함수의 제일 끝에 다음과 같이 한다.

```

.section .stack
_stack: .long 1

```

6.1.1.5 se3208.c

- Hardware Vector Table 정의
- Peek / Poke 함수 정의 : Memory 또는 Register Read / Write
 - 8 bit Write : pokec
 - 16 bit Write : pokes
 - 32 bit Write : pokel
 - 8 bit Read : peekc
 - 16 bit Read : peeks
 - 32 bit Read : peekl

```

#ifndef __SE3208_C__
#define __SE3208_C__

#include "se3208.h"

__inline__ long peekl( void * addr ) /* get memory data */
{
    return *((long*)addr);
}

short peeks( void * addr ) /* get memory data */
{
    return *((short*)addr);
}

char peekc( void * addr ) /* get memory data */
{
    return *((char*)addr);
}

void pokel( void * addr , long data ) /* set memory data */
{
    *((long*)addr) = data;
}

void pokes( void * addr , short data ) /* set memory data */
{
    *((short*)addr) = data;
}

void pokec( void * addr , char data ) /* set memory data */
{
    *((char*)addr) = data;
}

const fp HardwareVector[] __attribute__((section (".vectors"))) = {
    /* Reset Vector */
    start , /* V00 : Reset Vector */
    NMI , /* V01 : NMI Vector */
    AUTO_INT , /* V02 : Interrupt Auto Vector */
    NOTUSEDISR, /* V03 : Double Fault Exception */
    NOTUSEDISR, /* V04 : Bus Error exception */
    NOTUSEDISR, /* V05 : Reserved */
    NOTUSEDISR, /* V06 : Reserved */
    NOTUSEDISR, /* V07 : Reserved */
    NOTUSEDISR, /* V08 : Co-processor Exception */
    NOTUSEDISR, /* V09 : Co-processor Exception */
    NOTUSEDISR, /* V0A : Co-processor Exception */
    NOTUSEDISR, /* V0B : Co-processor Exception */
    NOTUSEDISR, /* V0C : OS Reset Vector */
    NOTUSEDISR, /* V0D : OS break exception */
    NOTUSEDISR, /* V0E : Reserved */
    NOTUSEDISR, /* V0F : Reserved */
    NOTUSEDISR, /* V10 : SWI 00 Vector */
    NOTUSEDISR, /* V11 : SWI 01 Vector */
    NOTUSEDISR, /* V12 : SWI 02 Vector */
    NOTUSEDISR, /* V13 : SWI 03 Vector */
    NOTUSEDISR, /* V14 : SWI 04 Vector */
    NOTUSEDISR, /* V15 : SWI 05 Vector */
    NOTUSEDISR, /* V16 : SWI 06 Vector */
}

```

```

SWI7 , /* V17 : SWI 07 Vector */
NOTUSEDISR, /* V18 : SWI 08 Vector */
NOTUSEDISR, /* V19 : SWI 09 Vector */
NOTUSEDISR, /* V1A : SWI 0A Vector */
NOTUSEDISR, /* V1B : SWI 0B Vector */
NOTUSEDISR, /* V1C : SWI 0C Vector */
NOTUSEDISR, /* V1D : SWI 0D Vector */
NOTUSEDISR, /* V1E : SWI 0E Vector */
NOTUSEDISR, /* V1F : SWI 0F Vector */

TIMER0 , /* V20 : User 00 Vector */
TIMER1 , /* V21 : User 01 Vector */
PPM0 , /* V22 : User 02 Vector */
PPM1 , /* V23 : User 03 Vector */
DMA0 , /* V24 : User 04 Vector */
DMA1 , /* V25 : User 05 Vector */
TIMER2 , /* V26 : User 06 Vector */
TIMER3 , /* V27 : User 07 Vector */
EXT_IRQ2 , /* V28 : User 08 Vector */
EXT_IRQ3 , /* V29 : User 09 Vector */
UART0_RXE , /* V2A : User 0A Vector */
UART0_RX , /* V2B : User 0B Vector */
UART0_TX , /* V2C : User 0C Vector */
UART1_RXE , /* V2D : User 0D Vector */
UART1_RX , /* V2E : User 0E Vector */
UART1_TX , /* V2F : User 0F Vector */

EXT_IRQ4 , /* V30 : User 10 Vector */
EXT_IRQ5 , /* V31 : User 11 Vector */
EXT_IRQ6 , /* V32 : User 12 Vector */
EXT_IRQ7 , /* V33 : User 13 Vector */
VEXT_SYNC , /* V34 : User 14 Vector */
VSP , /* V35 : User 15 Vector */
PPM_OVF , /* V36 : User 16 Vector */
PWM , /* V37 : User 17 Vector */
BITBLT , /* V38 : User 18 Vector */
SPRITE , /* V39 : User 19 Vector */
NOTUSEDISR, /* V3A : User 1A Vector */
NOTUSEDISR, /* V3B : User 1B Vector */
NOTUSEDISR, /* V3C : User 1C Vector */
NOTUSEDISR, /* V3D : User 1D Vector */
NOTUSEDISR, /* V3E : User 1E Vector */
NOTUSEDISR, /* V3F : User 1F Vector */

NOTUSEDISR, /* V40 : User 20 Vector */
NOTUSEDISR, /* V41 : User 21 Vector */
NOTUSEDISR, /* V42 : User 22 Vector */
NOTUSEDISR, /* V43 : User 23 Vector */
NOTUSEDISR, /* V44 : User 24 Vector */
NOTUSEDISR, /* V45 : User 25 Vector */
NOTUSEDISR, /* V46 : User 26 Vector */
NOTUSEDISR, /* V47 : User 27 Vector */
NOTUSEDISR, /* V48 : User 28 Vector */
NOTUSEDISR, /* V49 : User 29 Vector */
NOTUSEDISR, /* V4A : User 2A Vector */
NOTUSEDISR, /* V4B : User 2B Vector */
NOTUSEDISR, /* V4C : User 2C Vector */
NOTUSEDISR, /* V4D : User 2D Vector */
NOTUSEDISR, /* V4E : User 2E Vector */
NOTUSEDISR, /* V4F : User 2F Vector */

};

#endif /* SE3208 C */

```

Figure 6.1-8 se3208.c

- 위에서 attribute 문은 해당 변수의 속성을 주는 것으로 여기서는 vects라는 섹션으로 들어가라고 지정해 주기 위해 사용하였다. 이를 loader script 파일에서 어드레스 0x00000000으로 지정하면 해당 어드레스에 위치하게 된다.

- fp는 ae3208.h파일에 다음과 같이 선언되어 있다.

```
typedef void(*fp)(void)
```

즉 반환 값도 전달 값도 없는 함수형을 선언한다.

- NOTUSEDISR 는 (fp)(0)으로 선언되어 있어서 실제로 널 함수를 가리키는 널 포인터가 된다.
- 만약 NMI 를 사용하는 interrupt 함수의 이름이 NMI() 이고 이를 연결 시키기 위해서는 다음과 같이 한다.


```

const fp HardwareVector[] __attribute__((section (".vects"))) = {
    /* Reset Vector */
    _START, /* V00 : Reset Vector */
    NMI, /* V01 : NMI Vector */
}

```

6.1.1.6 se3208.h

- 함수 선언
- Macro 정의
 - Register, Memory Write / Read Function
 - Cache Configuration 을 위한 Macro 정의 (자세한 설명은 뒤에 Cache Configuration Block에서 다루어 진다.)

```

#ifndef __SE3208_H__
#define __SE3208_H__

extern void start();

#define NOTUSEDISR (fp)(0)
// #define __ASSEMBLE__
typedef void (*fp)(void);

void NMI();
void SWI7();
void AUTO_INT();

void TIMER0();
void TIMER1();
void PPM0();
void PPM1();
void DMA0();
void DMA1();
void TIMER2();
void TIMER3();
void EXT_IRQ2();
void EXT_IRQ3();
void UART0_RXE();
void UART0_RX();
void UART0_TX();
void UART1_RXE();
void UART1_RX();
void UART1_TX();
void EXT_IRQ4();
void EXT_IRQ5();
void EXT_IRQ6();
void EXT_IRQ7();
void VEXT_SYNC();
void VSP();
void PPM_OVF();
void PWM();
void BITBLT();
void SPRITE();

long peekl( void * addr ); /* get memory 32 bit data */
short peeks( void * addr ); /* get memory 16 bit data */
char peekc( void * addr ); /* get memory 8 bit data */

void pokel( void * addr , long data ); /* set memory data */
void pokes( void * addr , short data ); /* set memory data */
void pokec( void * addr , char data ); /* set memory data */

/*****/

#define __SET_REG_MEM_8_BIT( ADDR, WDATA) pokel((void*) ADDR, (char) WDATA );
#define __SET_REG_MEM_16_BIT(ADDR, WDATA) pokes((void*) ADDR, (short) WDATA );
#define __SET_REG_MEM_32_BIT(ADDR, WDATA) pokel((void*) ADDR, (long) WDATA );

#define __GET_REG_MEM_8_BIT( ADDR, RDATA) RDATA = peekc((void*) ADDR);
#define __GET_REG_MEM_16_BIT(ADDR, RDATA) RDATA = peeks((void*) ADDR);
#define __GET_REG_MEM_32_BIT(ADDR, RDATA) RDATA = peekl((void*) ADDR);

/*****/

#define __CACHE_INVALIDATE_SET(VALUE) \
asm( "CACHE_INVALIDATE_SET: "); \
asm( "push %R0"); \
asm( "ldi %0 , %%R0" : : "i" (VALUE) ); \
asm( "mvtc 0 , %CR5"); \
asm( "pop %R0");

```

```

#define __CACHE_INVALIDATE_CHECK()
asm( "CACHE_INVALIDATE_CHECK:" );
asm( "mvfc 0 , %CR6" );
asm( "andi %R0, 0x00100000, %R0" );
asm( "jnz  CACHE_INVALIDATE_CHECK" );

#define __CACHE_MODE_SET(VALUE)
asm( "CACHE_MODE_SET:" );
asm( "push %R0" );
asm( "ldi %0 , %%R0" : : "i" (VALUE) );
asm( "mvtc 0 , %CR3" );
asm( "pop %R0" );

#define __CACHE_ACTIVE_SET(VALUE)
asm( "CACHE_ACTIVE_SET:" );
asm( "push %R0" );
asm( "ldi %0 , %%R0" : : "i" (VALUE) );
asm( "mvtc 0 , %CR7" );
asm( "pop %R0" );

/*****
// CACHE MASTER COMMAND REGISTER
*****/
# define __CACHE_UNIT_DISABLE ( 0 << 11 )
# define __CACHE_UNIT_ENABLE ( 1 << 11 )

/*****
// CACHE INVALIDATE REGISTER
*****/
# define __CACHE_INVALIDATE_DISABLE ( 0 << 0 )
# define __CACHE_INVALIDATE_ENABLE ( 1 << 0 )

/*****
// CACHE INVALIDATE REGISTER
*****/
# define __CACHE_WRITE_THROUGH ( 0 << 6 )
# define __CACHE_WRITE_BACK ( 1 << 6 )

#####

#endif /* se3208.h */

```

Figure 6.1-9 Se3208.h

6.1.1.7 test.c

- 레지스터 Setting 및 실제 수행할 Main Routine Program 정의

```

/*****
VISA3000 TESTBENCH PROGRAM
*****/

#include "se3208.h"

#pragma interrupt
void SWI7()
{
}

#pragma interrupt
void AUTO_INT()
{
}

#pragma interrupt
void NMI()
{
}

/*****
/* Main Program Routine */
*****/
int main()
{
    asm("LOCAL_ROM_CONFIGURATION_SET:");
    _MEMIO_WR_32(0x01800400, 0x00000004)
        // bit 7 : Iromcs# decode method
        // bit 2-0 : Iromcs# wait state

    asm("SYSTEM_CONTROL_REGISTER_SET:");
    _MEMIO_WR_32(0x01800008, 0x000001C3)
        // bit 9-8 : Expansion ROM Data Bus Width
        // bit 7-6 : Frame Memory Internal Command Cycle Wait Control
        // bit 5-4 : Register Internal Command Cycle Wait Control
        // bit 2-0 : Expansion ROM Command Cycle Wait Control

    /***** CACHE CONFIGURATION *****/
    asm("CACHE_CONFIGURATION_SET:");
    _CACHE_INVALIDATE_SET(_CACHE_INVALIDATE_ENABLE)
    _CACHE_INVALIDATE_CHECK()
    _CACHE_MODE_SET(_CACHE_WRITE_BACK) // WRITE BACK MODE
    // _CACHE_MODE_SET(_CACHE_WRITE_THROUGH) // WRITE THROUGH MODE
    _CACHE_ACTIVE_SET(_CACHE_UNIT_ENABLE)
    /*****

    /***** VIDEO FRAME MEMORY CONTROL *****/
    asm("VIDEO_FRAME_MEMORY_CONTROL_REGISTER_SET:");
    _MEMIO_WR_32(0x01803004, 0x0000000A)
        // bit 11-10 : RAS to CAS Delay
        // bit 9-8 : RAS Precharge Time
        // bit 6 : SDRAM - CAS Latency, EDO/FP DRAM - CAS Precharge
        // bit 5 : EDO/FP DRAM - CAS Active Pulse Width Time
        // bit 4-3 : SDRAM - RAS Cycle Time after Auto-refresh Command
        // bit 2-1 : Refresh Cycle per Horizontal Display signal
        // bit 0 : Internal Read Data Latch Timing Control for FP DRAM

    asm("VIDEO_FRAME_MEMORY_CONFIG_REGISTER_SET:");
    _MEMIO_WR_32(0x01803000, 0x00000004)
        // bit 8 : Video Memory Control Reset
        // bit 6-5 : Row Address Line Number (SDRAM Only)
        // bit 4-3 : Column Address Line Number (FP, EDO DRAM / SDRAM)
        // bit 2 : Video Memory Bus Width
        // bit 1-0 : Video Memory Type

    /*****

    /***** Main Execute Program Routine Set *****/
    /*****

```

```

}
/* Main Program End */

```

Figure 6.1-10 test.c

6.1.1.8 Local Memory (RAM), Local ROM, Expansion ROM and System Control Configuration

1. Local Memory (RAM) Control Configuration
 - Initialize 과정인 crt0.s에 나타나 있다.
 - 외부에 연결된 Local Memory 의 Type 및 제어 방법을 Setting 하기 전에 Local Memory Operation이 발생하면 정확한 값을 Access 할 수가 없으므로, 반드시 메모리 Operation이 발생하기 전에 관련 Register 값을 Setting 해주어야 한다.
(단, SRAM을 Local Memory로 사용 할 경우는 Access Time Wait 만을 제어하므로 Main Routine 후에 Setting 해도 상관 없음)

```

#####
# LOCAL RAM CFG
#####
# SRAM TYPE
# ldi 0x01800408, %R6 # Local DRAM Control Register
# ldi 0x00000000, %R7
# st %R7, (%R6,0)
##-----
# EDO DRAM TYPE
# ldi 0x01800408, %R6 # Local DRAM Control Register
# ldi 0x00120011, %R7
# st %R7, (%R6,0)
##-----
# Fast Page DRAM TYPE
# ldi 0x01800408, %R6 # Local DRAM Control Register
# ldi 0x00020012, %R7
# st %R7, (%R6,0)
##-----
# SDRAM TYPE
# ldi 0x01800408, %R6 # Local DRAM Control Register
# ldi 0x00000013, %R7
# st %R7, (%R6,0)
#####

```

Figure 6.1-11 Local Memory (RAM) control Configuration

2. Local ROM, Expansion ROM & System Control Register Set

```

Asm("LOCAL_ROM_CONFIGURATION_SET:");
MEMIO_WR_32(0x01800400, 0x00000004)
// bit 7 : Iromcs# decode method
// bit 2-0 : Iromcs# wait state

asm("SYSTEM_CONTROL_REGISTER_SET:");
MEMIO_WR_32(0x01800008, 0x000001C3)
// bit 9-8 : Expansion ROM Data Bus Width
// bit 7-6 : Frame Memory Internal Command Cycle Wait Control
// bit 5-4 : Register Internal Command Cycle Wait Control
// bit 2-0 : Expansion ROM Command Cycle Wait Control

```

Figure 6.1-12 Local ROM, Expansion ROM and System control Configuration

- Local ROM Decode 방법 설정 및 Wait Control
- Expansion ROM Data Bus Width Set & Command Cycle Wait Control
 - Data Bus Width 는 Local RAM 의 Data Bus Width (Power On Configuration)보다 같거나 작아야 한다.
- Frame Memory 용 Write FIFO Interface를 위한 Wait Control
- Internal Peripheral (Timer, UART, DMA, PWM, PIO, PIC ...) 의 Write / Read 의 Wait Control

6.1.1.9 Cache Configuration

- se3208.h 에는 Cache Configuration을 위한 Header File 이 정의 되어 있음.
 - Cache Invalidation Process (Hardware)
 - Cache Invalidation 완료 Check
 - Cache Mode Set → Write Through / Back
 - Cache Enable

```

#define __CACHE_INVALIDATE_SET(VALUE)
asm( "CACHE_INVALIDATE_SET: ");
asm( "push %R0");
asm( "ldi %0 , %%R0" : : "i" (VALUE) );
asm( "mvtc 0 , %CR5");
asm( "pop %R0");

#define __CACHE_INVALIDATE_CHECK()
asm( "CACHE_INVALIDATE_CHECK:");
asm( "mvfc 0 , %CR6");
asm( "andi %R0, 0x00100000, %R0");
asm( "jnz __CACHE_INVALIDATE_CHECK");

#define __CACHE_MODE_SET(VALUE)
asm( "CACHE_MODE_SET: ");
asm( "push %R0");
asm( "ldi %0 , %%R0" : : "i" (VALUE) );
asm( "mvtc 0 , %CR3");
asm( "pop %R0");

#define __CACHE_ACTIVE_SET(VALUE)
asm( "CACHE_ACTIVE_SET: ");
asm( "push %R0");
asm( "ldi %0 , %%R0" : : "i" (VALUE) );
asm( "mvtc 0 , %CR7");
asm( "pop %R0");

/*****
// CACHE MASTER COMMAND REGISTER
*****/
# define __CACHE_UNIT_DISABLE ( 0 << 11 )
# define __CACHE_UNIT_ENABLE ( 1 << 11 )

/*****
// CACHE INVALIDATE REGISTER
*****/
# define __CACHE_INVALIDATE_DISABLE ( 0 << 0 )

```

```
# define __CACHE_INVALIDATE_ENABLE (1 << 0)

/*****
// CACHE INVALIDATE REGISTER
*****/
# define __CACHE_WRITE_THROUGH (0 << 6)
# define __CACHE_WRITE_BACK (1 << 6)

/*****
```

Figure 6.1-13 Cache Configuration Program

- test.c 에는 Cache Configuration 과정이 나와 있음.
 - Coprocessor Write – mvtc 명령어 사용
 - Coprocessor Read – mftc 명령어 사용
- Cache Configuration 순서
 - (1) Cache Invalidation Register Set – Cache Invalidate Enable
 - (2) Cache Invalidation Set – Cache Invalidation 완료 Check
 - (3) Cache Mode Set – Write Through / Write Back
 - (4) Cache Enable

```
-/***** CACHE CONFIGURATION *****/
asm("CACHE_CONFIGURATION_SET: ");
__CACHE_INVALIDATE_SET(__CACHE_INVALIDATE_ENABLE)
__CACHE_INVALIDATE_CHECK()
__CACHE_MODE_SET(__CACHE_WRITE_BACK) // WRITE BACK MODE
// __CACHE_MODE_SET(__CACHE_WRITE_THROUGH) // WRITE THROUGH MODE
__CACHE_ACTIVE_SET(__CACHE_UNIT_ENABLE)
/*****
```

6.1.1.10 Video Frame Memory Configuration

- Video Frame Memory를 제어하기 위한 Flow
 - (1) Video Frame Memory Control Register Set
 - (2) Video Frame Memory Configuration Register Set
 - Video Memory Control Reset 을 Active 시켜야 한다.
- 외부 Video Memory에 맞게 Memory Type Register Set

(주의) SDRAM 인 경우 Bank Address 가 있으므로 Row Address Line Number를 Setting해야 함.

```

/***** VIDEO FRAME MEMORY CONTROL *****/
asm("VIDEO_FRAME_MEMORY_CONTROL_REGISTER_SET:");
MEMIO_WR_32(0x01803004, 0x0000000A)
// bit 11-10 : RAS to CAS Delay
// bit 9-8 : RAS Precharge Time
// bit 6 : SDRAM - CAS Latency, EDO/FP DRAM - CAS Precharge
// bit 5 : EDO/FP DRAM - CAS Active Pulse Width Time
// bit 4-3 : SDRAM - RAS Cycle Time after Auto-refresh Command
// bit 2-1 : Refresh Cycle per Horizontal Display signal
// bit 0 : Internal Read Data Latch Timing Control for FP DRAM

asm("VIDEO_FRAME_MEMORY_CONFIG_REGISTER_SET:");
MEMIO_WR_32(0x01803000, 0x00000004)
// bit 8 : Video Memory Control Reset
// bit 6-5 : Row Address Line Number (SDRAM Only)
// bit 4-3 : Column Address Line Number (FP, EDO DRAM / SDRAM)
// bit 2 : Video Memory Bus Width
// bit 1-0 : Video Memory Type
/*****/

```

Figure 6.1-14 Video Frame Memory Configuration

6.1.1.11 Interrupt Controller Operation

```

void INTERRUPT_FUNCTION()
{
    ....
    interrupt 서비스 수행
    ....
    __ISR_END(SERVICE CHANNEL)
}

```

위의 골격이 interrupt 수행 함수이다.

실제 연결은 아주 단순하여 함수 명을 원하는 vector table 배열위치에 넣는 것으로 이루어 진다. vector table은 ae3208.c 파일에 선언 되어 있다.

만약 TIMER0 인터럽트를 사용하고자 하면 se3208.c 벡터 테이블의 위치를 그림의 밑줄 부분과 같이 연결한다. 즉, 인터럽트 발생시 벡터 값이 0x20 이면 TIMER0 인터럽트를 수행한다.

(참고) 인터럽트 벡터 생성과정

Interrupt Vector (8 bit) = Interrupt Vector High 3 Bit (Programmable) + Interrupt Vector Priority Encoder 5 Bit (Fixed)

- Timer0 인터럽트의 하위 5 bit 는 Priority Encoder 에 의해서 “00000 “ 이므로 Interrupt Vector 값이 0x20 (“00100000”) 이 되기 위해서 상위 3 bit는 “001”이 되어야 한다. 즉, 인터럽트를 수행하기 전에 Interrupt Vector Register (0x01800C04) 의Interrupt Vector High 3 Bit (Programmable) 은 “001”로 Setting 해야 한다.

```
const      fp      HardwareVector[]      __attribute__((section
(".vects"))) = {
    /* Reset Vector */
    _START      ,      /* V00 : Reset Vector          */
    NOTUSEDISR,      /* V01 : NMI Vector          */
    NOTUSEDISR, /* V02 : Interrupt Auto Vector */
    NOTUSEDISR, /* V03 : Reserved          */
    NOTUSEDISR, /* V04 : Reserved          */
    NOTUSEDISR, /* V05 : Reserved          */
    NOTUSEDISR, /* V06 : Reserved          */
    NOTUSEDISR, /* V07 : Reserved          */

    .....
    TIMER0      , /* V20 : User 00 Vector      */
    NOTUSEDISR, /* V21 : Reserved          */
    .....
}
```

Figure 6.1-15 Interrupt Vector 생성 관련 Program

TIMER0() 인터럽트 서비스 루틴은 다음과 같다.

```
void TIMER0()
{
    .....
    interrupt 서비스 수행
    .....
    _MEMIO_WR_8(0x01800C04, 0x00) // Service End for TIMER0 Interrupt
}
```

_MEMIO_WR_8(0x01800C04, 0x00) 부분은 인터럽트 서비스 수행이 완료 됐음을 의미한다.

6.2 DMA

다음은 메모리 스택 영역 내에서 DMA를 이용한 데이터 전송 예제 프로그램이다.

```
#define TSIZE 8

void dma_test()
{
    char dst32_buf[TSIZE];           // destination region
```



```

char src32_buf[TSIZE] = {0,1,2,3,4,5,6,7};          // source region

/* set interrupt */

asm( "clr 13 ;");                                     // disable interrupt
asm( "set 12 ;");                                     // set vectored interrupt mod

/* DMA test */

MEMIO_WR_32(0x01800018, 0x00000000)                 // DREQ0, DACK0, DTC 등 관련 신호 포트 선택

MEMIO_WR_32(0x01800C04, 0x0100)                     // interrupt vector high 3 bit = 001
MEMIO_WR_32(0x01800C08, 0x00000030)                 // enable DMA interrupt

asm( "set 13 ;");                                     // enable interrupt

MEMIO_WR_32(0x01800800, 0x01d7)                     // set operation mode :
MEMIO_WR_32(0x01800804, &src32_buf[TSIZE-1])        // high polarity, acnt, tcnt reload, dec address, 32 bit
MEMIO_WR_32(0x01800808, &dst32_buf[TSIZE])          // set source address
MEMIO_WR_32(0x0180080C, TSIZE)                      // set destination address
MEMIO_WR_32(0x0180080C, TSIZE)                      // set transfer size
MEMIO_WR_32(0x01800800, 0x04d7)                     // enable DMA
}

```

Figure 6.2-1 DMA test program

6.3 UART

```

void uart_test( )
{
    asm( "clr 13 ;");                                     // disable interrupt
    asm( "set 12 ;");                                     // set vectored interrupt mod

    MEMIO_WR_32(0x01800018, 0x00000000)                 // TX0, RX0 등 관련 신호 포트 선택
    MEMIO_WR_32(0x01800C04, 0x0100)                     // interrupt vector high 3 bit = 001
}

```

```

    _MEMIO_WR_32(0x01800C08, 0x0000fc00)           // enable UART interrupt

    asm( "set 13 ; ";                               // enable interrupt

    _MEMIO_WR_32(0x01801010, 0x0040)                // set baud rate = 19200 bps
    _MEMIO_WR_32(0x01801000, 0xa8)                  // internal clock, even parity, 1 stop bits, 7 bit data
    _MEMIO_WR_32(0x01801008, 0x0)                   // write data to TX FIFO
    _MEMIO_WR_32(0x01801008, 0x1)
    _MEMIO_WR_32(0x01801008, 0x2)
    _MEMIO_WR_32(0x01801008, 0x3)

}

short peeks( void * addr )                          // get memory data
{
    return *((short*)addr) ;
}

#pragma interrupt                                     // interrupt service routine #11
void UART0_RX( )
{
    int i, rxfl, rxstatus, rxdata;

    rxstatus = peeks(0x01801004);                     // to get RX FIFO water level
    rxfl = rxstatus >> 8;

    for( i=0; i<rxfl; i++ )
        rxdata = peeks(0x0180100C);

    _MEMIO_WR_8(0x01800C04, 11)                       // notifies service routine completed
}

#pragma interrupt                                     // interrupt service routine #12
void UART0_TX( )
{
    char count;

    for( count=0; count<4; count++ )
        _MEMIO_WR_32(0x01801008, count)               // write count value to TX FIFO

    _MEMIO_WR_8(0x01800C04, 12)                       // notifies service routine completed
}

```

Figure 6.3-1 UART test program

6.4 Timer

```

void timer_test( )
{
    /* set interrupt */

    asm( "clr 13 ; ";                               // disable interrupt
    asm( "set 12 ; ";                               // set vectored interrupt mod

```

```

    _MEMIO_WR_32(0x01800C04, 0x0100)           // interrupt vector high 3 bit = 001
    _MEMIO_WR_32(0x01800C08, 0x00000003)       // enable timer interrupt

    asm( "set 13 ;");                           // enable interrupt

    /* timer0 mode - periodic */

    _MEMIO_WR_32(0x01801404, 0x0063)           // set period 100
    _MEMIO_WR_32(0x01801400, 0x0903)           // set prescale value 10 and start operation

    /* timer1 mode - one shot */

    _MEMIO_WR_32(0x0180140C, 0x0031)           // set period 50
    _MEMIO_WR_32(0x01801408, 0x0e01)           // set prescale value 15 and start operation
}

```

Figure 6.4-1 Timer test program

6.5 Pulse Width Modulation (PWM)

```

void pwm_test( )
{
    /* set interrupt */

    asm( "clr 13 ;");                           // disable interrupt
    asm( "set 12 ;");                           // set vectored interrupt mod
}

```

```
_MEMIO_WR_32(0x01800C04, 0x0100)           // interrupt vector high 3 bit = 001
_MEMIO_WR_32(0x01800C08, 0x00800000)         // enable pwm interrupt

asm( "set 13 ; ");                             // enable interrupt

_MEMIO_WR_32(0x01801804, 0x0031)             // set duty 50
_MEMIO_WR_32(0x01801808, 0x0063)             // set period 100
_MEMIO_WR_32(0x0180180C, 0x0003)             // interrupt request every 4 period
_MEMIO_WR_32(0x01801800, 0x0901)             // set start level high and start operation
}
}
```

Figure 6.5-1 PWM test program

6.6 PPM

```
void ppm_test( )
{
    /* set interrupt */

    asm( "clr 13 ; ");                         // disable interrupt
    asm( "set 12 ; ");                         // set vectored interrupt mode
}
```

```

    _MEMIO_WR_32(0x01800018, 0x000000c0)           // PPM0, PPM1 포트 선택
    _MEMIO_WR_32(0x01800018, 0x00000018)

    _MEMIO_WR_32(0x01800C04, 0x0100)               // interrupt vector high 3 bit = 001
    _MEMIO_WR_32(0x01800C08, 0x0000000c)           // enable ppm interrupt

    asm( "set 13 ; ";                               // enable interrupt

    _MEMIO_WR_32(0x01801C00, 0x1303)               // count period 20, period (rising-to-rising)
    _MEMIO_WR_32(0x01801C08, 0x1d0f)               // count period 30, low pulse
}

#pragma interrupt                                   // interrupt service routine of PPM0
void PPM0()
{
    int pulse_width;

    pulse_width = peaks (0x01801C04);

    _MEMIO_WR_8(0x01800C04, 2)                     // notifies service routine completed
}

```

Figure 6.6-1 Timer test program

6.7 Video

6.7.1 TV-방식 setting

다음은 TV 방식을 setting 하는 예제 program 이다.

```

Main()
{
    asm( "TV방식-setting-NTSC: ");
}

```

```

MEMIO_WR_32 (0x01803400, MEMIO_RD_32 (0x01803400) & 0xff3f) // bit[7:6]=00

asm( "TV방식-setting-PAL-M: ");
MEMIO_WR_32 (0x01803400, MEMIO_RD_32 (0x01803400) & 0xff3f) // bit[7:6]=00

asm( "TV방식-setting-PAL-BDGHIN: ");
MEMIO_WR_32 (0x01803400, MEMIO_RD_32 (0x01803400) | 0x00c0) // bit[7:6]=11

asm( "TV방식-setting-PAL-Combi-N: ");
MEMIO_WR_32 (0x01803400, (MEMIO_RD_32 (0x01803400) | 0x0080) & 0xffbf) // bit[7:6]=10

asm( "Local-Mode-setting: ");
MEMIO_WR_32 (0x01803400, MEMIO_RD_32 (0x01803400) & 0xffcf) // bit[5:4]=00

asm( "Remote-Model-setting: ");
MEMIO_WR_32 (0x01803400, (MEMIO_RD_32 (0x01803400) | 0x0010) & 0xffdf) // bit[5:4]=01

asm( "Remote-Mode2-setting: ");
MEMIO_WR_32 (0x01803400, (MEMIO_RD_32 (0x01803400) | 0x0020) & 0xffef) // bit[5:4]=10

asm( "Remote-Mode3-setting: ");
MEMIO_WR_32 (0x01803400, MEMIO_RD_32 (0x01803400) | 0x0030) // bit[5:4]=11

asm( "vsync-width-0-setting: ");
MEMIO_WR_32 (0x01803400, MEMIO_RD_32 (0x01803400) & 0xffc0) // bit[1:0]=00 =>serration only
}

```

6.7.2 NTSC-local , 720 x 480 ,pg0-8bit color, make lut0, fill-with-white-color example

Vclk = 28.6363 이라고 가정함.

```

MAIN()
{
MEMIO_WR_32 (0x01803400,0x0000); //525,LOCAL,EXT-VSYNCHIN,SERR-ONLY
MEMIO_WR_32 (0x01803404,0x0001); //VCLK=28,6MHZ DCLK=VCLK/2=14.3MHZ
MEMIO_WR_32 (0x01803408, (INT)(4.7*14.31818) | (INT)(4.7*14.31818) <<8); //HW , HBP
MEMIO_WR_32 (0x0180340c,(SHORT)(52.6*14.31818)); //HDISP
MEMIO_WR_32 (0x01803410,(SHORT)(1.5*14.31818)); //HFP
MEMIO_WR_32 (0x01803414, 9 | (38<<8)); //10.6*DCLK/16=LOW-WIN //42.4*DCLK/16=UPPER-WIN
MEMIO_WR_32 (0x01803418,38); //VSYNC BACK PORCH
MEMIO_WR_32 (0x0180341c,239); //VDISP
MEMIO_WR_32 (0x01803c00,0x83); //PAGE0EN, COLOR=256C
MEMIO_WR_32 (0x01803c10,0x03); //PAGE1 COLOR=256C
MEMIO_WR_32 (0x01803c20,0x03); //PAGE2 COLOR=256C
MEMIO_WR_32 (0x01803c30,0x03); //PAGE3 COLOR=256C

MEMIO_WR_32 (0x01803c04,0x0ffc); //PAGE0 LUT-START=0xFFC0000
MEMIO_WR_32 (0x01803c14,0x0ffd); //PAGE1 LUT-START=0xFFD0000
MEMIO_WR_32 (0x01803c24,0x0ffe); //PAGE2 LUT-START=0xFFE0000
MEMIO_WR_32 (0x01803c34,0x0fff); //PAGE3 LUT-START=0xFFF0000
MEMIO_WR_32 (0x01804000,0x0104); //PLL RAMDAC ARE ALL ENABLE
MEMIO_WR_32 (0x01804004,(62 <<8) | 0x21); PLLCLK=50MHZ
//256 COLOR SETTING GRAY LEVEL
FOR(I=0;I<256;I++) //GRAY LEVEL
{
MEMIO_WR_32 (0x01804400 + I*4, (I<<16) | (I<<8) | I); // PALETTE0 FOR 256C
MEMIO_WR_32 (0x01804800 + I*4, (I<<16) | (I<<8) | I); // PALETTE0 FOR 256C
}
}

```

```

}
MEMIO_WR_32 (0x01804000,0x0004);
MAKELUTTABLEFORPAGE0(0x65,0,1,0);
FOR(I=0;I<720*640/4;I++)
{
    MEMIO_WR_32 (0x03000000 + 4*I, 0xFFFFFFFF); //FILL WITH WHITE COLOR
}
WHILE(1); //FOR EVER LOOP
} //END OF MAIN()

VOID MAKELUTTABLEFORPAGE0(INT PGLBP,INT PGVRAMS,INT PGLNEN,INT PGPAN)
{
    INT HRESOL;
    INT LINESTART;
    INT HLINESIZE;
    INT I,LBP,EN;
    LBP=PGLBP<<22;
    IF(PGLNEN)EN=0x04; ELSE EN=0;
    HRESOL = _MEMIO_RD_32 (0x0180340C) & 0x03FF ;
    HLINESIZE = HRESOL  + 31;
    HLINESIZE = HLINESIZE & 0x000FE0;
    FOR(I=0;I<480;I++)
    {
        LINESTART=(PGVRAMS + I * HLINESIZE)>>2;
        _MEMIO_WR_32 (0x03FFC000 + I*4 , LBP | LINESTART | EN | PGPAN);
    }
} // END OF VOID MAKELUTTABLEFORPAGE0()

```

6.8 Bitblt & Sprite Programming

Bit block Transfer 및 Sprite function의 programming은 Graphic controller register를 이용한다. 모든 register는 memory mapped IO로서 port가 할당 되어있음에 따라 memory write 형식으로 register를 setting 할 수 있다. 다음 예와 같이 해당 parameter register를 set한 후 Bitblt 또는 Sprite active bit을 마지막으로 set 한다.

6.8.1 Bit Block Transfer

```

asm( "Graphic_Reg_SET: ");

MEMIO_WR_32 (0x01803800, 0x30201000) // Page 0 Start Address 0x0300_0000
MEMIO_WR_32 (0x01803804, 0x00000123) // Horizontal 640 pixel
MEMIO_WR_32 (0x01803808, 0x00000066) // ROPV Xor = 66
MEMIO_WR_32 (0x0180380c, 0x0f0f0f0f) // Pattern Data

MEMIO_WR_32 (0x01803810, 0x00570010) // SRC XY (87,16)
MEMIO_WR_32 (0x01803814, 0x00080028) // DST XY (8,40)
MEMIO_WR_32 (0x01803818, 0x0019001e) // WID, HIG (25,30)
MEMIO_WR_32 (0x0180381c, 0x00000001) // BITBLT Page 0

asm("halt 1");

MEMIO_WR_32 (0x01803810, 0x00620006) // SRC XY (98, 6)
MEMIO_WR_32 (0x01803814, 0x0030001e) // DST XY (48,30)
MEMIO_WR_32 (0x01803818, 0x000a004d) // WID, HIG (10,77)
MEMIO_WR_32 (0x0180381c, 0x00000002) // BITBLT Page 1

asm("halt 1");

MEMIO_WR_32 (0x01803808, 0x000000ee) // (Src | Dst)
MEMIO_WR_32 (0x01803810, 0x003c0064) // SRC XY (60,100)
MEMIO_WR_32 (0x01803814, 0x002a0003) // DST XY (42, 3)
MEMIO_WR_32 (0x01803818, 0x000c0008) // WID, HIG (12, 8)
MEMIO_WR_32 (0x0180381c, 0x00000004) // BITBLT Page 2

asm("halt 1");

```

640 Pixel 수평해상도를 갖는 경우에 대하여 Bitblt 를 수행하기 위한 Program 예이다. Program 중간의 “halt 1”의 의미는 Bitblt가 연속적으로 수행되어야 할 경우, 이전 Bitblt function이 끝난 후 다음 명령이 들어와야 하므로 bitblt의 완료에 따른 interrupt 발생때까지 다음 program의 진행을 hold하는 의미이다. 실제 halt 대신 bitblt와 무관한 programming을 계속할 수 있으며, CPU는 bitblt 수행과 별도로 program을 수행한다. (Bitblt, sprite는 CPU와는 무관하게 자체 엔진으로 수행하므로) 그리고 User는 bitblt control register(BITBLT)를 읽어봄으로써 bitblt가 수행을 완료했는지 알 수 있다. 완료가 확인된 후 다음 bitblt 명령을 수행 시킬 수 있다.

Screen에 보여지는 수평 해상도가 640 dot 일지라도 memory 상에 지정한 수평해상도는 800 또는 1024dot로 크게 programming하여 off-screen 영역을 활용할 수 있다. 이러한 경우 0x01803804 HRES register에 set해야 할 값은 보여지는 수평해상도가 아닌, 메모리 상에 할당된 size를 의미한다. 또한 LUT에 지정하는 Line start address는 모든 line이 linear 하게 이어질 수 있도록 program되어야 한다.

6.8.2 SPRITE

```

asm( "Sprite_Reg_SET: ");

MEMIO_WR_32 (0x01803800, 0x30201000) // Page 0 Start Address 0x0300_0000
MEMIO_WR_32 (0x01803804, 0x00000123) // Horizontal 640 pixel

// Sprite 1 : Start addr 0x0315_0000

MEMIO_WR_32 (0x03150000, 0x000000cc) // ROPV Mov = cc
MEMIO_WR_32 (0x03150004, 0x0f0f0f0f) // PATD
MEMIO_WR_32 (0x03150008, 0x005e00de) // SRC XY (94,222)
MEMIO_WR_32 (0x0315000c, 0x0041000a) // DST XY (65,10)
MEMIO_WR_32 (0x03150010, 0x001e0010) // WID, HIG (30,16)
MEMIO_WR_32 (0x03150014, 0x00000001) // BITBLT
MEMIO_WR_32 (0x03150018, 0x00000000) // dummy
MEMIO_WR_32 (0x0315001c, 0x00000000) // dummy

// Sprite 2

MEMIO_WR_32 (0x03150020, 0x000000cc) // ROPV Mov = cc
MEMIO_WR_32 (0x03150024, 0x0f0f0f0f) // PATD
MEMIO_WR_32 (0x0315002c, 0x0041001b) // DST XY (65,27)
MEMIO_WR_32 (0x03150030, 0x001f0010) // WID, HIG (31,16)
MEMIO_WR_32 (0x03150034, 0x00000001) // BITBLT
MEMIO_WR_32 (0x03150038, 0x00000000) // dummy

// Sprite 3

MEMIO_WR_32 (0x03150040, 0x000000cc) // ROPV Mov = cc
MEMIO_WR_32 (0x03150044, 0x0f0f0f0f) // PATD
MEMIO_WR_32 (0x03150048, 0x005e00de) // SRC XY (94,222)
MEMIO_WR_32 (0x0315004c, 0x0041002c) // DST XY (65,44)
MEMIO_WR_32 (0x03150050, 0x00200010) // WID, HIG (32,16)
MEMIO_WR_32 (0x03150054, 0x00000001) // BITBLT
MEMIO_WR_32 (0x03150058, 0x00000000) // dummy
MEMIO_WR_32 (0x0315005c, 0x00000000) // dummy

// Sprite 4

MEMIO_WR_32 (0x03150060, 0x000000cc) // ROPV Mov = cc
MEMIO_WR_32 (0x03150064, 0x0f0f0f0f) // PATD
MEMIO_WR_32 (0x03150068, 0x005e00de) // SRC XY (94,222)
MEMIO_WR_32 (0x0315006c, 0x0041003d) // DST XY (65,61)
MEMIO_WR_32 (0x03150070, 0x00210010) // WID, HIG (33,16)
MEMIO_WR_32 (0x03150074, 0x00000001) // BITBLT
MEMIO_WR_32 (0x03150078, 0x00000000) // dummy
MEMIO_WR_32 (0x0315007c, 0x00000000) // dummy

// Sprite 5

MEMIO_WR_32 (0x03150080, 0x00000066) // ROPV Xor = 66
MEMIO_WR_32 (0x03150084, 0x0f0f0f0f) // PATD
MEMIO_WR_32 (0x03150088, 0x005e00de) // SRC XY (94,222)
MEMIO_WR_32 (0x0315008c, 0x0041004e) // DST XY (65,78)
MEMIO_WR_32 (0x03150090, 0x00220010) // WID, HIG (34,16)
MEMIO_WR_32 (0x03150094, 0x00000001) // BITBLT
MEMIO_WR_32 (0x03150098, 0x00000000) // dummy
MEMIO_WR_32 (0x0315009c, 0x00000000) // dummy

```

```
// Sprite 6

MEMIO_WR_32 (0x031500a0, 0x000000f0) // ROPV Pattern move = f0
MEMIO_WR_32 (0x031500a4, 0x33337777) // PATD
MEMIO_WR_32 (0x031500a8, 0x005e00de) // SRC XY (94,222)
MEMIO_WR_32 (0x031500ac, 0x0041005f) // DST XY (65,95)
MEMIO_WR_32 (0x031500b0, 0x00230010) // WID, HIG (35,16)
MEMIO_WR_32 (0x031500b4, 0x00000001) // BITBLT
MEMIO_WR_32 (0x031500b8, 0x55aa0000) // EOT
MEMIO_WR_32 (0x031500bc, 0x00000000) // dummy

// End of sprite data

MEMIO_WR_32 (0x01803820, 0x00150001) // SPRITE Active
```

6개의 Block image를 Sprite 기능을 이용하여 처리할 경우, 위의 예처럼 frame memory의 한 영역에 각 block에 대한 bitblt parameter를 32byte 단위로 저장해 놓는다. 마지막으로 전송되는 block의 parameter에는 Sprite의 끝을 알리는 EOT (0x55aa)가 7번째 dword의 상위 16bit에 반드시 들어가야 한다.

Memory에 sprite data가 모두 준비되어 있을 때, sprite control register에 sprite data가 위치한 시작 address와 함께 bit 0를 set 함으로써 block 전송이 시작된다.

6.9 RAMDAC & PLL

다음은 palette ram 을 작성하는 예제로 256 color, 16 color, 4 color, 2color 를 작성하는 예이다. PLLCLK 는 50Mhz 이다.

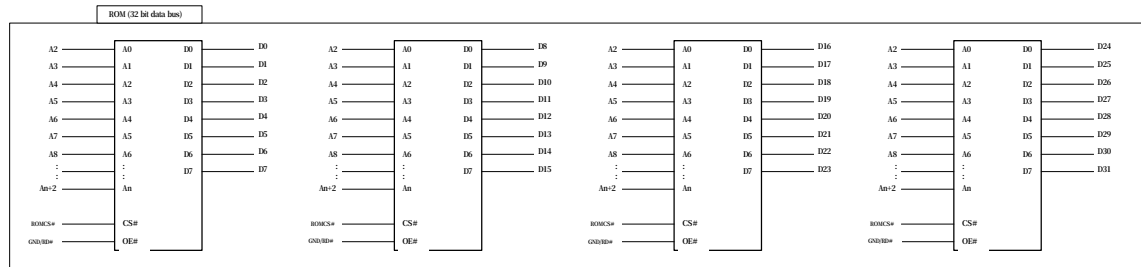
```
main()
{
MEMIO_WR_32 (0x01804000,0x0104); //PLL RAMDAC are all enable
MEMIO_WR_32 (0x01804004,(62 <<8 ) | 0x21); PLLCLK=50Mhz

//256 color setting gray level
for(i=0;i<256;i++) //gray level
{
MEMIO_WR_32 (0x01804400 + I*4, (i<<16) | (i<<8) | I); // palette0 for 256c
MEMIO_WR_32 (0x01804800 + I*4, (i<<16) | (i<<8) | I); // palette1 for 256c
}

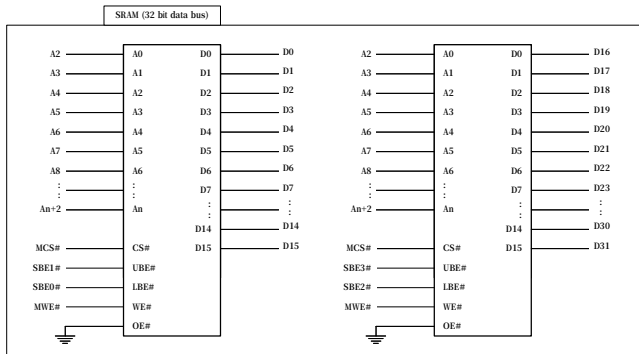
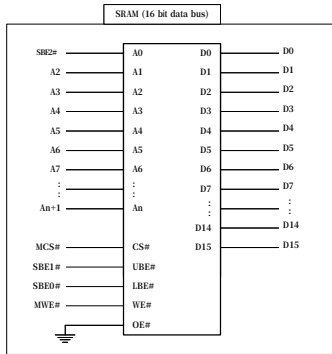
// 2 color
MEMIO_WR_32 (0x01804ba8,0); //black
MEMIO_WR_32 (0x01804bac,0xffffffff); //white
// 4 color
MEMIO_WR_32 (0x01804bb0,0); //black
MEMIO_WR_32 (0x01804bb4,0xff0000); //red
MEMIO_WR_32 (0x01804bb8,0x00ff00); //green
MEMIO_WR_32 (0x01804bbc,0x0000ff); //blue

//16 color
for(i=0;i<16;i++) //gray level
{
MEMIO_WR_32 (0x01804bc0 + I*4, (i<<20) | (i<<12) | (i<<4));
}
MEMIO_WR_32 (0x01804000,0x0004);
} //end of main()
```

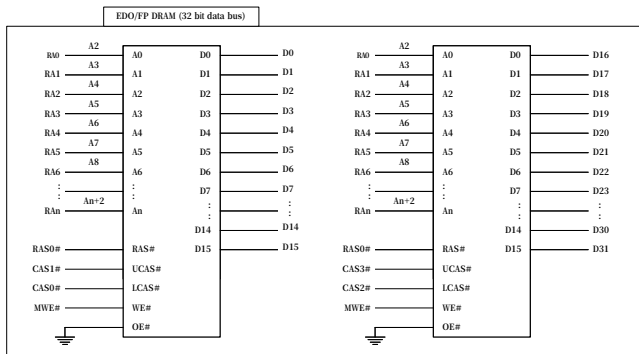
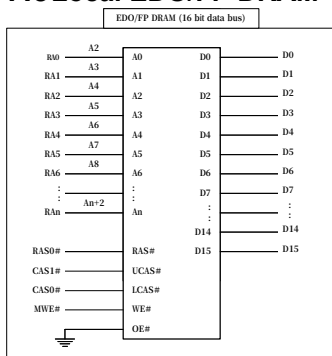

7.1 Local / Expansion ROM



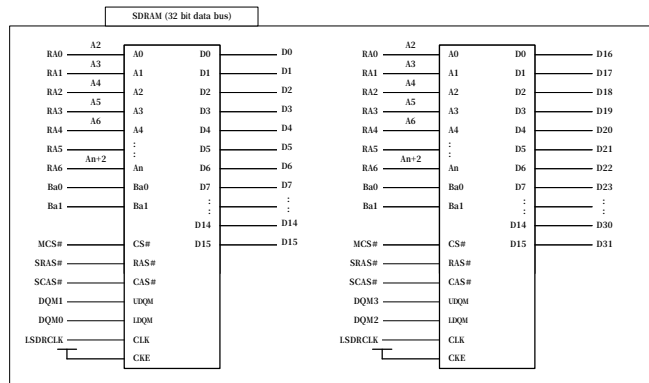
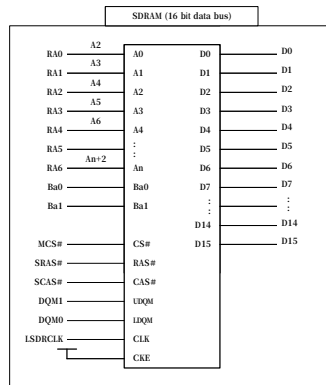
7.2 Local SRAM



7.3 Local EDO/FP DRAM

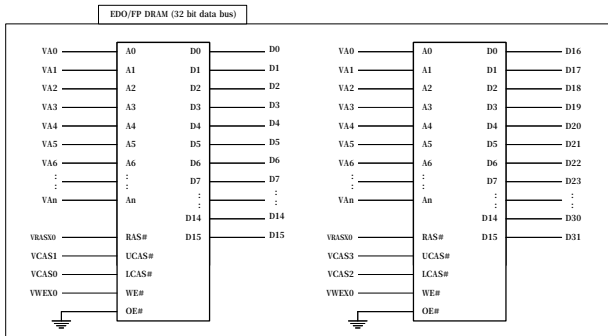
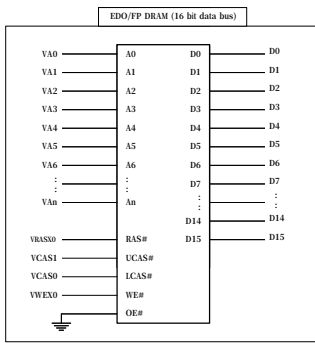


7.4 Local SDRAM

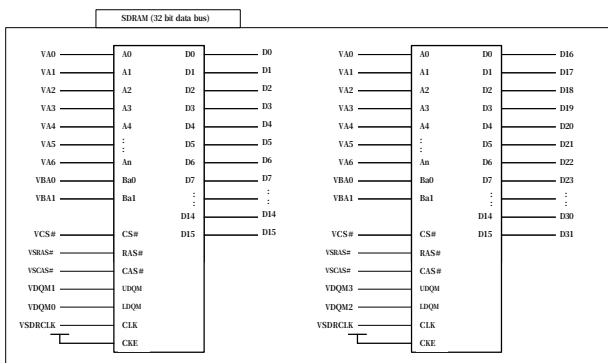
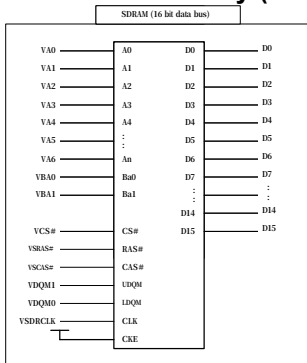


Frame Memory (EDO/FP DRAM)

7.5 Frame Memory (EDO/FP DRAM)



7.6 Frame Memory (SDRAM)

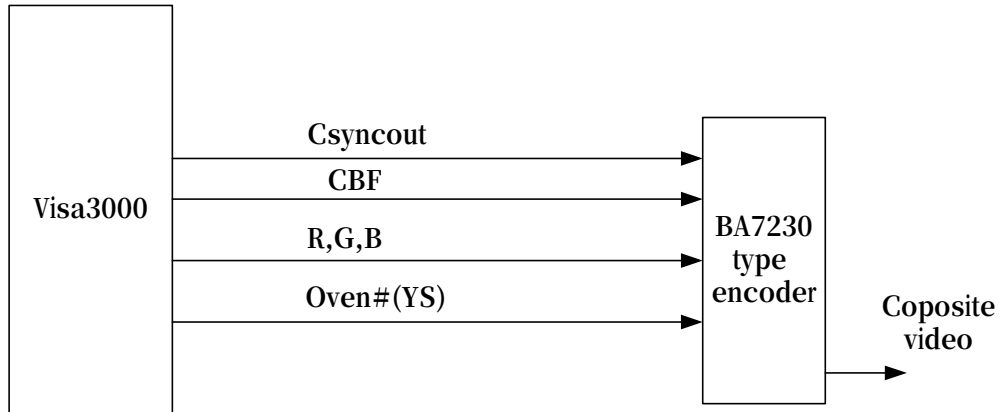


8 VIDEO APPLICATION

8.1 BA7230 Application

1. simple application

video clk 은 visa3000 의 xtal clk 14.3Mhz 사용 .

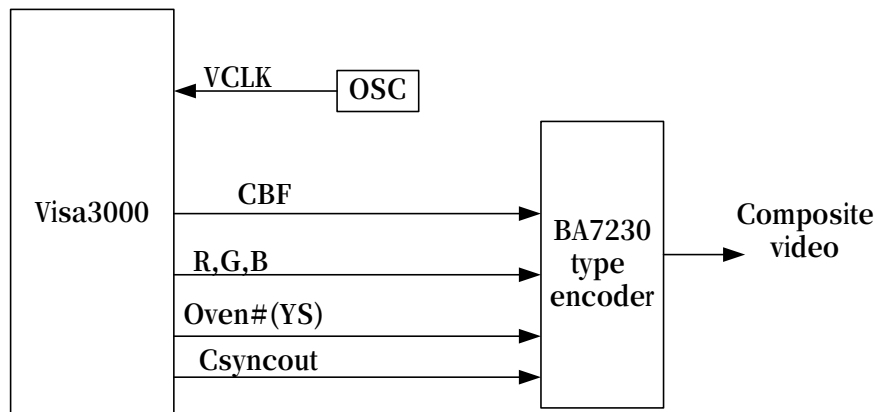


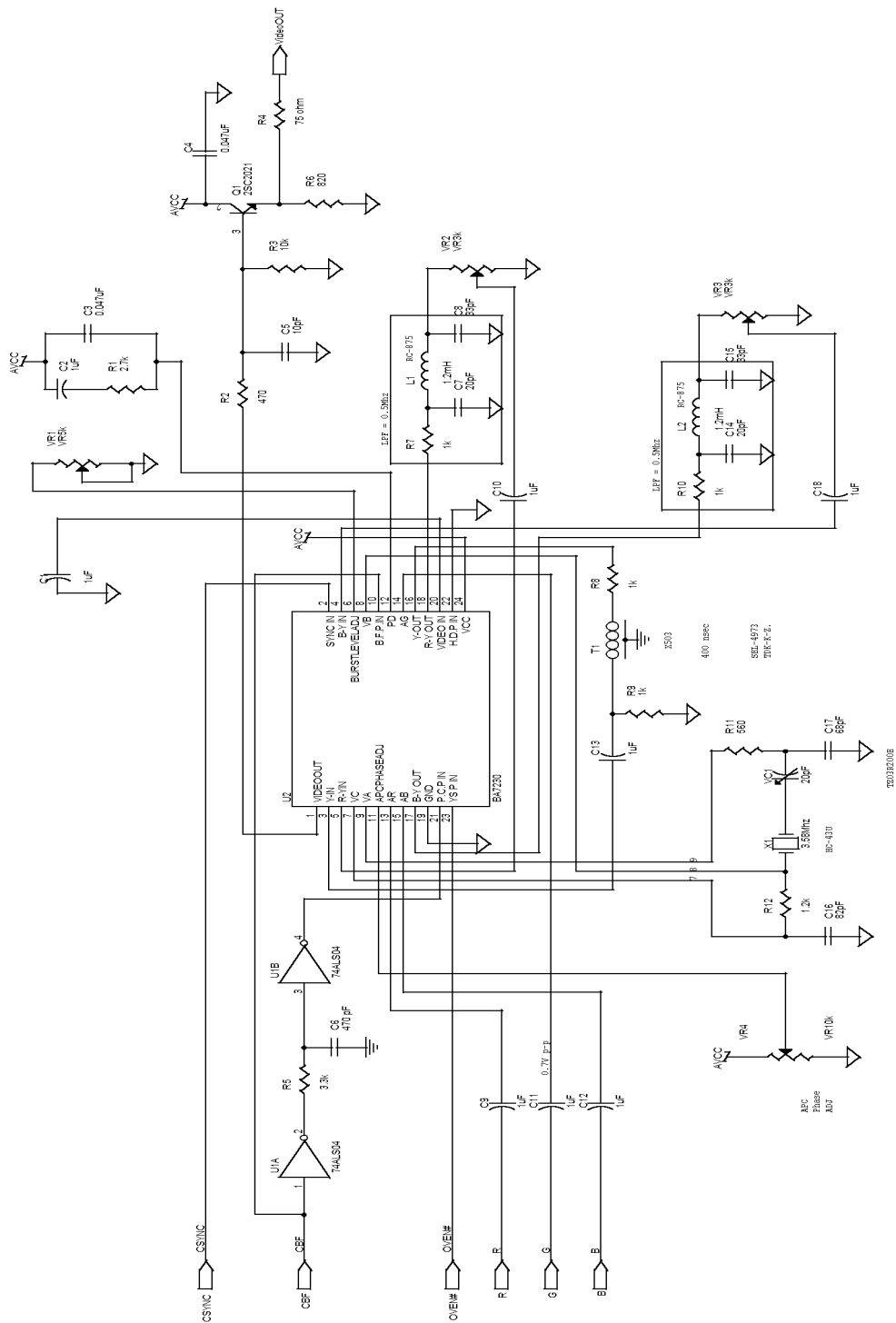
다음 회로에서 20번 핀을 cap 을 통해 ground 시킨다. local mode에서 BA7230의 외부 비디오 입력은 AC 성분이 없어야 color 가 흔들리지 않는다.

2. 외부의 osc를 사용하는 경우

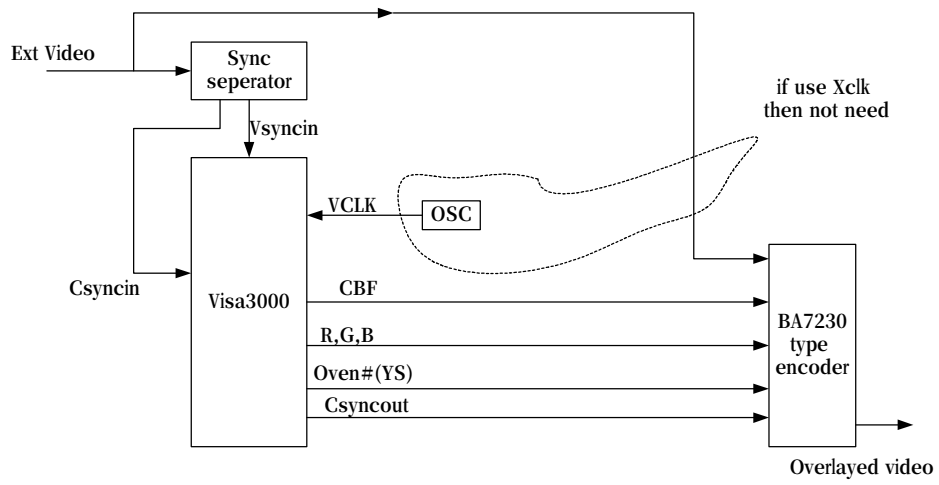
실제 구현 회로는 같고 osc 입력을 Visa 3000 의 vclk에 입력하면 된다.

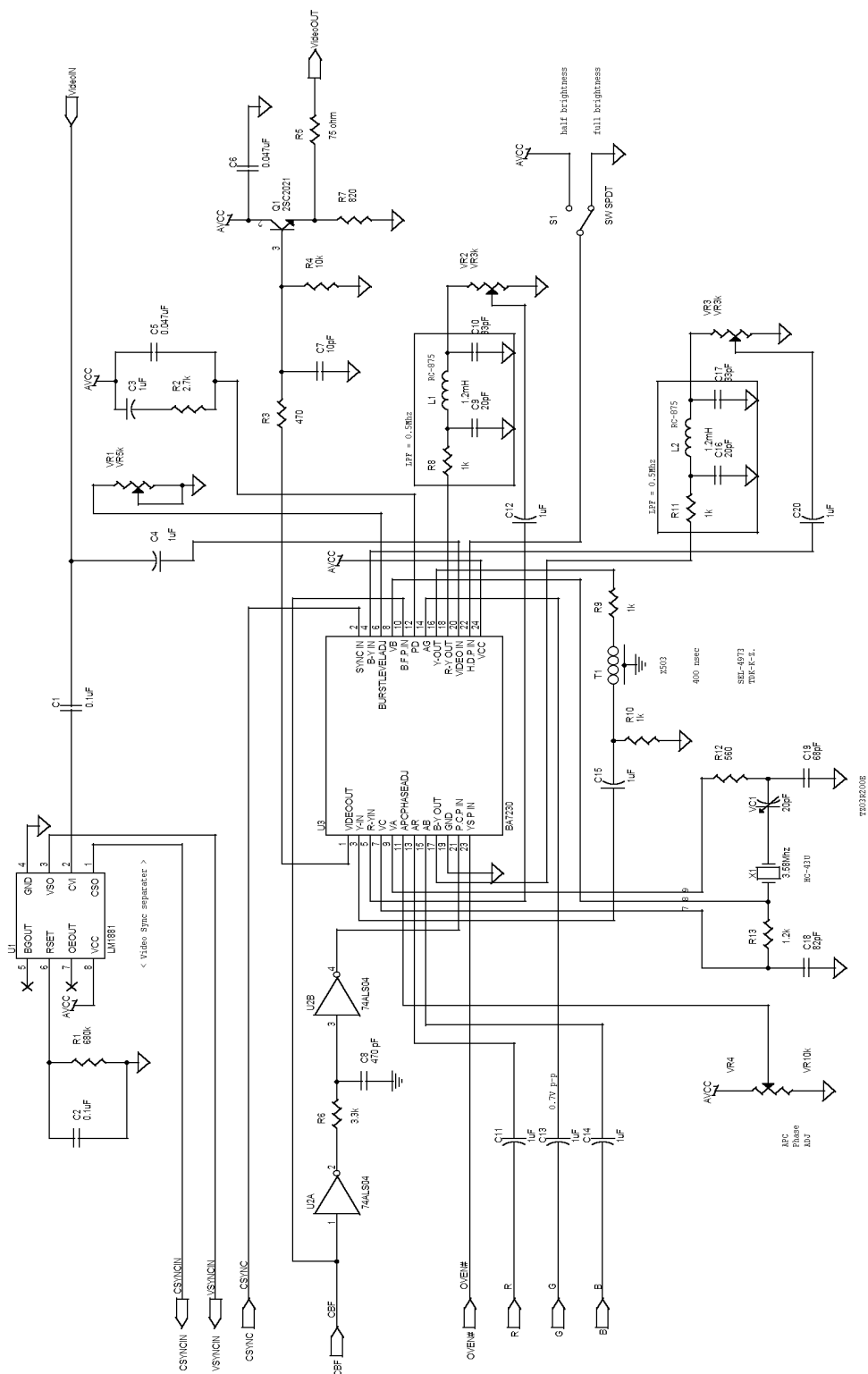
14.3 Mhz 이외의 clk 이 필요 할 때 사용.



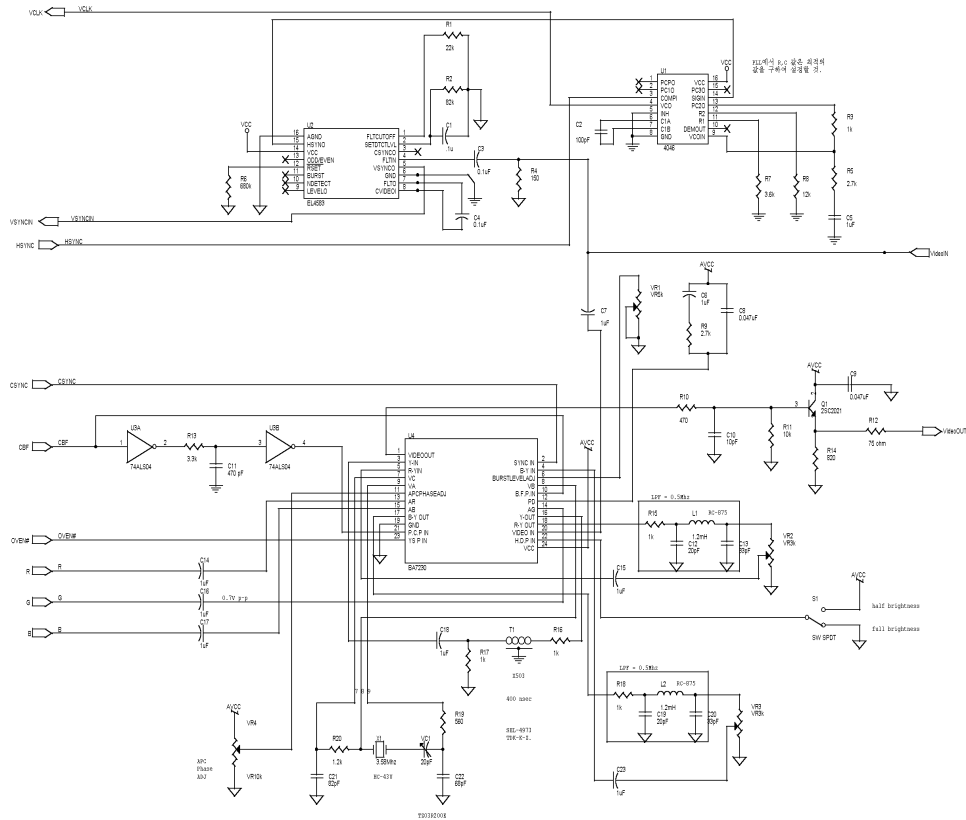
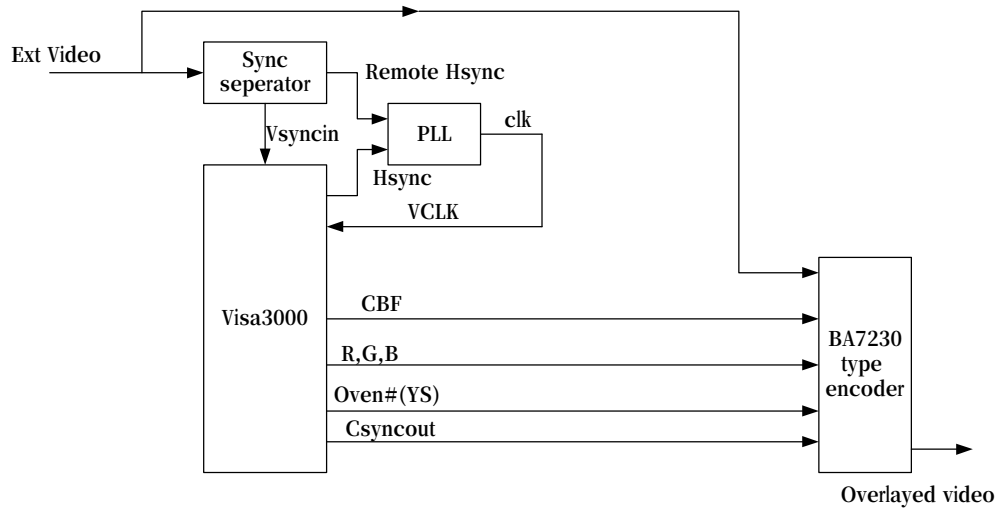


3. no PLL 을 이용한 Remote

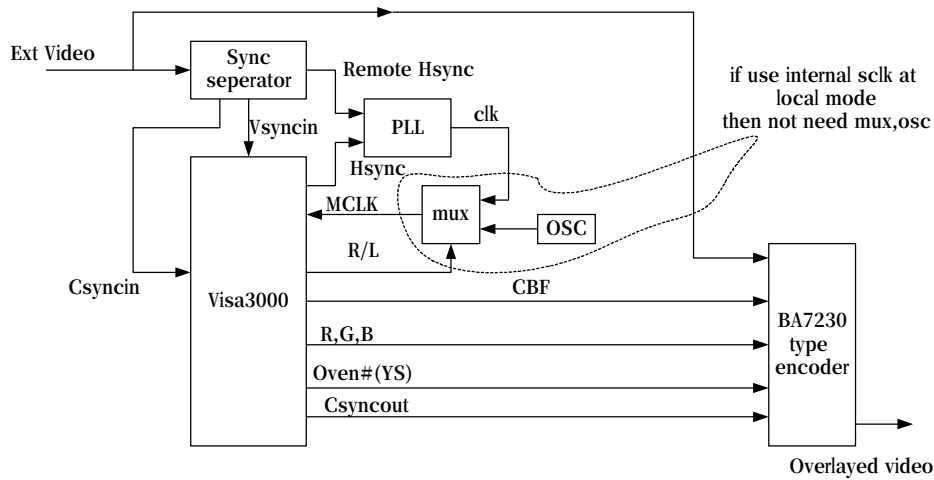




4. PLL 을 이용한 Remote



5. Local Remote를 모두 지원하는 경우



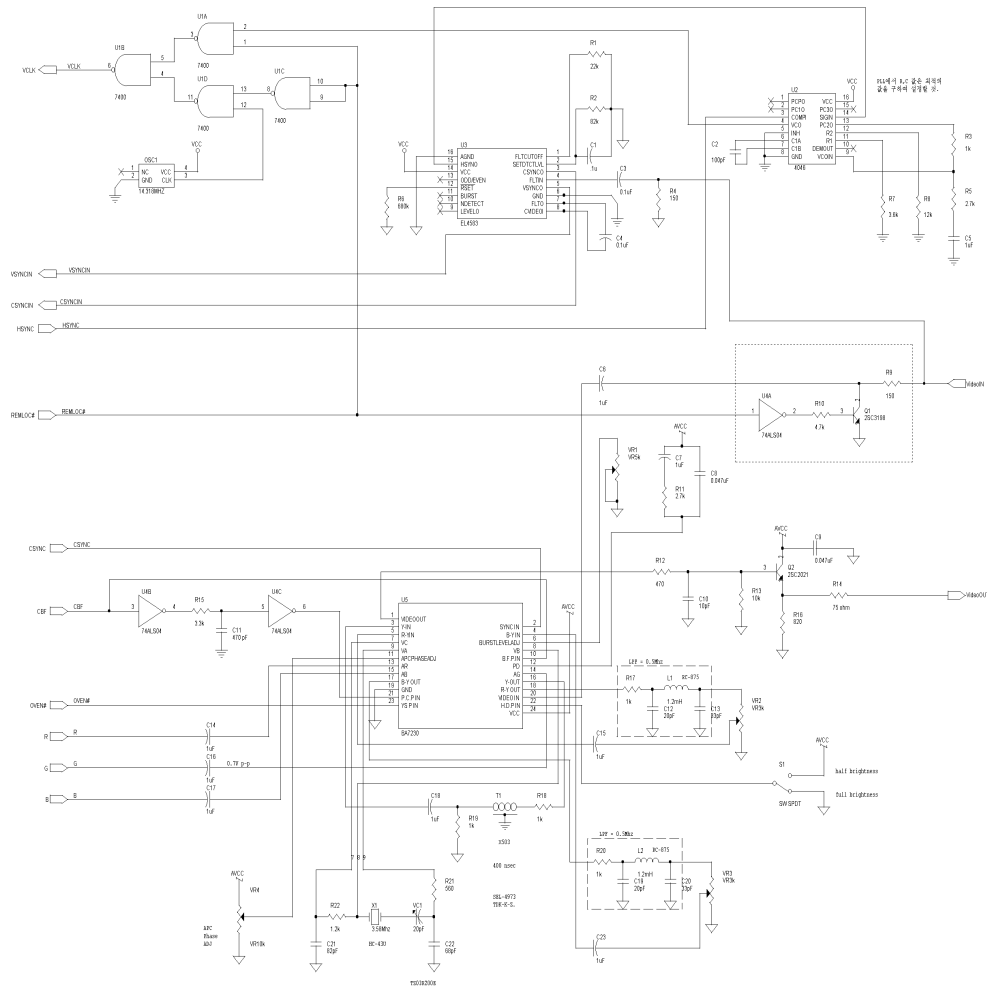
<5. BA7230 type interface PLL mode >

다음은 이것을 구현한 회로이다. 점선 안의 transistor 부분은 local mode에서 외부 비디오가 입력되는 것을 막기 위해서 이다. (local Mode에서 외부 외부비디오가 입력되면 color가 흔들린다.)

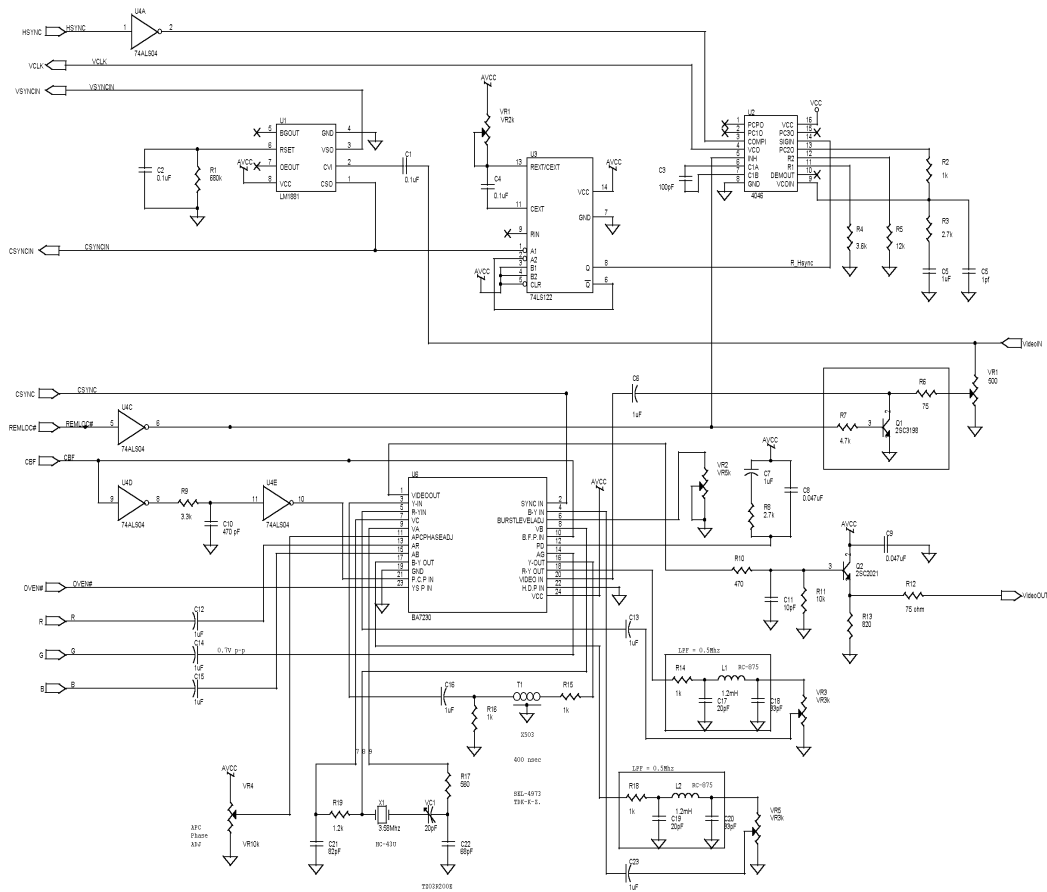
* ba7230 tuning 법

1. 가장 먼저 6번 핀에 연결된 가변저항을 맞춘다. Local mode에서 color burst 진폭의 크기를 맞추는 것으로 remote 입력과 같게 맞추거나 표준에 맞게 맞춘다. (local mode에서 실행)
2. 11번 핀에 연결된 가변저항을 중간 정도에 놓고 3.58 crystal 옆의 trimmer (가변 capacitor)를 맞춘다. 이 때 돌리다가 칼라가 고정되는 순간에 아주 조금 더 돌릴 것. (remote mode로 된 상태에서 실행할 것)
3. 11번 핀에 연결된 가변저항을 맞춘다. 아직 색은 원하는 색이 아닐 것이다. 2번에서는 color가 고정되게만 하였다. 이제 11번 핀에 연결된 가변저항을 돌려 원하는 색으로 맞춘다. (remote mode로 된 상태에서 실행할 것) local mode에서는 어느 위치에 놓아도 원하는 색이 나온다. 즉 이는 local mode 와 remote mode에서 같은 색이 나오게 맞추는 것이다.

참고 : 1,2,3 tuning 할 때 local , remote mode 전환이 쉽게 프로그래밍하고 overlay 되는 local sample image 는 color bar 나 R,G,B bar 등으로 하면 편하다.



Lm1881 사용 시



program example

crt0.S 는 assembler 이고

ba7230.c 는 C로 되어 있다


```

##=====
##      crt0.S
##      SE3208 startup code
##=====
## Copyright (c) 1995, 1996, 1997, 1998 Cygnus Solutions
## The authors hereby grant permission to use, copy, modify, distribute,
## and license this software and its documentation for any purpose, provided
## that existing copyright notices are retained in all copies and that this
## notice is included verbatim in any distributions. No written agreement,
## license, or royalty fee is required for any of the authorized uses.
## Modifications to this software may be copyrighted by their authors
## and need not follow the licensing terms described here, provided that
## the new terms are clearly indicated on the first page of each file where
## they apply.
##=====

        .file      "crt0.S"
        .section .text
        .global    _start

_start:
        ldi        _stack-8, %r3      # Load up the stack pointer and allocate ;stack=0x020f:ffff
        lea        (%r3), %sp        # our current frame.
#####
# PLL Register Set
#####
        ldi        0x01804000,        %R6    # RAMDAC & PLL Control Register
        ldi        0x00000004,        %R7
        st         %R7 ,    (%R6,0)

        ldi        0x01804004,        %R6    # PLL Program Register
        ldi        0x00003e21,        %R7    # 50Mhz
        st         %R7 ,    (%R6,0)
#####
# PIN MUX SET
#####
        ldi        0x01800018,        %R6    # Pin Mux Control Register 1
        ldi        0x00000000,        %R7
        st         %R7 ,    (%R6,0)

        ldi        0x0180001C,        %R6    # Pin Mux Control Register 2
        ldi        0x00000000,        %R7
        st         %R7 ,    (%R6,0)
#####
# LOCAL ROM CFG
#####
        ldi        0x01800400,        %R6    # Local DRAM Control Register
        ldi        0x00000087,        %R7
        st         %R7 ,    (%R6,0)
#####
# LOCAL RAM CFG
#####
# EDO DRAM
        ldi        0x01800408,        %R6    # Local DRAM Control Register
        ldi        0x001a0805,        %R7
        st         %R7 ,    (%R6,0)
#####
ROMDATA_COPY:
        ldi        _etext, %r3          # ROM has data at end of text; copy it. */
        ldi        _data_reload, %r1
        cmp        %r3, %r1
        jz         .L2                  # If no data, then do nothing
        ldi        _edata, %r4
        cmp        %r4, %r1
        jz         .L2
.L3:
        ldb        (%r3), %r2
        stb        %r2, (%r1)
        addi       %r3, 1, %r3
        addi       %r1, 1, %r1
        cmp        %r4, %r1
        jnz        .L3

```

```

.L2:
        call    _main          # Call main program
        .section _stack
_stack: .long    1

//////////
//file= ba7230.c

typedef unsigned int  dword;
typedef unsigned short word;
typedef unsigned char  byte;

byte *memb=(byte *) 0x00000000;
#define memw *(word*)&memb
#define meml *(int*)&memb
word *memW=(word *) 0x000000; /// special
dword *memL=(dword *) 0x000000; /// special

unsigned char *vdramb=(unsigned char *) 0x03000000;
#define vdramw *(word*)&vdramb
#define vdraml *(int*)&vdramb
word *vdramW=(word *) 0x03000000; /// special
int *vdramL=(int *) 0x03000000; /// special

byte *regb=(byte *) 0x01800000;
#define regw *(word*)&regb
#define regl *(int*)&regb

int *PLTT0 = (int *) 0x01804400; // palette0 for 256c
int *PLTT1 = (int *) 0x01804800; // palette1 for 256c
int *PLT1C = (int *) 0x01804ba8; // palette for 2c // address verify
int *PLT2C = (int *) 0x01804bb0; // palette for 4c
int *PLT4C = (int *) 0x01804bc0; // palette for 16c

int *LUT0=(int *) 0x03ffc000;
int *LUT1=(int *) 0x03ffd000;
int *LUT2=(int *) 0x03ffe000;
int *LUT3=(int *) 0x03fff000; // ~ 0xc04000L

#define VCLK 14181818
#define XCLK 14318181

int pg1HlineSize;
int pg2HlineSize;
int pg3HlineSize;
int pg4HlineSize;

int pg1luts;
int pg2luts;
int pg3luts;
int pg4luts;

int pg1VRAMs;
int pg2VRAMs;
int pg3VRAMs;
int pg4VRAMs;

int pg1Lbp;
int pg2Lbp;
int pg3Lbp;
int pg4Lbp;

```

```
char pg1LnEn;
char pg2LnEn;
char pg3LnEn;
char pg4LnEn;
```

```
int pg1Pan;
int pg2Pan;
int pg3Pan;
int pg4Pan;
```

```
#define __CACHE_INVALIDATE_SET(VALUE) \
asm( "CACHE_INVALIDATE_SET: "; \
asm( "push %R0"; \
asm( "ldi %0 , %%R0" :: "i" (VALUE) ); \
asm( "mvtc 0 , %CR5"; \
asm( "pop %R0";

#define __CACHE_INVALIDATE_CHECK() \
asm( "CACHE_INVALIDATE_CHECK: "; \
asm( "mvfc 0 , %CR6"; \
asm( "andi %R0, 0x00100000, %R0"; \
asm( "jnz __CACHE_INVALIDATE_CHECK";

#define __CACHE_MODE_SET(VALUE) \
asm( "CACHE_MODE_SET: "; \
asm( "push %R0"; \
asm( "ldi %0 , %%R0" :: "i" (VALUE) ); \
asm( "mvtc 0 , %CR3"; \
asm( "pop %R0";

#define __CACHE_ACTIVE_SET(VALUE) \
asm( "CACHE_ACTIVE_SET: "; \
asm( "push %R0"; \
asm( "ldi %0 , %%R0" :: "i" (VALUE) ); \
asm( "mvtc 0 , %CR7"; \
asm( "pop %R0";

// CACHE MASTER COMMAND REGISTER
# define __CACHE_UNIT_DISABLE ( 0 << 11 )
# define __CACHE_UNIT_ENABLE ( 1 << 11 )
// CACHE INVALIDATE REGISTER
# define __CACHE_INVALIDATE_DISABLE ( 0 << 0 )
# define __CACHE_INVALIDATE_ENABLE ( 1 << 0 )
// CACHE INVALIDATE REGISTER
# define __CACHE_WRITE_THROUGH ( 0 << 6 )
# define __CACHE_WRITE_BACK ( 1 << 6 )
```

```
void null(void){ return; }
```

```
int delay(int n)
{ int i;
for(i=0;i<n;i++) ;
return i;
}
```

```
extern void start();
```

```
#pragma interrupt
void TIMER0(){ memb[0x01800c04]=0; }
#pragma interrupt
void TIMER1(){ memb[0x01800c04]=1; }
#pragma interrupt
void PPM0(){ memb[0x01800c04]=2; }
#pragma interrupt
void PPM1(){ memb[0x01800c04]=3; }
#pragma interrupt
void DMA0(){ memb[0x01800c04]=4; }
#pragma interrupt
```

```

void DMA1() { memb[0x01800c04]=5;}
#pragma interrupt
void TIMER2() { memb[0x01800c04]=6;}
#pragma interrupt
void TIMER3() { memb[0x01800c04]=7;}
#pragma interrupt
void EXT_IRQ2() { memb[0x01800c04]=8;}
#pragma interrupt
void EXT_IRQ3() { memb[0x01800c04]=9;}
#pragma interrupt
void UART0_RXE() { memb[0x01800c04]=10;}
#pragma interrupt
void UART0_RX() { memb[0x01800c04]=11;}
#pragma interrupt
void UART0_TX() { memb[0x01800c04]=12;}
#pragma interrupt
void UART1_RXE() { memb[0x01800c04]=13;}
#pragma interrupt
void UART1_RX() { memb[0x01800c04]=14;}
#pragma interrupt
void UART1_TX() { memb[0x01800c04]=15;}
#pragma interrupt
void EXT_IRQ4() { memb[0x01800c04]=16;}
#pragma interrupt
void EXT_IRQ5() { memb[0x01800c04]=17;}
#pragma interrupt
void EXT_IRQ6() { memb[0x01800c04]=18;}
#pragma interrupt
void EXT_IRQ7() { memb[0x01800c04]=19;}
#pragma interrupt
void VEXT_SYNC() { memb[0x01800c04]=20;}
#pragma interrupt
void VSP() { memb[0x01800c04]=21;}
#pragma interrupt
void PPM_OVF() { memb[0x01800c04]=22;}
#pragma interrupt
void PWM() { memb[0x01800c04]=23;}
#pragma interrupt
void BITBLT() { memb[0x01800c04]=24;}
#pragma interrupt
void SPRITE() { memb[0x01800c04]=25;}
#pragma interrupt
void NMI() {}
#pragma interrupt
void SWI7() {}
#pragma interrupt
void AUTO_INT() {}

#define NOTUSEDISR (fp)(0)
typedef void (*fp)(void);
const fp HardwareVector[] __attribute__((section(".vects"))) = {
    /* Reset Vector */
    start , /* V00 : Reset Vector */
    NMI , /* V01 : NMI Vector */
    AUTO_INT , /* V02 : Interrupt Auto Vector */
    NOTUSEDISR, /* V03 : Double Falule Exception */
    NOTUSEDISR, /* V04 : Bus Error exception */
    NOTUSEDISR, /* V05 : Reserved */
    NOTUSEDISR, /* V06 : Reserved */
    NOTUSEDISR, /* V07 : Reserved */
    NOTUSEDISR, /* V08 : Co-processor Exception */
    NOTUSEDISR, /* V09 : Co-processor Exception */
    NOTUSEDISR, /* V0A : Co-processor Exception */
    NOTUSEDISR, /* V0B : Co-processor Exception */
    NOTUSEDISR, /* V0C : OSI Reset Vector */
    NOTUSEDISR, /* V0D : OSI break exception */
    NOTUSEDISR, /* V0E : Reserved */
    NOTUSEDISR, /* V0F : Reserved */
    NOTUSEDISR, /* V10 : SWI 00 Vector */
    NOTUSEDISR, /* V11 : SWI 01 Vector */
    NOTUSEDISR, /* V12 : SWI 02 Vector */
    NOTUSEDISR, /* V13 : SWI 03 Vector */
    NOTUSEDISR, /* V14 : SWI 04 Vector */
    NOTUSEDISR, /* V15 : SWI 05 Vector */
    NOTUSEDISR, /* V16 : SWI 06 Vector */

```

```

    SWI7      , /* V17 : SWI 07 Vector      */
    NOTUSEDISR, /* V18 : SWI 08 Vector      */
    NOTUSEDISR, /* V19 : SWI 09 Vector      */
    NOTUSEDISR, /* V1A : SWI 0A Vector      */
    NOTUSEDISR, /* V1B : SWI 0B Vector      */
    NOTUSEDISR, /* V1C : SWI 0C Vector      */
    NOTUSEDISR, /* V1D : SWI 0D Vector      */
    NOTUSEDISR, /* V1E : SWI 0E Vector      */
    NOTUSEDISR, /* V1F : SWI 0F Vector      */

    TIMER0    , /* V20 : User 00 Vector      */
    TIMER1    , /* V21 : User 01 Vector      */
    PPM0      , /* V22 : User 02 Vector      */
    PPM1      , /* V23 : User 03 Vector      */
    DMA0      , /* V24 : User 04 Vector      */
    DMA1      , /* V25 : User 05 Vector      */
    TIMER2    , /* V26 : User 06 Vector      */
    TIMER3    , /* V27 : User 07 Vector      */
    EXT_IRQ2   , /* V28 : User 08 Vector      */
    EXT_IRQ3   , /* V29 : User 09 Vector      */
    UART0_RXE , /* V2A : User 0A Vector      */
    UART0_RX   , /* V2B : User 0B Vector      */
    UART0_TX   , /* V2C : User 0C Vector      */
    UART1_RXE , /* V2D : User 0D Vector      */
    UART1_RX   , /* V2E : User 0E Vector      */
    UART1_TX   , /* V2F : User 0F Vector      */

    EXT_IRQ4   , /* V30 : User 10 Vector      */
    EXT_IRQ5   , /* V31 : User 11 Vector      */
    EXT_IRQ6   , /* V32 : User 12 Vector      */
    EXT_IRQ7   , /* V33 : User 13 Vector      */
    VEXT_SYNC  , /* V34 : User 14 Vector      */
    VSP        , /* V35 : User 15 Vector      */
    PPM_OVF    , /* V36 : User 16 Vector      */
    PWM        , /* V37 : User 17 Vector      */
    BITBLT     , /* V38 : User 18 Vector      */
    SPRITE     , /* V39 : User 19 Vector      */
    NOTUSEDISR, /* V3A : User 1A Vector      */
    NOTUSEDISR, /* V3B : User 1B Vector      */
    NOTUSEDISR, /* V3C : User 1C Vector      */
    NOTUSEDISR, /* V3D : User 1D Vector      */
    NOTUSEDISR, /* V3E : User 1E Vector      */
    NOTUSEDISR, /* V3F : User 1F Vector      */

    NOTUSEDISR, /* V40 : User 20 Vector      */
    NOTUSEDISR, /* V41 : User 21 Vector      */
    NOTUSEDISR, /* V42 : User 22 Vector      */
    NOTUSEDISR, /* V43 : User 23 Vector      */
    NOTUSEDISR, /* V44 : User 24 Vector      */
    NOTUSEDISR, /* V45 : User 25 Vector      */
    NOTUSEDISR, /* V46 : User 26 Vector      */
    NOTUSEDISR, /* V47 : User 27 Vector      */
    NOTUSEDISR, /* V48 : User 28 Vector      */
    NOTUSEDISR, /* V49 : User 29 Vector      */
    NOTUSEDISR, /* V4A : User 2A Vector      */
    NOTUSEDISR, /* V4B : User 2B Vector      */
    NOTUSEDISR, /* V4C : User 2C Vector      */
    NOTUSEDISR, /* V4D : User 2D Vector      */
    NOTUSEDISR, /* V4E : User 2E Vector      */
    NOTUSEDISR, /* V4F : User 2F Vector      */
};

int GetFieldR(int regaddr,int msb,int lsb)
{
    int tmp=0;int i;
    for(i=msb;i>=lsb;i--)
    {
        tmp=tmp*2;
        if((regl[regaddr]) & (1<<i)) tmp=tmp+1;
    }
    return tmp;
}

void SetFieldR(int regaddr,int msb,int lsb, int value)

```

```

{
int tmp;int i;
tmp=regl[regaddr];
for(i=lsb;i<=msb;i++)
{
    if(value & (1<<(i-lsb)))
    {
        tmp=tmp | (1<<i);
    }
    else
    {
        tmp=tmp & ( (1<<i)^0x7ffffff);
    }
}
regl[regaddr]=tmp;
}

void PutPixel(int x,int y,int color)
{
int addr;
addr=((LUT0[y] & 0x3fff8)<<2) + (x<<1);
vdramw[addr]=color;
}

#define abs(x) ( (x)>0 ) ? (x) : (-(x))
#define two(x) ( (x)<<1 )
void Line(int x1,int y1,int x2,int y2,int color) //bresen ham algorithm
{
int x, y, count, dx, dy, Xinc, Yinc, err, twodx;
x = x1; y = y1;
count = dx = abs ( x2 - x1 );
twodx = two ( dx );
dy = abs ( y2 - y1 );
Xinc = ( x2 > x1 ) ? (1) : (-1);
Yinc = ( y2 > y1 ) ? (1) : (-1);
err = 0;
while ( count-- >= 0 )
{
    if ( err < dx )
        err += dy;
    else
    {
        y += Yinc;
        err += dy - twodx;
    }
    PutPixel(x,y,color);
    while ( (y != y2) && (err > twodx) )
    {
        y += Yinc;
        err -= twodx;
        PutPixel(x,y,color);
    }
    x += Xinc;
    err += dx;
}
}

void Box(int x1,int y1,int x2,int y2,int color)
{
Line(x1,y1,x2,y1,color);
Line(x1,y1,x1,y2,color);
Line(x2,y1,x2,y2,color);
Line(x1,y2,x2,y2,color);
}

void Bar(int x1,int y1,int x2,int y2,int color)
{
int X1,Y1,X2,Y2,i,j;
X1=x1; X2=x2; Y1=y1; Y2=y2;
if(x1>x2)
{

```

```

        X1=x2;
        X2=x1;
    }
    if(y1>y2)
    {
        Y1=y2;
        Y2=y1;
    }
    for(j=Y1;j<=Y2;j++)for(i=X1;i<=X2;i++)PutPixel(i,j,color);
}

void MakeLut(int LutStartAddr,int LutNumb,int HlineSize,int pgLbp,int PgVramStartAddr,int pgLnEn,int pgPan)
{
    int i,LutValue;
    LutValue=(pgLbp<<22)|(pgLnEn<<2)|pgPan;
    LutStartAddr>=2;
    for(i=0;i<LutNumb;i++)
    {
        vdrAmL[LutStartAddr+i]=LutValue | ((PgVramStartAddr + i * HlineSize)>>2);
    }
}

void DefaultSetting(void)
{int i;

    pg1luts=0xffc000;
    pg2luts=0xffd000;
    pg3luts=0xffe000;
    pg4luts=0xff0000;

    pg1HlineSize=2048;
    pg2HlineSize=2048;
    pg3HlineSize=2048;
    pg4HlineSize=2048;

    pg1VRAMs=0x000000;
    pg2VRAMs=0x100000;
    pg3VRAMs=0x200000;
    pg4VRAMs=0x300000;

    pg1Lbp=0x60;
    pg2Lbp=0x60;
    pg3Lbp=0x60;
    pg4Lbp=0x60;

    pg1LnEn=1;
    pg2LnEn=1;
    pg3LnEn=1;
    pg4LnEn=1;

    pg1Pan=0;
    pg2Pan=0;
    pg3Pan=0;
    pg4Pan=0;

    regl[0x3400]=0x0000;//NTSC,local,ext-vsyncin-pin
    regl[0x3404]=0x2e88;//xtal-sel,oven#-active-high,skew=6,vclk=2*14.3,cpclk=14.3,vclk-from-xclk, dclk=vclk/2=14.3Mhz
    regb[0x3408]=(short)(4.7*14.31818);//HW
    regb[0x3409]=(short)(4.7*14.31818);//HBP
    regw[0x340c]=(short)(52.6*14.31818);//HDISP
    regw[0x3410]=(short)(1.5*14.31818)//HFP
    regb[0x3414]=9//39; //10.6*dclk/16=low-win =9
    regb[0x3415]=38; //42.4*dclk/16=upper-win
    regw[0x3418]=20;//vsync back porch
    regw[0x341c]=239;//vdisp
    //pgen,color
    regb[0x3c00]=0x84;//page0en, color=64kc
    regb[0x3c10]=0x03;//page1 color=256c
    regb[0x3c20]=0x03;//page2 color=256c
    regb[0x3c30]=0x03;//page3 color=256c
    //lut start
    regw[0x3c04]=0x0ffc;//page0 lut-start=0xffc0000
    regw[0x3c14]=0x0ffd;//page1 lut-start=0xffd0000
    regw[0x3c24]=0x0ffe;//page2 lut-start=0xffe0000

```

```

regw[0x3c34]=0x0fff; //page3 lut-start=0xfff0000

regw[0x4000]=0x0104; //PLL RAMDAC are all enable

//256 color
for(i=0;i<256;i++) //gray level
{
    PLTT0[i]=(i<<16) | (i<<8) | i; // palette0 for 256c
    PLTT1[i]=(i<<16) | (i<<8) | i; // palette1 for 256c
}

// 2 color
PLT1C[0]=0;
PLT1C[1]=0xfffff;

// 4 color
PLT2C[0]=0;
//...
PLT2C[3]=0x0000ff;

//16 color
PLT4C[0]=0;
//...
PLT4C[15]=0xfffff;
regw[0x4000]=0x0004;

MakeLut(pg1luts,480,pg1HlineSize,pg1Lbp,pg1VRAMs,pg1LnEn,pg1Pan);
MakeLut(pg2luts,480,pg2HlineSize,pg2Lbp,pg2VRAMs,pg2LnEn,pg2Pan);
MakeLut(pg3luts,480,pg3HlineSize,pg3Lbp,pg3VRAMs,pg3LnEn,pg3Pan);
MakeLut(pg4luts,480,pg4HlineSize,pg4Lbp,pg4VRAMs,pg4LnEn,pg4Pan);
} //default setting

void BarD(void)
{
    int i;

    for(i=0;i<0x40000;i++) vdramL[i]=0; //erase page 0

    Box( 50,50,640,400,0xfffff);
    Box( 50+1,50+1,640-1,400-1,0xfffff);

    Bar( 60,100,140,300, (0x1f<<11) | (0x3f<<5) | 0x1f); //white
    Bar(140,100,220,300, (0x1f<<11) | (0x3f<<5) | 0x00); //yellow
    Bar(220,100,300,300, (0x00<<11) | (0x3f<<5) | 0x1f); //cyan
    Bar(300,100,380,300, (0x00<<11) | (0x3f<<5) | 0x00); //green
    Bar(380,100,460,300, (0x1f<<11) | (0x00<<5) | 0x1f); //magenta
    Bar(460,100,540,300, (0x1f<<11) | (0x00<<5) | 0x00); //red
    Bar(540,100,620,300, (0x00<<11) | (0x00<<5) | 0x1f); //blue
}

int main()
{
    asm("CACHE_CONFIGURATION_SET:");
    __CACHE_INVALIDATE_SET(__CACHE_INVALIDATE_ENABLE)
    __CACHE_INVALIDATE_CHECK()
    __CACHE_MODE_SET(__CACHE_WRITE_THROUGH)
    __CACHE_ACTIVE_SET(__CACHE_UNIT_ENABLE)

    regw[0x4000]=0x0004; //PLL set enable
    regl[0x4004]=(62<<8) | (8<<2) | 1; //clk2x=50Mhz
    regb[0x0400]=0x81; //rom speed
    regl[0x0008]=(2<<8) | (0<<6) | (0<<4) | 1; //erom speed
    regw[0x0408]=(0<<13)|(1<<11)|(0<<9)|(0<<8)|(0<<6)|(0<<4)|(1<<3)|(0<<2)|(1<<0); //local edo speed
    regl[0x3000]=0x00b6; //edo
    regw[0x3004]=(1<<10)|(1<<8)|(0<<6)|(0<<5)|(0<<3)|(0<<1)|(0<<0); //video ram speed

    DefaultSetting(); // video set

    BarD();

```



```
delay(10000000); //while(1); //current local mode use-inner-xclk
```

```
//BA7230 Remote mode By PLL.
```

```
regl[0x3400]=0x0020; //NTSC,remote,ext-vsyncin-pin
```

```
regl[0x3404]=0x2e80; //xtal-sel,oven#-active-high,skew=6,vclk=2*14.3,cpclk=14.3,vclk-from-pin, dclk=vclk/2=14.3Mhz
```

```
delay(10000000); while(1);
```

```
//BA7230 Local mode By use vclk-pin
```

```
regl[0x3400]=0x0000; //NTSC,local,ext-vsyncin-pin
```

```
regl[0x3404]=0x2e80; //xtal-sel,oven#-active-high,skew=6,vclk=2*14.3,cpclk=14.3,vclk-from-pin, dclk=vclk/2=14.3Mhz
```

```
delay(10000000); while(1);
```

```
//BA7230 Remote mode By no PLL.
```

```
regl[0x3400]=0x0014; //NTSC,remote,vsync-from-CsyntoVH
```

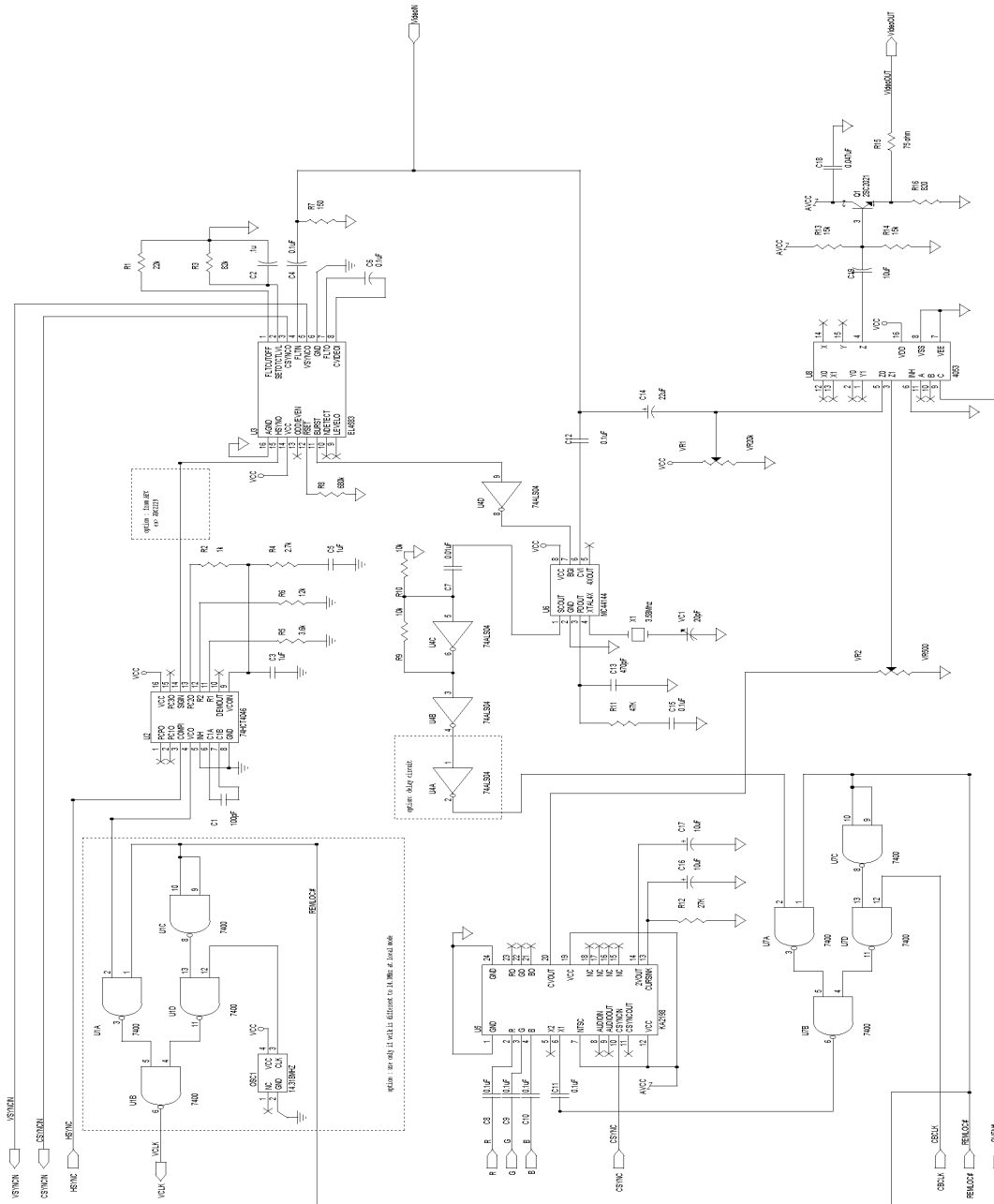
```
regl[0x3404]=0x2e88; //xtal-sel,oven#-active-high,skew=6,vclk=2*14.3,cpclk=14.3,vclk-from-xclk, dclk=vclk/2=14.3Mhz
```

```
while(1);
```

```
}
```

8.2 KA2198 Application

1. MC44144, 4053 interface



163

